# What Aeneas supports today: an evidence-based report

Aeneas (Rust → Lean) and its front-end Charon, tested on 2026-11-10

2026-11-10


- How to read this report
- 1. One-page summary
  - 1.1 Versions tested
  - 1.2 Bottom line on loops and iterators
  - 1.3 Feature status table
- 2. Loops and iterators in detail
  - 2.1 What the documentation says
  - 2.2 How a loop is translated (pipeline)
  - 2.3 Results for all loop and iterator experiments
  - 2.4 Selected cases: Rust input, Lean output, explanation
  - 2.5 What the user has to prove, and with which mechanism
  - 2.6 Iterator model coverage in the Lean library
- 3. Other features
  - 3.1 Traits and generics
  - 3.2 Closures and function pointers
  - 3.3 Borrows and lifetimes
  - 3.4 Arrays, slices, Vec, strings, Option/Result, Box
  - 3.5 Integer arithmetic, casts, overflow, panics
  - 3.6 Recursion and mutual recursion
  - 3.7 Unsafe code
  - 3.8 Interior mutability and reference counting
  - 3.9 Concurrency and async
  - 3.10 FFI, extern functions, opaque declarations
  - 3.11 Standard-library coverage (summary)
  - 3.12 Drops
- 4. Known open issues that affect users
  - 4.1 Loops
  - 4.2 Iterators
  - 4.3 Other
- 5. How to reproduce
- Appendix A — Loop and iterator experiments (full)
- Appendix B — Other feature experiments
- Appendix C — Loop proofs (checked with Lean 4.36.0-rc2, exit code 0)
  - L04, default mode (loop combinator, loop.spec_decr_nat)
  - L04 with -loops-to-rec (recursion + termination_by)
  - L30, for-over-range, default mode


## How to read this report

**Audience.** Engineers who know formal verification (e.g. the K framework) but have not used Aeneas.

**The question.** An LLM claimed that “Aeneas does not support loops and iterators”. This report checks that claim, and Aeneas’s other limits, against evidence.

**Short answer.** The claim is **false**, though it has a real core.

- **What works.** `while`, `loop`/`break`, `continue`, early `return`, nested loops, and `for` over ranges, slices, `Vec` and iterators all translate to Lean that type-checks. Loops that mutate through `&mut` and loops that return borrows also work.
- **Proofs work too.** I proved termination and functional correctness for two of the translated loops in Lean (§2.5).

- **What is missing:**
  - **Loop control flow:**
    - `break`/`continue` to an *outer* loop.
    - `return` inside a *nested* loop.
    - Some sequences of loops that each contain an early `return`.
    - Loops with no exit at all.
  - **Iterator adaptors.** Several common adaptors (`map`, `filter`, `fold`, `sum`, `any`, `count`) have no Lean model. Aeneas then emits `axiom`s, or Lean that does not type-check.

**Evidence labels.** Every claim carries one or more of these:

| Label | Meaning |
|---|---|
| **[doc]** | Stated in documentation in the repository at the commit tested. |
| **[test]** | Observed in an experiment run for this report (`experiments/` in this repository). |
| **[src]** | Read from source code at the commit tested (`file:line`). |
| **[issue]** | Taken from a GitHub issue or PR (number, state and dates as of 2026-11-10). |

**Path conventions.**

- `aeneas/…` paths refer to AeneasVerif/aeneas at commit `0d21a9e`.
- `charon/…` paths refer to AeneasVerif/charon at commit `454586e` (the commit Aeneas pins; see §1.1).
- Issue numbers without a prefix are Aeneas issues: `https://github.com/AeneasVerif/aeneas/issues/N` (PRs live at the same number under `/pull/N`). Charon issues are written `charon#N`.

**Rule followed.** I used no prior knowledge of Aeneas as a source. Where I could not check something, I say so.

# 1. One-page summary

## 1.1 Versions tested

| Component | Version / commit | Notes |
|---|---|---|
| Aeneas (upstream `main`) | `0d21a9ec7bbbb110110cacc14464a47fd71c8705`, 2026-11-09 | "Update to leanprover/lean4 v4.36.0 rc2 (#1547)". Built from source. `aeneas -version` prints `aeneas 0d21a9ec`. |
| Charon used by Aeneas | `454586e9b770231e66d2062993e632dfc06463e2`, 2026-11-08 | Pinned in `aeneas/charon-pin`. `charon version` prints `0.1.315`. Built from source with Rust `nightly-2026-09-17`. |
| Charon upstream `main` | `b9da4e3a108a5c2092fe80162bbd20ca1986d616`, 2026-11-09 | Cloned and recorded but **not used**: Aeneas requires the exact pinned Charon (`aeneas/README.md:53-63`, `aeneas/charon-pin`). |
| Lean | 4.36.0-rc2 | From `aeneas/backends/lean/lean-toolchain`. Mathlib `9d581df` (lake manifest). |
| OCaml | 5.3.0 | Isolated opam root inside the work dir. `visitors 20260520` and `ppx_deriving_hash 0.1.5` were pinned, as `charon/flake.nix:203-212` and `charon/nix/charon-ml.nix:13-26` require. |

**Local install found on this machine (not used).**

- A pre-existing Aeneas checkout is at `3a8586f` (2026-07-25). `aeneas -version` prints `3a8586f-dirty` because a local file is modified. That checkout pins Charon `527ea8e`.
- Its `bin/charon` is a dangling symlink to `/tmp/charon-pinned`.
- A pre-existing Charon checkout is at `909ff09` (2026-07-06) and its binary reports `0.1.220`.
- The local install is about 3.5 months behind upstream and its Charon link is broken. Everything was therefore tested against **upstream main**, built inside a dedicated work directory (the root of this repository). No existing repo on the machine was modified.

## 1.2 Bottom line on loops and iterators

The status labels used in the tables below:

- **Clean:** translates with no axioms and the Lean type-checks.
- **Caveats:** the Lean type-checks, but contains `axiom`s for missing library models, or `sorry`.
- **Fails:** Aeneas reports an error, or the generated Lean does not type-check.

38 loop/iterator programs were run **[test]**:

| Status | Count |
|---|---|
| Clean | 24 |
| Caveats | 7 |
| Fails | 7 |

Running them again with `-loops-to-rec` gave the same statuses.

## 1.3 Feature status table

| Feature | Status | Evidence |
|---|---|---|
| `while`, `loop` + `break`, `continue` | **Supported** | [test] L01–L03; [doc] `aeneas/documentation/aeneas-overview.md:135-191` |
| Early `return` inside a (single) loop | **Supported** (some shapes fail) | [test] L04, L27, L28 clean; L29 fails; [issue] #822 open, #1501 open |
| Nested loops (no cross-level jumps) | **Supported** | [test] L05 |
| `break`/`continue` to an outer loop, `return` in a nested loop | **Not supported** | [doc] `aeneas/README.md:128-131`; [src] `aeneas/src/PrePasses.ml:636-648`; [test] L05b/c/d |
| Loop with no exit (`loop {}`) | **Not supported** | [src] `aeneas/src/interp/InterpLoops.ml:409-412`; [test] L26 |
| `for` over a range (`..`, `..=`, `step_by`, `rev`) | **Supported** | [test] L06, L06b, L17, L30 |
| `for` over a slice / `&Vec` / `Vec` / `v.iter()` / `iter_mut()` / `into_iter()` | **Supported**; `for x in &vec` gives one axiom | [test] L07, L08, L08b, L08c, L10, L11, L12 |
| `for` over an array (`[T; N]`, `&[T; N]`) | **Caveats** (array `IntoIter` is an axiom) | [test] L09 |
| `enumerate`, `zip`, `rev`, `step_by`, `collect` (from a range) | **Supported** | [test] L15, L16, L17, L06b, L20 |
| `map`, `filter`, `fold`, `sum`, `any`, `count` | **Not modelled**: axioms; `map` + `collect` gives ill-typed Lean | [test] L13, L14, L18, L19, L21; [issue] #1043, #1510, #1542, #1544, #1552 (open) |
| User-defined `Iterator` impl in a `for` loop | **Supported** | [test] L25 |
| Loop mutating through `&mut`; loop returning a borrow | **Supported** | [test] L11, L11b, L22, L23 |
| Termination / correctness proofs of loops | **Supported**, extrinsic, by the user | [test] proofs in §2.5 (Lean-checked); [src] `aeneas/backends/lean/Aeneas/Std/WP.lean:757-797` |
| Traits, generics, associated types, const generics, `impl Trait`, supertraits | **Supported** | [test] T01–T03, T05–T08 |
| `dyn Trait` method calls | **Not supported** (body becomes `sorry`) | [test] T04, T04b; [issue] #1500 open |
| GATs | **Not supported** | [src] `aeneas/src/symbolic/SymbolicToPureTypes.ml:131`; `charon/…/expand_associated_types.rs:1145-1150`; [issue] #1112 open |
| Closures (`Fn`, `FnOnce`, captured env, returned `impl Fn`) | **Supported** | [test] C01, C02, C05, T07 |
| `FnMut` closures | **Fails** (ill-typed Lean) | [test] C03; [issue] #960 open |
| Function pointers | **Not supported** | [test] C04; [src] `aeneas/src/symbolic/SymbolicToPureTypes.ml:190,422` |
| `&mut &mut T`, structs holding `&mut`, returned borrows, two-phase borrows | **Supported** | [test] B01–B03, B05, B06 |
| Generic type instantiated with `&mut` (`id(&mut x)`) | **Fails** (no Aeneas error, ill-typed Lean) | [doc] `aeneas/README.md:132-135`; [test] B04 |
| Arrays, slices, `Box`, `Option`/`Result`, `?`, structs/enums/match | **Supported** (some std methods are axioms) | [test] D01, D02, D05–D07 |
| `Vec` | **Supported**; `pop`, `remove`, `vec![..]` partly axioms | [test] D03 |
| `String`/`&str` | **Caveats**: every operation tested is an axiom | [test] D04 |

| Feature | Status | Evidence |
|---|---|---|
| Integer arithmetic, overflow checks, wrapping/checked/overflowing/saturating, bit ops, `as` between integers, `bool as u8` | **Supported** | [test] A01, A02, A04 (`count_ones` is an axiom) |
| `u8 as char` | **Fails** (internal "Unreachable", broken Lean file) | [test] A03 |
| Floats | **Not supported** | [test] A06; [issue] #828 open |
| Panics, `assert!`, `unwrap`, `expect`, `unreachable!` | **Supported** (modelled as `fail`) | [test] A05 |
| Recursion, mutual recursion (`even`/`odd`) | **Supported** (`partial_fixpoint`) | [test] R01, R02 |
| Mutual recursion over a mutually inductive type | **Fails** in Lean (monotonicity) | [test] R03 |
| Raw-pointer creation or dereference, pointer→int casts | **Not supported** | [test] U01, U02, U05; [src] `aeneas/src/interp/InterpPaths.ml:191-197` |
| Unions | **Not supported** | [test] U03; [src] `aeneas/src/llbc/TypesAnalysis.ml:544` |
| `transmute` | Becomes an **axiom** (no error) | [test] U04 |
| `static mut` | **Not supported** (`static`/`const` are fine) | [test] U06 |
| `Cell`, `RefCell`, `Rc`, `Arc` | **Opaque axioms only**: types and methods are `axiom`s | [test] I01–I04; [issue] #1525 open |
| Threads, `Mutex`, atomics | **Opaque axioms only**; `thread::spawn` gives ill-typed Lean | [test] X01–X03; [doc] `aeneas/README.md:137-139` |
| `async` | **Not supported** | [test] X04; [src] `charon/src/bin/charon-driver/translate/translate_bodies.rs:1487` |
| `extern "C"` functions, `--opaque` modules | **Supported as axioms** (by design) | [test] F01, F02 |
| `HashMap`, `BTreeMap`, `VecDeque`, `format!`, `sort` | **Axioms only** (no models) | [test] S01, S04–S06 |
| `Drop` impls | **Silently ignored** (never called) | [test] S08; [src] `aeneas/src/Config.ml:625-627`; [issue] #1410 open |

**Important behaviour for tool users [test][src].** By default, an item that Aeneas cannot translate does not stop the run. Instead:

- An untranslatable body is emitted as a Lean `axiom` (e.g. L05b) or as `sorry` (e.g. T04).
- Aeneas exits with status 1, but the generated Lean may still type-check.
- A green `lake build` therefore does **not** mean every function was translated.

To make the first error fatal, pass `-abort-on-error`: `aeneas/src/Main.ml:148`, and the error helpers in `aeneas/src/Errors.ml:121-201`.

# 2. Loops and iterators in detail

## 2.1 What the documentation says

**README [doc].** `aeneas/README.md:126-131` lists one loop limitation:

> "**loops**: `return` inside *nested* loops, or `break`/`continue` to *outer* loops are not supported yet (e.g., `'a : loop { loop { break 'a; } }`). This is a technical limitation, not a fundamental issue..."

**Two translation modes [doc].** These are described in `aeneas/documentation/aeneas-overview.md:135-191` and `aeneas/documentation/tips-and-tricks.md:13-26`.

- **Default:** a `loop` fixed-point combinator over `ControlFlow`. Proofs use `loop.spec_decr_nat` or `loop.spec`.
- **`-loops-to-rec`:** a directly recursive `_loop` function marked `partial_fixpoint`. Proofs add `termination_by` and `decreasing_by`.
- The docs currently recommend `-loops-to-rec` for proof work. Quoting `tips-and-tricks.md:23-26`: "the proof infrastructure for it [the combinator] is not yet fully developed. Until it matures, **use `-loops-to-rec`**".

**Outdated user docs [doc].** `aeneas/docs/user/src/README.md:11-12` still says "loops: no nested loops for now" and "no functions pointers/closures". The experiments below contradict both statements for nested loops and for closures (function pointers do fail). That file is outdated.

**Charon's control-flow model [doc].** `charon/docs/limitations.md:31-33` says Charon does not distinguish `loop`, `while` and `for`; "only semantic information is kept".

- A `for` loop therefore reaches Aeneas as a `loop` that calls `Iterator::next` and matches on the result. This is visible in every `for` example below **[test]**.

## 2.2 How a loop is translated (pipeline)

Read from source **[src]**. The source-survey agent mapped these line references; I re-checked the ones marked ✓.

1. **Charon** turns the Rust MIR control-flow graph back into structured `loop`/`break`/`continue` statements.
   - Irreducible control flow is reported as an error (`charon/src/transform/control_flow/ullbc_to_llbc.rs:1362-1370`).
   - **Charon has no restriction on labelled breaks**: they are represented with a depth index.
2. **Aeneas pre-pass `update_loops`** (`aeneas/src/PrePasses.ml:392-680`). It rewrites a `return` inside a loop into a `break` whose exit path carries the return. It rejects the following:
   - ✓ `break` to an outer loop: "Breaks to outer loops are not supported yet" (`:637`).
   - ✓ `continue` to an outer loop: "Continue to outer loops are not supported yet" (`:641`).
   - ✓ `return` at loop depth > 1: "Returns inside of nested loops are not supported yet" (`:644-648`).
   - ✓ Some early-return shapes: "Early returns inside of loops are not supported yet" (`:571-576`).
3. **Symbolic execution with a fixed point** (`aeneas/src/interp/InterpLoopsFixedPoint.ml`, `InterpLoops.ml`).
   - The loop body is executed symbolically. The abstract states reaching `continue` are joined with the entry state until the state stops changing. ✓ This is capped at `loop_fixed_point_max_num_iters = 2` (`aeneas/src/Config.ml:182`).
   - Borrows that flow through the loop become "region abstractions". This is what lets loops that mutate through `&mut`, or return borrows, become pure functions that also return a "backward" function (see L11 and L23 below).
   - ✓ A loop with no `break` is rejected: "(Infinite) loops which do not contain breaks are not supported yet" (`aeneas/src/interp/InterpLoops.ml:409-412`).
4. **Pure-code passes** (`aeneas/src/pure/PureMicroPassesLoops.ml`).
   - By default the loop becomes a `loop` combinator application, with the body lifted into `<fn>_loop.body`.
   - The loop instead becomes a directly recursive function when `-loops-to-rec` is given, or when a heuristic holds. ✓ `loop_satisfies_recursive_criteria_aux` (`:2475`) uses `loop_rel_has_termination_measure` (`:2436`); per [issue] PR #779, this applies to loops over recursive data structures.

    - Observed **[test]**: L23 (a `while let` walking a linked list) was emitted as a recursive `partial_fixpoint` function even in default mode. All other loops used the combinator.
5. **Lean extraction.** `loop` is the library combinator. ✓ `aeneas/backends/lean/Aeneas/Std/Primitives.lean:452-457`:

```
def loop {α : Type u} {β : Type v} (body : α → Result (ControlFlow α β)) (x : α) : Result β := do
  bind (body x) fun r =>
  match r with
  | ControlFlow.cont x => loop body x
  | ControlFlow.done x => ok x
partial_fixpoint
```

The `Result` monad and its effects:

- ✓ `Result α` is an interaction tree with three outcomes: `ok`, `fail e` (panic, overflow, out-of-bounds) and `div` (`Primitives.lean:68-84`).
- Because `loop` and recursive functions are defined by `partial_fixpoint`, **Lean accepts the definition without a termination proof**.
- Non-termination is a value (`div`). Termination is a theorem the user proves afterwards (§2.5).
- Aeneas does not use a `divergent` keyword. Lean is not supported by the fuel-based alternative `-use-fuel` (`aeneas/src/Main.ml:532-535`) [src].

## 2.3 Results for all loop and iterator experiments

**Columns.** `A/L` are the Aeneas and Lean exit codes. “Axioms” counts `axiom` declarations in the generated file; these are missing std-library models, or the user’s own function when it could not be translated. Every program is listed in full, with its Lean output, in **Appendix A**.

| Case | Rust feature | A | L | Axioms | Status | Notes |
|---|---|---|---|---|---|---|
| L01 | `while` | 0 | 0 | 0 | Clean | `loop` combinator |
| L02 | `loop` + `break` | 0 | 0 | 0 | Clean | |
| L03 | `continue` | 0 | 0 | 0 | Clean | |
| L04 | early `return` in `while` | 0 | 0 | 0 | Clean | `done (some i)` |
| L05 | nested `while` | 0 | 0 | 0 | Clean | two `_loop`s |
| L05b | `break 'outer` | 1 | 0 | 1 | **Fails** | “Breaks to outer loops are not supported yet”; fn becomes `axiom` |
| L05c | `return` in inner loop | 1 | 0 | 1 | **Fails** | “Returns inside of nested loops are not supported yet” |
| L05d | `continue 'outer` | 1 | 0 | 1 | **Fails** | “Continue to outer loops are not supported yet” |
| L06 | `for i in 0..n` | 0 | 0 | 0 | Clean | `IteratorRange.next` |
| L06b | `0..=n`, `(0..n).step_by(2)` | 0 | 0 | 0 | Clean | |
| L07 | `for x in slice` | 0 | 0 | 0 | Clean | `slice::Iter` |
| L07b | index loops over slice | 0 | 0 | 0 | Clean | |
| L08 | `for x in &vec` | 0 | 0 | 1 | Caveats | `IntoIterator for &Vec` is an axiom |
| L08b | `for x in vec` (owned) | 0 | 0 | 0 | Clean | `vec::IntoIter` modelled |
| L08c | `vec.iter()`, `vec.iter_mut()` | 0 | 0 | 0 | Clean | via `Vec.deref` |
| L09 | `for x in [T;4]` / `&[T;4]` | 0 | 0 | 3 | Caveats | `array::IntoIter` and its `next` are axioms |
| L10 | `slice.iter()` | 0 | 0 | 0 | Clean | |

| Case | Rust feature | A | L | Axioms | Status | Notes |
|---|---|---|---|---|---|---|
| L11 | `slice.iter_mut()` | 0 | 0 | 0 | Clean | backward function threaded through loop |
| L11b | index loop mutating slice | 0 | 0 | 0 | Clean | |
| L12 | `vec.into_iter()` | 0 | 0 | 0 | Clean | |
| L13 | `.iter().map(..).collect()` | 0 | **1** | 3 | **Fails** | Lean: "`map` is not a field of structure `Iterator`" |
| L14 | `.iter().filter(..).count()` | 0 | 0 | 5 | Caveats | `Filter`, `filter`, `count` are axioms |
| L15 | `.enumerate()` | 0 | 0 | 0 | Clean | |
| L16 | `.zip()` | 0 | 0 | 1 | Caveats | zip modelled; `&u32 * &u32` is an axiom |
| L17 | `(0..n).rev()` | 0 | 0 | 0 | Clean | |
| L18 | `.fold(0, \|a,x\| a+x)` | 0 | 0 | 2 | Caveats | `Iter::fold` is an axiom |
| L19 | `.sum()` | 0 | 0 | 2 | Caveats | `sum` is an axiom |
| L20 | `(0..n).collect::<Vec<_>>()` | 0 | 0 | 0 | Clean | `FromIterator for Vec` modelled |
| L21 | closures capturing env in `map`/`any` | 0 | **1** | 4 | **Fails** | same error as L13 |
| L22 | loop mutating `&mut Vec` / `&mut struct` | 0 | 0 | 0 | Clean | |
| L23 | loop returning `&mut` into a list; returning `&` into a slice | 0 | 0 | 0 | Clean | list loop becomes recursive fn with backward closure |
| L24 | `while let Some(x) = v.pop()` | 0 | 0 | 1 | Caveats | `Vec::pop` is an axiom |
| L25 | user `impl Iterator` in `for` | 0 | 0 | 0 | Clean | |
| L26 | `loop {}` (no exit) | 1 | 0 | 0 | **Fails** | "(Infinite) loops which do not contain breaks are not supported yet" |
| L27 | `loop` exited only by `return` | 0 | 0 | 0 | Clean | |
| L28 | `return` and `break` in one loop, code after | 0 | 0 | 0 | Clean | |
| L29 | two sequential loops, each with `return` | 1 | 0 | 1 | **Fails** | "Breaks to outer loops are not supported yet" (reproduces [issue] #1501) |
| L30 | `for _ in 0..n { c += 1 }` | 0 | 0 | 0 | Clean | used for the proof in §2.5 |

**`-loops-to-rec` [test].** Re-running all L-cases with `-loops-to-rec` (`experiments/out-loops-to-rec/`, `experiments/results-loops-to-rec.md`) produced exactly the same exit codes. Only the shape of the Lean changed: direct recursion plus `partial_fixpoint` instead of `loop`.

**Warnings in clean range cases [test].** Every range case logs two Aeneas warnings, for example:

> "When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the corresponding field."

This matches open [issue] #1514. It did not stop the output from type-checking.

## 2.4 Selected cases: Rust input, Lean output, explanation

### (a) `while` loop → `loop` combinator (L01, default mode) [test]

```rust
pub fn sum_to(n: u32) -> u32 {
    let mut i = 0; let mut s = 0;
    while i < n { s += i; i += 1; }
    s
}
```

```
def sum_to_loop.body (n : Std.U32) (i : Std.U32) (s : Std.U32) :
  Result (ControlFlow (Std.U32 × Std.U32) Std.U32) := do
  if i < n
  then let s1 ← s + i          -- `+` is checked: overflow ⇒ `fail`
       let i1 ← i + 1#u32
       ok (cont (i1, s1))      -- next iteration with the new loop state
  else ok (done s)             -- exit with the loop's result

def sum_to_loop (n : Std.U32) (i : Std.U32) (s : Std.U32) : Result Std.U32 := do
  loop (fun (i1, s1) => sum_to_loop.body n i1 s1) (i, s)

def sum_to (n : Std.U32) : Result Std.U32 := do
  sum_to_loop n 0#u32 0#u32
```

**How the translation works.**

- The mutable variables live across iterations (`i`, `s`), so they become the loop state (a tuple).
- Each iteration returns `cont state'` or `done result`.
- Arithmetic is checked: `s + i` returns `fail` on overflow. Proving absence of overflow is part of proving the function correct.

### (b) Same loop with `-loops-to-rec` (L01) [test]

```
def sum_to_loop (n : Std.U32) (i : Std.U32) (s : Std.U32) : Result Std.U32 := do
  if i < n
  then let s1 ← s + i
       let i1 ← i + 1#u32
       sum_to_loop n i1 s1
  else ok s
partial_fixpoint
```

### (c) `for` over a range (L06) [test]

The `for` loop is desugared by rustc and Charon into calls to `Iterator::next`. The iterator becomes part of the loop state.

```rust
pub fn sum_range(n: u32) -> u32 { let mut s = 0; for i in 0..n { s += i; } s }
```

```
def sum_range_loop.body (iter : core.ops.range.Range Std.U32) (s : Std.U32) :
  Result (ControlFlow ((core.ops.range.Range Std.U32) × Std.U32) Std.U32) := do
  let (o, iter1) ← core.iter.range.IteratorRange.next core.iter.range.StepU32 iter
  match o with
  | none => ok (done s)
  | some i => let s1 ← s + i
              ok (cont (iter1, s1))

def sum_range (n : Std.U32) : Result Std.U32 := do
  sum_range_loop { start := 0#u32, «end» := n } 0#u32
```

`IteratorRange.next` is a **defined** (not axiomatised) model in the Lean library: `aeneas/backends/lean/Aeneas/Std/Core/Iter.lean:604` [src]. It has `@[step]` specification lemmas (`aeneas/backends/lean/Aeneas/Std/RangeIter.lean:263-276`), added by PR #1172, merged 2026-06-30 [src][issue].

**Docs and issues vs experiment.**

- Open [issue] #1132 (opened 2026-06-17) still says “IteratorRange::next has no reasoning lemma, range-for loops can’t be proved”.

- The lemma does exist at the tested commit, and I used it in the proof of L30 (§2.5) **[test]**. The issue appears to be out of date.

### (d) Mutation through an iterator over `&mut [T]` (L11) [test]

```rust
pub fn incr_all(s: &mut [u32]) { for x in s.iter_mut() { *x = x.wrapping_add(1); } }
```

```
def incr_all_loop.body (iter : core.slice.iter.IterMut Std.U32)
  (back : core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32) : ... := do
  let (o, iter1, next_back) ← core.slice.iter.IteratorIterMut.next iter
  match o with
  | none => ok (done (let im := next_back iter1 none
                      back im))
  | some x =>
    let x1 ← lift (core.num.U32.wrapping_add x 1#u32)
    ok (cont (iter1, fun im => let im1 := next_back im (some x1)
                               back im1))

def incr_all (s : Slice Std.U32) : Result (Slice Std.U32) := do
  let (iter, iter_mut_back) ← core.slice.Slice.iter_mut s
  let back ← incr_all_loop iter (fun im => im)
  ok (iter_mut_back back)
```

**How the translation works.**

- `&mut` borrows are not modelled with a heap. Each mutable borrow becomes a value plus a **backward function** that puts the updated value back into its owner.
- Inside the loop, the backward functions are composed into the loop state (`back`).
- After the loop, the composed function is applied to rebuild the updated slice.
- The user reasons about this purely functionally. No separation logic is needed.

### (e) Loop that returns a mutable borrow (L23) [test]

```rust
pub fn nth_mut<T>(mut l: &mut List<T>, mut i: u32) -> Option<&mut T> {
    while let List::Cons(x, tl) = l {
        if i == 0 { return Some(x); }
        i -= 1; l = tl;
    }
    None
}
```

```
def nth_mut_loop {T : Type _} (l : List T) (i : Std.U32) :
  Result ((Option T) × (Option T → List T)) := do
  match l with
  | List.Cons x tl =>
    if i = 0#u32
    then ok (some x, fun o => let t := match o with | some t1 => t1 | _ => x
                              List.Cons t tl)
    else
      let i1 ← i - 1#u32
      let (o, back) ← nth_mut_loop tl i1
      ok (o, fun o1 => let l1 := back o1
                       List.Cons x l1)
  | List.Nil => ok (none, fun o => List.Nil)
partial_fixpoint
```

**How the translation works.**

- The returned `&mut T` becomes a pair: the current value, and a function that writes a new value back into the list.
- This loop walks a recursive structure, so Aeneas emitted it as a recursive function even in default mode (see the heuristic in §2.2, step 4).

### (f) Iterator adaptor without a model (L13) [test]

```rust
pub fn doubled(s: &[u32]) -> Vec<u32> { s.iter().map(|x| x * 2).collect() }
```

Aeneas succeeds (exit 0) and emits axioms such as `core.iter.traits.iterator.Iterator.map.default` and `core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator.next`. Lean then rejects the file:

```
L13Map.lean:65:2: error: `map` is not a field of structure `core.iter.traits.iterator.Iterator`
L13Map.lean:71:2: error: `collect` is not a field of structure `core.iter.traits.iterator.Iterator`
```

**Cause [src].** The Lean model of the `Iterator` trait has only the fields `next`, `step_by`, `enumerate` and `take`. The file carries the comments "TODO: adding more fields like rev leads to a circularity" and "TODO: collect" (`aeneas/backends/lean/Aeneas/Std/Core/Iter.lean:92-105`). The generated `Map` instance tries to fill `map :=` and `collect :=` fields that do not exist.

**Tracking [issue].**

- This is open issue #1043, opened 2026-05-20.
- Open PRs that add models: #1510 (`map`, `chain`), #1542 (`fold`), #1544 (`any`), #1552 (`count`).

**Workaround.** Rewrite the adaptor as an explicit `for`/`while` loop. Every loop form in L01–L12 is supported.

### (g) What fails: cross-level control flow (L05b, L29) [test]

```
'outer: while i < n { ... while j < n { if i + j == t { r = i; break 'outer; } j += 1; } i += 1; }
```

```
[Error] Breaks to outer loops are not supported yet
Source: 'cases/L05b_nested_break_outer.rs', lines 1:0-16:1
[Error] Ignoring the body of 'l05b_nested_break_outer::find_pair' because of previous error
```

The function is emitted as `axiom find_pair : Std.U32 → Std.U32 → Result Std.U32`, and the Lean file still type-checks.

L29 has **no** labelled break in the source. It has two loops in sequence, each with an early `return`, and it fails with the same message. This reproduces open [issue] #1501, opened 2026-10-30. The issue explains that Charon places the second loop inside the first loop's exit branch.

## 2.5 What the user has to prove, and with which mechanism

**Semantics [src].** Specifications use Hoare-style triples over `Result`, defined in `aeneas/backends/lean/Aeneas/Std/WP.lean`:

- `f x ⦃ r => P r ⦄` is **total correctness**: `f x` returns `ok r` with `P r`; it neither fails nor diverges. Divergence makes it false: `spec_div : spec div p ↔ False`, `WP.lean:106`; notation at `WP.lean:191-192`.
- `f x ⦃ r => P r ⦄div` is **partial correctness**: divergence is allowed. `dspec_div … ↔ True`, `WP.lean:164`; notation at `WP.lean:195-196`.

So a total-correctness proof of a loop simultaneously proves:

1. No panic: no overflow, no out-of-bounds indexing, no failed `unwrap`, etc.
2. Termination.
3. The functional postcondition.

**Mechanisms [src][doc].**

| Translation mode | What Lean sees | What the user proves | Library support |
|---|---|---|---|
| Default (`loop` combinator) | `loop body init`, where `loop` is `partial_fixpoint` | A **measure** that decreases, a loop **invariant** and a **postcondition**, via `loop.spec_decr_nat` (Nat measure) or `loop.spec` (any well-founded relation) | `WP.lean:757-797` |

| Translation mode | What Lean sees | What the user proves | Library support |
|---|---|---|---|
| `-loops-to-rec` | Recursive `f_loop …` `partial_fixpoint` | A spec theorem about `f_loop` proved by recursion on itself. Termination is shown by `termination_by <measure>` and `decreasing_by scalar_decr_tac`; the invariant is the theorem's precondition | `aeneas-overview.md:143-175`; tutorial `aeneas/tests/lean/Tutorial/Solutions.lean:229-255` |

**Automation.** The `step` tactic applies `@[step]` lemmas for library functions and for the user's own loop lemmas. `scalar_tac` discharges bounded-integer arithmetic goals.

**Experiment [test].** I wrote the following proofs against the generated code. All three type-check with Lean 4.36.0-rc2 and the Aeneas library (exit 0, no `sorry`). Full text is in Appendix C; files are in `experiments/proofs/`.

1. **`find` (L04, early return), default mode**, using `loop.spec_decr_nat`:
   - measure `s.length - i`;
   - invariant "`i ≤ len` and no earlier element equals `x`";
   - postcondition "`some j` ⇒ `s[j] = x`; `none` ⇒ `x` is absent".

```
theorem find_loop_spec (s : Slice Std.U32) (x : Std.U32) (i : Std.Usize)
    (hi : i.val ≤ s.length) (hprev : ∀ k, k < i.val → s.val[k]! ≠ x) :
    find_loop s x i ⦃ r => FindPost s x r ⦄ := by
  unfold find_loop
  apply loop.spec_decr_nat
    (measure := fun (i : Std.Usize) => s.length - i.val)
    (inv := fun (i : Std.Usize) => i.val ≤ s.length ∧ ∀ k, k < i.val → s.val[k]! ≠ x)
  · rintro i' ⟨hi', hprev'⟩
    unfold find_loop.body
    ...   -- 15 lines: `step` through indexing and `+ 1`, `scalar_tac` for bounds
  · exact ⟨hi, hprev⟩
```

2. **The same `find`, `-loops-to-rec` mode.** The recursive spec ends with:

```
termination_by s.length - i.val
decreasing_by scalar_decr_tac
```

3. **`for _ in 0..n { c += 1 }` (L30), default mode.**
   - The loop state contains the `Range` iterator.
   - The invariant relates `it.start` to the counter: `it.end = n ∧ it.start ≤ n ∧ c = it.start`.
   - The measure is `n - it.start`.
   - The library's `@[step]` lemma for `IteratorRange.next` is applied automatically by `step`.
   - Proving absence of overflow of `c + 1` falls out of the invariant.

**Effort.** Each loop proof is 20–35 lines. Each took a few rounds of fixing against Lean's error messages; only the final, successful check logs are kept, in `experiments/proofs/*.log`. The `step` and `scalar_tac` tactics discharged all the bounded-integer arithmetic, including absence of overflow and index bounds.

**Caveats from the sources.**

- **[doc]** The combinator mode's proof infrastructure "is less mature — fewer lemmas, less automation" (`tips-and-tricks.md:21-22`).
- **[doc]** `partial_fixpoint_induct` should not be used, because it needs an admissibility proof that is `sorry`'d (`aeneas/documentation/skills/aeneas-lean-core.instructions.md:262-265`; per the docs survey, not re-checked).
- **[issue]** #1367 (open): recursion *through* a loop body does not pass Lean's `partial_fixpoint` monotonicity check.

## 2.6 Iterator model coverage in the Lean library

Read from source **[src]**: `aeneas/backends/lean/Aeneas/Std/Core/Iter.lean` and related files. Rows marked [test] were also confirmed by the experiments.

| Item | Modelled? |
|---|---|
| `Iterator` trait | Fields: `next`, `step_by`, `enumerate`, `take` only (`Iter.lean:92-105`) |
| `next` for `Range`, `RangeInclusive`, `StepBy`, `Enumerate`, `Take`, `TakeWhile`, `Zip`, `Rev`, slice `Iter/IterMut/ChunksExact`, `vec::IntoIter` | Yes (docs survey; [test] for Range, StepBy, Enumerate, Zip, Rev, Iter, IterMut, vec IntoIter) |
| `Map` | Type only, no `next`; axioms [test] |
| `Filter`, `fold`, `sum`, `any`, `count`, `chain`, `skip`, `windows`, `copied/cloned`, `peekable` | No; axioms [test for filter/fold/sum/any/count] |
| `array::IntoIter` (by value) | No; axioms [test] |
| `IntoIterator for &Vec` | No; axiom [test] |
| `FromIterator for Vec` (`collect`) | Yes [test L20] |
| `String/&str` | `Str` is a slice of bytes; `String` is opaque; `Chars::next` is opaque (`StringIter.lean:9-22`) |

**The axiom risk.** An `axiom` declared for a std function (e.g. `Iter::fold`) has no defining equations, so nothing can be proved about code that calls it. Users must either:

- write a model and its spec themselves (`-split-files` generates a `FunsExternal_Template.lean`: `aeneas/README.md:153-174` [doc]), or
- rewrite the code as an explicit loop.

# 3. Other features

## 3.1 Traits and generics

**Supported [test]:**

- traits with bounds (T01);
- associated types (T02);
- const generics (T03);
- default methods and supertraits (T05);
- generic structs with `impl` blocks (T06);
- `impl Trait` in argument and return position (T07);
- static trait methods `T::zero()` (T08).

All are clean.

**`dyn Trait` [test].** In T04, `fn area_dyn(x: &dyn Shape) -> u32 { x.area() }`:

- the *type* is translated as `Dyn (fun _dyn => Shape _dyn)`;
- the unsizing coercion at the call site works;
- the method call fails with "Function pointers are not supported yet", and the body becomes `sorry`.

`Box<dyn Trait>` (T04b) fails the same way.

- [src] Charon lowers dyn method calls to vtable function-pointer calls, and Aeneas rejects calls through function pointers (`aeneas/src/interp/InterpStatements.ml:1230,1249,1270`).
- [issue] #1500 (open, 2026-10-30) and PR #729 (preliminary `dyn` support, merged 2026-02-05).
- `dyn Trait` in struct fields is also rejected: "Dynamic trait types are not supported yet" (`aeneas/src/symbolic/SymbolicToPureTypes.ml:132,192,542,677` [src]; [issue] #1343).

**Generic associated types: not supported [src][issue].**

- "Generic Associated Types are not supported yet" (`SymbolicToPureTypes.ml:131`).
- Charon's `--preset=aeneas` cannot lift GATs: "GATs cannot work with the `--lift-associated-types` option (implied by `--preset=aeneas`)" (`charon/src/transform/normalize/expand_associated_types.rs:1145-1150`).
- Tracked in [issue] #1112 (open).

**Mutually recursive traits [test-suite].** These are a declared known failure: `aeneas/tests/src/mutually-recursive-traits.rs:1` (`//@ known-failure`); [issue] #176 (open).

**Higher-ranked bounds [test-suite].**

- Preliminary support landed in PR #1158 (merged 2026-07-02).
- Three known-failure tests remain: `aeneas/tests/src/higher_ranked_implied_bounds_{borrow,regions,types}.rs`. Their message: "Unimplemented … lifetime constraint relating a higher-ranked lifetime to a free lifetime".

## 3.2 Closures and function pointers

**Closures [test].** Local closures (C01), closures passed to generic `F: Fn` (C02), `FnOnce` with a moved capture (C05), and returning `impl Fn` (T07) are all clean.

- A closure becomes a Lean structure holding its captured state, plus `FnOnce`/`FnMut`/`Fn` instances (visible in L18, Appendix A).

**`FnMut` that mutates a capture (C03): fails in Lean [test].**

```
error: Type mismatch … has type closure → Unit → Result closure
       but is expected to have type closure → Unit → Result Unit
```

This matches open [issue] #960 (“Wrong binder for call to `FnMut` closure returning unit”) and the commented-out FnMut tests in `aeneas/tests/src/closures.rs:13-28` [test-suite].

**Function pointers (C04): not supported [test][src].**

- `fn(u32) -> u32` parameters give “Arrow types are not supported yet” (`SymbolicToPureTypes.ml:190,422,540,675`).
- Calls through them give “Function pointers are not supported yet” (`InterpStatements.ml:1230`).
- Turning a `fn` item into a pointer gives “Invalid input for unop: fn_def_to_fn_ptr”.

## 3.3 Borrows and lifetimes

**Supported [test]:**

- `&mut &mut T` parameters (B01);
- a struct with a lifetime holding `&'a mut [u32]`, with methods (B02);
- returning a borrow chosen between two inputs, and borrowing a field (B03);
- two-phase borrow `v.push(v.len())` (B05);
- recursion returning `&mut` into a `Box` list (B06).

**Generic instantiated with `&mut` (B04) [doc][test].** The README lists this case as unsupported (`aeneas/README.md:132-135`).

- In the experiment, Aeneas reports **no error** (exit 0) but emits ill-typed Lean: `let (i, id_back) ← id 0#u32`, where `id` returns a plain `T`.
- Lean reports “expected a product type”. The documentation and experiment agree that the case is unsupported, but the failure is **silent at the Aeneas level**.

**Other limits [src]:**

- “ADTs containing nested mutable borrows are not supported yet” (`SymbolicToPureTypes.ml:295-300`).
- “Mutable references for lifetime ’static are not supported” (`aeneas/src/interp/InterpExpressions.ml:459-461`).
- Open [issue]s: #929, #1186, #1503 (nested borrows).

## 3.4 Arrays, slices, Vec, strings, Option/Result, Box

**Arrays and slices (D01, D02) [test].**

- Clean: indexing, update, `len`, 2-D arrays, sub-slicing `&s[1..3]`, `split_at_mut`, `swap`.

**`Vec` (D03) [test].**

- `new`, `push`, index, index-assign and `extend_from_slice` are modelled.
- `insert` is modelled.
- `pop` and `remove` are **axioms**.
- `vec![1,2,3]` introduces an axiom for `MaybeUninit`.
- `vec![0; n]` translates.

**Strings (D04) [test].** `str::len`, `&str == &str`, `String::new` and `String::push_str` are all **axioms**; only string literals translate.

**`Option`/`Result` (D05) [test].**

- `match`, `unwrap_or` and `map` with a closure are modelled.
- `?` on `Result` is modelled.
- `?` on `Option` uses the axiom `Option…Try.branch`.

**`Box` (D06) [test].** Box is erased (`Box<T>` becomes `T`); recursive enums through `Box` work.

**Derives (D08) [test].**

- `#[derive(Clone, PartialEq, Default)]` translate.
- `Debug` introduces a `Formatter` axiom.

## 3.5 Integer arithmetic, casts, overflow, panics

**Arithmetic (A01, A02, A04) [test].**

- Every `+ - * / %` and unary minus on `u8…u128`/`i16`/`i32` is translated to a checked operation in `Result`: overflow or division by zero give `fail`.
- `wrapping_*`, `checked_add`, `overflowing_add`, `saturating_sub`, bit ops, shifts and `rotate_left` are modelled.
- `count_ones` is an axiom.

**Casts (A03) [test].**

- Integer↔integer `as`, `bool as u8` and `u32::from(u16)` are modelled.
- `u8::try_from(u32)` is an axiom.
- **`u8 as char` crashes Aeneas** with an internal "Unreachable" error. Aeneas then writes a *partial* Lean file whose `c4` body is cut off, so **the whole file fails to parse** (`unexpected token '/--'`).

**Panics (A05) [test].** `panic!`, `assert!`, `assert_eq!`, `unwrap`, `expect` and `unreachable!` all become `fail` in the `Result` monad.

- [src] Unwinding is not supported: "unwinding is not supported by Aeneas" (`InterpStatements.ml:794`).

**Floats (A06) [test].** "Invalid inputs for binop", and the body becomes `sorry`.

- [src] `aeneas/src/symbolic/SynthesizeSymbolic.ml:79` "Float are not supported in Aeneas yet".
- [issue] #828 (open).

## 3.6 Recursion and mutual recursion

**Supported [test].** Simple recursion (`fact`, list length; R01) and mutual recursion between functions (`even`/`odd`; R02).

- Both are emitted as `partial_fixpoint` definitions, inside a `mutual … end` block for R02. Termination is left to the user, exactly as for loops.

**Mutually inductive types (R03) [test].** Mutual recursion over a mutually inductive pair of types (`Tree`/`Forest`) translates, but **Lean rejects it**:

```
Could not prove 'r03_mutual_inductive.tsum' to be monotone in its recursive calls:
  Cannot eliminate recursive call `fsum f` enclosed in  let (v, f) := t ...
```

**Related [src].**

- `-decreases-clauses` (generate `termination_by` templates) is refused for Lean together with mutually recursive groups (`aeneas/src/Main.ml:604-616`).
- "Mutually recursive globals are not supported" (`aeneas/src/Translate.ml`).

## 3.7 Unsafe code

The README states that unsafe code is out of scope for now, pending separation-logic work (`aeneas/README.md:120-124, 137-139`) **[doc]**.

| Case | Result |
|---|---|
| U01 `&mut x as *mut u32` + deref | "Unsupported operation: &raw mut (*x)"; body becomes `sorry` [test]. Deref also rejected in source: `InterpPaths.ml:191-197` "Aeneas does not yet support dereferencing raw pointers." [src] |
| U02 pointer casts, pointer→`usize` | "Unsupported operation: &raw const (*x)" [test] |
| U03 `union` | "unions are not supported" (`TypesAnalysis.ml:544`), then internal errors [test][src]; [issue] #1199 open |
| U04 `mem::transmute` | **No error**: emitted as `axiom core.intrinsics.transmute` [test]. Source also has "Unsupported: transmute" for transmute *casts* (`SymbolicToPureExpressions.ml:625`) [src] |

| Case | Result |
|---|---|
| U05 `unsafe fn` + `ptr::read` | `ptr::read` becomes an axiom over `ConstRawPtr`; the caller fails on `&raw const` [test] |
| U06 `static`/`const` | Reading `static K` and `const C` works; `static mut` gives "Unexpected access to a global" [test] |

**[issue]** PR #1428 "Heap model for raw pointers" (merged 2026-10-13) added library groundwork. The experiments show that the translator does not yet use it for ordinary raw-pointer code.

## 3.8 Interior mutability and reference counting

**`Cell`, `RefCell`, `Rc`, `Arc` (I01–I04) [test].** All translate with the types and every method as **axioms**, for example:

- `axiom core.cell.Cell (T : Type _) : Type`
- `axiom core.cell.Cell.set … : Cell T → T → Result Unit`

**Consequence.** In I01, `fn c(x: &Cell<u32>) { x.set(x.get() + 1) }` becomes:

```
def c (x : core.cell.Cell Std.U32) : Result Unit := do
  let i ← core.cell.Cell.get core.marker.CopyU32 x
  let i1 ← i + 1#u32
  core.cell.Cell.set x i1
```

The write is not threaded back to the caller, so the model cannot express the mutation.

- [issue] #1525 (open, 2026-11-03) reports the same problem for `Cell::set` and `Atomic*::fetch_add` and calls it potentially unsound.
- [doc] `aeneas/docs/user/src/README.md:15` lists interior mutability as "ongoing work".

## 3.9 Concurrency and async

**Threads and synchronisation (X01–X03) [test].**

- `Mutex::lock`, `MutexGuard`, `AtomicU32::fetch_add` and `thread::spawn`/`join` are all axioms.
- `thread::spawn(..).join()` additionally produces Lean that does not type-check ("contains universe level metavariables").

**async (X04) [test].**

- Charon warns "Coroutine types are not supported yet" and "Coroutines are not supported" (`charon/src/bin/charon-driver/translate/translate_types.rs:332`, `translate_bodies.rs:1487,2174` [src]).
- Aeneas then reports "Found type error in the output of charon" plus internal errors.
- [issue] #1265 (open) depends on charon#609 (open). charon PR #1602 "Translate coroutines" was opened 2026-11-09.

**Scope [doc].** README: concurrency will be addressed by the separation-logic work (`aeneas/README.md:137-139`).

## 3.10 FFI, extern functions, opaque declarations

**What happens [test].**

- `extern "C" { fn ext_add(..) }` (F01) becomes `axiom ext_add : Std.U32 → Std.U32 → Result Std.U32`. The caller translates normally.
- A module marked opaque with Charon's `--opaque 'crate::hidden'` (F02) becomes an axiom in the same way.
- `#[no_mangle] pub extern "C" fn` (F03) translates normally.

**How opacity works [src].**

- Charon has four opacity levels: Transparent, Foreign, Opaque, Invisible (`charon/src/ast/meta.rs`, per the source survey).
- Options and attributes control them: `--include`, `--opaque`, `--exclude`, and `#[charon::opaque]`/`#[aeneas::opaque]`/`#[verify::opaque]` (`charon/src/bin/charon-driver/translate/translate_meta.rs:692-711`).

- `--preset=aeneas` (`charon/src/options.rs:566-581`) does **not** extract bodies of external crates. Every std function that lacks a hand-written Lean model therefore becomes an axiom.

**Recommended practice [doc].** Use `-split-files` so that missing definitions go to a template the user fills in (`aeneas/README.md:153-174`).

## 3.11 Standard-library coverage (summary)

**Size [src].**

- The Lean library `aeneas/backends/lean/Aeneas/Std/` is about 15.5k lines.
- Per the docs survey, `aeneas/src/extract/ExtractBuiltinLean.ml` lists 517 Rust-to-Lean mappings: 39 types, 345 functions, 47 trait declarations, 86 trait impls. The largest groups are `core::iter` (78), `core::slice` (55) and `core::num` (54).

**Coverage.**

- **Modelled:** scalars and their methods; arrays; slices; much of `Vec`; `Option`/`Result` basics; ranges and several iterators; `Box` (erased); `cmp`/`min`/`max`; `mem::swap`/`replace`; `From`/`Into`; `to_le_bytes`/`from_le_bytes` [test S02, S03, S07].
- **Not modelled (axioms) [test]:**
  - `Cell`, `RefCell`, `Rc`, `Arc`;
  - `HashMap`, `BTreeMap`, `VecDeque`;
  - `String` operations;
  - `format!`/`println!`;
  - `slice::sort`, `mem::take`;
  - `Vec::pop`/`remove`;
  - most iterator adaptors.

## 3.12 Drops

**`Drop` impls (S08) [test].** A struct with a `Drop` impl translates, but `drop` is never called: the function body is just `ok 1#u32`.

- [src] By default drops are no-ops: `drop_as_no_op = ref true`, `aeneas/src/Config.ml:625-627`. The option `-eval-drops` exists.
- [issue] #1410 (open): “user `Drop` impls never run, so a destructor’s effect is silently lost”.

# 4. Known open issues that affect users

All issues below were verified against the GitHub API dump in `notes/gh/`: state as of 2026-11-10, and the date is the creation date. The full classified list (237 open Aeneas issues, 69 open Charon issues) is in `notes/github-issues.md`.

## 4.1 Loops

| # | Title (abridged) | Opened |
|---|---|---|
| 822 | Support for early returns inside loops that can be replaced with breaks | 2026-03-04 |
| 1501 | “Breaks to outer loops are not supported yet” for two returning loops in sequence (reproduced here, L29) | 2026-10-30 |
| 1221 | Lean backend silently emits a bodyless axiom for a function with a `return` nested two loops deep | 2026-07-18 |
| 1367 | No `partial_fixpoint` monotonicity lemma for `loop`; recursion through a loop fails in Lean | 2026-09-26 |
| 1395 | Fieldless enum variant written into an array is translated as `()` when a later loop borrows the array (miscompilation) | 2026-10-03 |
| 1513 | “Unimplemented” when a loop reads an array after an indexed write | 2026-10-30 |
| 1519 | Early break causes problems when joining abstractions | 2026-11-02 |
| 1162 | Internal error on a loop inside a trait method | 2026-06-24 |
| charon#955 | More stable looping control flow | 2025-12-22 |

**Related history.** PR #958 “Support nested loop exits” was closed **unmerged** on 2026-06-22.

**Recently fixed.**

- #964 and #1206: “Could not match the contexts” with a conditional `break` in a `for` loop. Closed 2026-07-15 and 2026-09-16.
- charon#1051: control-flow misreconstruction with loops. Closed 2026-06-26 via charon PR #1306.

## 4.2 Iterators

| # | Title (abridged) | Opened | State |
|---|---|---|---|
| 1043 | Iterator trait impl generates `map :=` and `collect :=` fields (reproduced here, L13/L21) | 2026-05-20 | open |
| 1132 | `IteratorRange::next` has no reasoning lemma (a lemma now exists, see §2.4c) | 2026-06-17 | open |
| 1192 | Second loop zipping two mutable slice iterators | 2026-07-07 | open |
| 1193 | Trait method passed to `.map` is “Unimplemented” | 2026-07-08 | open |
| 1222 | `Iterator::try_for_each` aborts with “Internal error” | 2026-07-20 | open |
| 1053 | `Iterator::zip` test fails with `RErased` | 2026-05-22 | open |
| 1435 | Uncaught exception `CFailure` | 2026-10-13 | open |
| 1437 | Missing backwards function with iterators | 2026-10-14 | open |

| # | Title (abridged) | Opened | State |
|---|---|---|---|
| 1152, 1153 | Ill-typed `Iterator.next` / `IntoIter.zip` output | 2026-06-22 | open |
| 1511 | No model for `Iterator::chain` | 2026-10-30 | open |
| 1514 | `Step` lacks `forward_overflowing`/`backward_overflowing` (warning seen here) | 2026-10-30 | open |
| 1408, 1287 | Signed `Step::backward_checked` modelled wrongly (`rev()` over signed ranges) | 2026-10-09, 2026-08-24 | open |
| 1534 | `RangeFrom::get_mut` model writes back to the wrong part of the slice | 2026-11-03 | open |

**Open PRs adding iterator models:**

- #1510 (`map`, `chain`)
- #1542 (`fold`, `rfold`)
- #1544 (`any`)
- #1552 (`count`)
- #1057 (`DoubleEndedIterator`/`Rev`)
- #1314 (iterators over arrays)
- #1133 and #1468 (range step specs)

**Recently merged iterator work:**

- #698 basic iterators (2026-01-14)
- #786 `enumerate`/`take` (2026-02-19)
- #1143 iterator transformers (2026-06-20)
- #1172 std models and theorems (2026-06-30)
- #1131 `Iterator` extraction fix (2026-09-03)
- #1386 `next_back` (2026-10-28)
- #1373 `TakeWhile` (2026-11-07)
- #1486 `IntoIterator for &mut [T; N]` (2026-11-09)

## 4.3 Other

| # | Topic | State |
|---|---|---|
| 960 | `FnMut` closure call returning unit is mistyped (reproduced, C03) | open |
| 1046 | `FnOnce` `call_once` return type wrong | open |
| 1119 | Internal error: returning a closure that borrows its environment | open |
| 1500 | Method calls through `&dyn Trait` (reproduced, T04) | open |
| 1343 | `Box<dyn Iterator>` field rejected | open |
| 1112 | GATs | open |
| 176, 1264 | Mutually recursive traits / functions + trait impls | open |
| 1227 | Full higher-ranked trait bounds | open |
| 929, 1186, 1503 | Nested borrows | open |
| 1199 | Unions | open |
| 1396 | Inline asm | open |
| 1525 | `Cell`/atomics modelled as pure, which can be unsound (cf. I01) | open |
| 1410 | `Drop` impls never run (reproduced, S08) | open |
| 828 | Floats (reproduced, A06) | open |
| 1265 / charon#609 | `async` (reproduced, X04) | open |

| # | Topic | State |
|---|---|---|
| charon#142 | Charon’s tracking issue for unsupported Rust features | open |
| charon#1040 | Missing lifetime information for closure captures | open |

**Stale Charon documentation [doc vs issue].** `charon/docs/limitations.md:13-15` lists charon#583 as a “known unsoundness”, but charon#583 was closed on 2025-12-08.

# 5. How to reproduce

Everything lives in the work directory (the root of this repository); nothing outside it was modified. Toolchains are isolated:

- `RUSTUP_HOME` and `CARGO_HOME` under `tools/`;
- `ELAN_HOME` under `tools/elan`;
- `OPAMROOT` under `tools/opam`;
- `gh` 2.63.2 and `pandoc` 3.5 binaries under `tools/`.

All of them are set by `env.sh` (copy it from `env.sh.example`).

```
cd aeneas-limitations                                    # repo root
git clone https://github.com/AeneasVerif/aeneas.git        # tested at 0d21a9e
git clone https://github.com/AeneasVerif/charon.git        # main b9da4e3 (recorded only)
git -C charon worktree add ../charon-pinned $(tail -1 aeneas/charon-pin)   # 454586e
cp env.sh.example env.sh && source env.sh
# OCaml deps (isolated opam root; pins required by Charon's nix files):
opam init --bare -y --disable-sandboxing --no-setup
opam switch create default ocaml-base-compiler.5.3.0
opam pin add -n visitors.20260520 "git+https://gitlab.inria.fr/fpottier/visitors.git#20260520"
opam pin add -n ppx_deriving_hash.0.1.5 "git+https://github.com/sim642/ppx_deriving_hash.git#0.1.5"
opam install dune calendar domainslib easy_logging logs menhir ocamlgraph ppx_deriving \
  ppx_deriving_yojson ppxlib progress unionFind visitors yojson zarith ppx_deriving_hash re
(cd charon-pinned && make build-charon-rust build-charon-ml)
(cd aeneas && ln -sfn ../charon-pinned charon && make)
(cd aeneas/backends/lean && lake exe cache get && lake build)
# Experiments
experiments/run_charon.sh                # Charon on every experiments/cases/*.rs
experiments/run_aeneas.sh                # Aeneas (Lean) + `lake env lean` type-check
OUT=out-loops-to-rec AENEAS_EXTRA=-loops-to-rec experiments/run_aeneas.sh L01_while ...
python3 -I experiments/classify.py experiments/out > experiments/results.md
# Proofs: concatenate generated Lean + proof, then check
cd experiments/leancheck && lake env lean ../proofs/FindLoopComb.lean
```

**Commands used per case.**

```
charon rustc --preset=aeneas --dest-file out/C/C.llbc -- --crate-type=lib --edition=2021 cases/C.rs
aeneas -backend lean out/C/C.llbc -dest out/C/lean
```

F02 additionally passes `--opaque 'crate::hidden'` to Charon.

**Problems met while building, and how they were solved.**

1. The system opam index was stale: `visitors 20251114` and `ppx_deriving_hash 0.1.3` are too old, and the latter causes `Error: This kind of expression is not allowed as right-hand side of "let rec"` in charon-ml. I did not update the global index; instead I used a private `OPAMROOT` plus git pins.
2. A Lean toolchain was accidentally downloaded into the global elan home (`.elan` in the home directory) once, when `env.sh` had not been sourced. It was removed immediately with `elan toolchain uninstall`.

**Output layout.**

| Path | Contents |
|---|---|
| `experiments/cases/*.rs` | 100 Rust programs |
| `experiments/out/<case>/` | `.llbc`, `charon.log`, `aeneas.log`, generated `lean/*.lean`, `lean.log`, exit codes |
| `experiments/out-loops-to-rec/` | Same, for L-cases with `-loops-to-rec` |
| `experiments/results*.md`, `summary*.txt` | Tables and per-case error/axiom summaries |
| `experiments/proofs/` | Proofs and their check logs |
| `notes/` | Survey notes (docs and tests, source, GitHub). The raw GitHub API JSON (`notes/gh/`) is not included; `scripts/fetchall.sh` refetches it |
| `logs/` | Build logs |

**What was not checked.**

- I did not run Aeneas's own test suite (`make test`). Its contents and known-failure markers were read from the test files instead.
- I did not test the F*, Coq/Rocq and HOL4 backends.
- Charon upstream `main` was not run, because Aeneas pins an older Charon.
- GitHub data covers all open issues and those closed in the last 12 months. Comments were read for 17 threads only, and 200 of about 400 tags were fetched (all releases were fetched).
- The behaviour of the `-decreases-clauses` mode for Lean was not exercised.
- How exactly Aeneas decides, in default mode, between the `loop` combinator and recursion was read from source but not characterised experimentally beyond L23.

# Appendix A — Loop and iterator experiments (full)

Every case below was produced by `experiments/run_charon.sh` and `experiments/run_aeneas.sh` (default options). Lean output is shown with each `/-- ... -/` doc comment collapsed to a one-line `--` comment and blank lines removed; nothing else is edited. Long outputs are truncated (marked).

### L01_while — CLEAN

```rust
pub fn sum_to(n: u32) -> u32 {
    let mut i = 0;
    let mut s = 0;
    while i < n {
        s += i;
        i += 1;
    }
    s
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l01_while
-- [l01_while::sum_to]: loop body 0:
@[rust_loop_body]
def sum_to_loop.body
  (n : Std.U32) (i : Std.U32) (s : Std.U32) :
  Result (ControlFlow (Std.U32 × Std.U32) Std.U32)
  := do
  if i < n
  then let s1 ← s + i
       let i1 ← i + 1#u32
       ok (cont (i1, s1))
  else ok (done s)
-- [l01_while::sum_to]: loop 0:
@[rust_loop]
def sum_to_loop
  (n : Std.U32) (i : Std.U32) (s : Std.U32) : Result Std.U32 := do
  loop
    (fun (i1, s1) => sum_to_loop.body n i1 s1)
    (i, s)
-- [l01_while::sum_to]:
@[reducible]
def sum_to (n : Std.U32) : Result Std.U32 := do
  sum_to_loop n 0#u32 0#u32
end l01_while
```

### L02_loop_break — CLEAN

```rust
pub fn first_pow2_ge(n: u32) -> u32 {
    let mut p = 1;
    loop {
        if p >= n {
            break;
        }
        p *= 2;
    }
    p
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l02_loop_break
-- [l02_loop_break::first_pow2_ge]: loop body 0:
@[rust_loop_body]
def first_pow2_ge_loop.body
  (n : Std.U32) (p : Std.U32) : Result (ControlFlow Std.U32 Std.U32) := do
  if p >= n
```

```
  then ok (done p)
  else let p1 ← p * 2#u32
       ok (cont p1)
-- [l02_loop_break::first_pow2_ge]: loop 0:
@[rust_loop]
def first_pow2_ge_loop (n : Std.U32) (p : Std.U32) : Result Std.U32 := do
  loop
    (fun p1 => first_pow2_ge_loop.body n p1)
    p
-- [l02_loop_break::first_pow2_ge]:
@[reducible]
def first_pow2_ge (n : Std.U32) : Result Std.U32 := do
  first_pow2_ge_loop n 1#u32
end l02_loop_break
```

### L03_continue — CLEAN

```
pub fn sum_even(n: u32) -> u32 {
    let mut i = 0;
    let mut s = 0;
    while i < n {
        i += 1;
        if i % 2 == 1 {
            continue;
        }
        s += i;
    }
    s
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l03_continue
-- [l03_continue::sum_even]: loop body 0:
@[rust_loop_body]
def sum_even_loop.body
  (n : Std.U32) (i : Std.U32) (s : Std.U32) :
  Result (ControlFlow (Std.U32 × Std.U32) Std.U32)
  := do
  if i < n
  then
    let i1 ← i + 1#u32
    let i2 ← i1 % 2#u32
    if i2 = 1#u32
    then ok (cont (i1, s))
    else let s1 ← s + i1
         ok (cont (i1, s1))
  else ok (done s)
-- [l03_continue::sum_even]: loop 0:
@[rust_loop]
def sum_even_loop
  (n : Std.U32) (i : Std.U32) (s : Std.U32) : Result Std.U32 := do
  loop
    (fun (i1, s1) => sum_even_loop.body n i1 s1)
    (i, s)
-- [l03_continue::sum_even]:
@[reducible]
def sum_even (n : Std.U32) : Result Std.U32 := do
  sum_even_loop n 0#u32 0#u32
end l03_continue
```

### L04_early_return — CLEAN

```
pub fn find(s: &[u32], x: u32) -> Option<usize> {
    let mut i = 0;
    while i < s.len() {
        if s[i] == x {
            return Some(i);
        }
        i += 1;
    }
    None
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l04_early_return
-- [l04_early_return::find]: loop body 0:
@[rust_loop_body]
def find_loop.body
  (s : Slice Std.U32) (x : Std.U32) (i : Std.Usize) :
  Result (ControlFlow Std.Usize (Option Std.Usize))
  := do
  let i1 := Slice.len s
  if i < i1
  then
    let i2 ← Slice.index_usize s i
    if i2 = x
    then ok (done (some i))
    else let i3 ← i + 1#usize
         ok (cont i3)
  else ok (done none)
-- [l04_early_return::find]: loop 0:
@[rust_loop]
def find_loop
  (s : Slice Std.U32) (x : Std.U32) (i : Std.Usize) :
  Result (Option Std.Usize)
  := do
  loop
    (fun i1 => find_loop.body s x i1)
    i
-- [l04_early_return::find]:
@[reducible]
def find (s : Slice Std.U32) (x : Std.U32) : Result (Option Std.Usize) := do
  find_loop s x 0#usize
end l04_early_return
```

## L05_nested — CLEAN

```
pub fn count_pairs(n: u32) -> u32 {
    let mut c = 0;
    let mut i = 0;
    while i < n {
        let mut j = 0;
        while j < i {
            c += 1;
            j += 1;
        }
        i += 1;
    }
    c
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l05_nested
-- [l05_nested::count_pairs]: loop body 1:
@[rust_loop_body]
def count_pairs_loop0_loop0.body
  (i : Std.U32) (c : Std.U32) (j : Std.U32) :
  Result (ControlFlow (Std.U32 × Std.U32) Std.U32)
  := do
  if j < i
  then let c1 ← c + 1#u32
       let j1 ← j + 1#u32
       ok (cont (c1, j1))
  else ok (done c)
-- [l05_nested::count_pairs]: loop 1:
@[rust_loop]
def count_pairs_loop0_loop0
  (c : Std.U32) (i : Std.U32) (j : Std.U32) : Result Std.U32 := do
  loop
    (fun (c1, j1) => count_pairs_loop0_loop0.body i c1 j1)
    (c, j)
-- [l05_nested::count_pairs]: loop body 0:
@[rust_loop_body]
```

```
def count_pairs_loop0.body
  (n : Std.U32) (c : Std.U32) (i : Std.U32) :
  Result (ControlFlow (Std.U32 × Std.U32) Std.U32)
  := do
  if i < n
  then
    let c1 ← count_pairs_loop0_loop0 c i 0#u32
    let i1 ← i + 1#u32
    ok (cont (c1, i1))
  else ok (done c)
-- [l05_nested::count_pairs]: loop 0:
@[rust_loop]
def count_pairs_loop0
  (n : Std.U32) (c : Std.U32) (i : Std.U32) : Result Std.U32 := do
  loop
    (fun (c1, i1) => count_pairs_loop0.body n c1 i1)
    (c, i)
-- [l05_nested::count_pairs]:
@[reducible]
def count_pairs (n : Std.U32) : Result Std.U32 := do
  count_pairs_loop0 n 0#u32 0#u32
end l05_nested
```

## L05b_nested_break_outer — FAIL

```rust
pub fn find_pair(n: u32, t: u32) -> u32 {
    let mut i = 0;
    let mut r = 0;
    'outer: while i < n {
        let mut j = 0;
        while j < n {
            if i + j == t {
                r = i;
                break 'outer;
            }
            j += 1;
        }
        i += 1;
    }
    r
}
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Breaks to outer loops are not supported yet
[Error] Ignoring the body of 'l05b_nested_break_outer::find_pair' because of previous error
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l05b_nested_break_outer
-- [l05b_nested_break_outer::find_pair]:
axiom find_pair : Std.U32 → Std.U32 → Result Std.U32
end l05b_nested_break_outer
```

## L05c_nested_return — FAIL

```rust
pub fn find_pair(n: u32, t: u32) -> Option<(u32, u32)> {
    let mut i = 0;
    while i < n {
        let mut j = 0;
        while j < n {
            if i + j == t {
                return Some((i, j));
            }
            j += 1;
        }
        i += 1;
    }
    None
}
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Returns inside of nested loops are not supported yet
[Error] Ignoring the body of 'l05c_nested_return::find_pair' because of previous error
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l05c_nested_return
-- [l05c_nested_return::find_pair]:
axiom find_pair : Std.U32 → Std.U32 → Result (Option (Std.U32 × Std.U32))
end l05c_nested_return
```

## L05d_nested_continue_outer — FAIL

```rust
pub fn f(n: u32) -> u32 {
    let mut i = 0;
    let mut c = 0;
    'outer: while i < n {
        i += 1;
        let mut j = 0;
        while j < i {
            j += 1;
            if j == 3 {
                continue 'outer;
            }
            c += 1;
        }
    }
    c
}
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Continue to outer loops are not supported yet
[Error] Ignoring the body of 'l05d_nested_continue_outer::f' because of previous error
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l05d_nested_continue_outer
-- [l05d_nested_continue_outer::f]:
axiom f : Std.U32 → Result Std.U32
end l05d_nested_continue_outer
```

## L06_for_range — CLEAN

```rust
pub fn sum_range(n: u32) -> u32 {
    let mut s = 0;
    for i in 0..n {
        s += i;
    }
    s
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
```

```
information for item 'backward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
```

Generated Lean:

```
namespace l06_for_range
-- [l06_for_range::sum_range]: loop body 0:
@[rust_loop_body]
def sum_range_loop.body
  (iter : core.ops.range.Range Std.U32) (s : Std.U32) :
  Result (ControlFlow ((core.ops.range.Range Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ←
    core.iter.range.IteratorRange.next core.iter.range.StepU32 iter
  match o with
  | none => ok (done s)
  | some i => let s1 ← s + i
              ok (cont (iter1, s1))
-- [l06_for_range::sum_range]: loop 0:
@[rust_loop]
def sum_range_loop
  (iter : core.ops.range.Range Std.U32) (s : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, s1) => sum_range_loop.body iter1 s1)
    (iter, s)
-- [l06_for_range::sum_range]:
@[reducible]
def sum_range (n : Std.U32) : Result Std.U32 := do
  sum_range_loop { start := 0#u32, «end» := n } 0#u32
end l06_for_range
```

## L06b_for_range_inclusive_step — CLEAN

```rust
pub fn sum_incl(n: u32) -> u32 {
    let mut s = 0;
    for i in 0..=n {
        s += i;
    }
    s
}
pub fn sum_step(n: usize) -> usize {
    let mut s = 0;
    for i in (0..n).step_by(2) {
        s += i;
    }
    s
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'backward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
```

Generated Lean:

```
namespace l06b_for_range_inclusive_step
-- [l06b_for_range_inclusive_step::sum_incl]: loop body 0:
@[rust_loop_body]
def sum_incl_loop.body
  (iter : core.ops.range.RangeInclusive Std.U32) (s : Std.U32) :
  Result (ControlFlow ((core.ops.range.RangeInclusive Std.U32) × Std.U32)
    Std.U32)
  := do
  let (o, iter1) ←
    core.ops.range.RangeInclusive.Insts.CoreIterTraitsIteratorIterator.next
      core.iter.range.StepU32 iter
  match o with
  | none => ok (done s)
```

```
  | some i => let s1 ← s + i
              ok (cont (iter1, s1))
-- [l06b_for_range_inclusive_step::sum_incl]: loop 0:
@[rust_loop]
def sum_incl_loop
  (iter : core.ops.range.RangeInclusive Std.U32) (s : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, s1) => sum_incl_loop.body iter1 s1)
    (iter, s)
-- [l06b_for_range_inclusive_step::sum_incl]:
def sum_incl (n : Std.U32) : Result Std.U32 := do
  let iter ← core.ops.range.RangeInclusive.new 0#u32 n
  sum_incl_loop iter 0#u32
-- [l06b_for_range_inclusive_step::sum_step]: loop body 0:
@[rust_loop_body]
def sum_step_loop.body
  (iter : core.iter.adapters.step_by.StepBy (core.ops.range.Range Std.Usize))
  (s : Std.Usize) :
  Result (ControlFlow ((core.iter.adapters.step_by.StepBy (core.ops.range.Range
    Std.Usize)) × Std.Usize) Std.Usize)
  := do
  let (o, iter1) ←
    core.iter.adapters.step_by.IteratorStepBy.next
      (core.iter.traits.iterator.IteratorRange core.iter.range.StepUsize) iter
  match o with
  | none => ok (done s)
  | some i => let s1 ← s + i
              ok (cont (iter1, s1))
-- [l06b_for_range_inclusive_step::sum_step]: loop 0:
@[rust_loop]
def sum_step_loop
  (iter : core.iter.adapters.step_by.StepBy (core.ops.range.Range Std.Usize))
  (s : Std.Usize) :
  Result Std.Usize
  := do
  loop
    (fun (iter1, s1) => sum_step_loop.body iter1 s1)
    (iter, s)
-- [l06b_for_range_inclusive_step::sum_step]:
def sum_step (n : Std.Usize) : Result Std.Usize := do
  let iter ←
    core.iter.traits.iterator.Iterator.step_by.trait_default
      (core.iter.traits.iterator.IteratorRange core.iter.range.StepUsize)
      { start := 0#usize, «end» := n } 2#usize
  sum_step_loop iter 0#usize
end l06b_for_range_inclusive_step
```

## L07_for_slice — CLEAN

```rust
pub fn sum_slice(s: &[u32]) -> u32 {
    let mut t = 0;
    for x in s {
        t += *x;
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l07_for_slice
-- [l07_for_slice::sum_slice]: loop body 0:
@[rust_loop_body]
def sum_slice_loop.body
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) :
  Result (ControlFlow ((core.slice.iter.Iter Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ← core.slice.iter.IteratorSliceIter.next iter
  match o with
  | none => ok (done t)
  | some x => let t1 ← t + x
              ok (cont (iter1, t1))
-- [l07_for_slice::sum_slice]: loop 0:
@[rust_loop]
```

```
def sum_slice_loop
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, t1) => sum_slice_loop.body iter1 t1)
    (iter, t)
-- [l07_for_slice::sum_slice]:
def sum_slice (s : Slice Std.U32) : Result Std.U32 := do
  let iter ←
    SharedSlice.Insts.CoreIterTraitsCollectIntoIteratorSharedIter.into_iter s
  sum_slice_loop iter 0#u32
end l07_for_slice
```

## L07b_index_loop_slice — CLEAN

```rust
pub fn sum_slice_idx(s: &[u32]) -> u32 {
    let mut t = 0;
    let mut i = 0;
    while i < s.len() {
        t += s[i];
        i += 1;
    }
    t
}
pub fn sum_slice_for_idx(s: &[u32]) -> u32 {
    let mut t = 0;
    for i in 0..s.len() {
        t += s[i];
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'backward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
```

Generated Lean:

```
namespace l07b_index_loop_slice
-- [l07b_index_loop_slice::sum_slice_idx]: loop body 0:
@[rust_loop_body]
def sum_slice_idx_loop.body
  (s : Slice Std.U32) (t : Std.U32) (i : Std.Usize) :
  Result (ControlFlow (Std.U32 × Std.Usize) Std.U32)
  := do
  let i1 := Slice.len s
  if i < i1
  then
    let i2 ← Slice.index_usize s i
    let t1 ← t + i2
    let i3 ← i + 1#usize
    ok (cont (t1, i3))
  else ok (done t)
-- [l07b_index_loop_slice::sum_slice_idx]: loop 0:
@[rust_loop]
def sum_slice_idx_loop
  (s : Slice Std.U32) (t : Std.U32) (i : Std.Usize) : Result Std.U32 := do
  loop
    (fun (t1, i1) => sum_slice_idx_loop.body s t1 i1)
    (t, i)
-- [l07b_index_loop_slice::sum_slice_idx]:
@[reducible]
def sum_slice_idx (s : Slice Std.U32) : Result Std.U32 := do
  sum_slice_idx_loop s 0#u32 0#usize
-- [l07b_index_loop_slice::sum_slice_for_idx]: loop body 0:
@[rust_loop_body]
def sum_slice_for_idx_loop.body
  (s : Slice Std.U32) (iter : core.ops.range.Range Std.Usize) (t : Std.U32) :
  Result (ControlFlow ((core.ops.range.Range Std.Usize) × Std.U32) Std.U32)
  := do
```

```
  let (o, iter1) ←
    core.iter.range.IteratorRange.next core.iter.range.StepUsize iter
  match o with
  | none => ok (done t)
  | some i =>
    let i1 ← Slice.index_usize s i
    let t1 ← t + i1
    ok (cont (iter1, t1))
-- [l07b_index_loop_slice::sum_slice_for_idx]: loop 0:
@[rust_loop]
def sum_slice_for_idx_loop
  (iter : core.ops.range.Range Std.Usize) (s : Slice Std.U32) (t : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, t1) => sum_slice_for_idx_loop.body s iter1 t1)
    (iter, t)
-- [l07b_index_loop_slice::sum_slice_for_idx]:
def sum_slice_for_idx (s : Slice Std.U32) : Result Std.U32 := do
  let i := Slice.len s
  sum_slice_for_idx_loop { start := 0#usize, «end» := i } s 0#u32
end l07b_index_loop_slice
```

## L08_for_vec — CAVEATS

```rust
pub fn sum_vec_ref(v: &Vec<u32>) -> u32 {
    let mut t = 0;
    for x in v {
        t += *x;
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l08_for_vec
-- [alloc::vec::{impl core::iter::traits::collect::IntoIterator<&'a T, core::slice::iter::Iter<'a, T>> for &'a
alloc::vec::Vec<T>}::into_iter]:
@[rust_fun
  "alloc::vec::{core::iter::traits::collect::IntoIterator<&'a alloc::vec::Vec<@T>, &'a @T,
core::slice::iter::Iter<'a, @T>>}::into_iter"]
axiom SharedAVec.Insts.CoreIterTraitsCollectIntoIteratorSharedATIter.into_iter
  {T : Type _} (A : Type _) :
  alloc.vec.Vec T → Result (core.slice.iter.Iter T)
-- [l08_for_vec::sum_vec_ref]: loop body 0:
@[rust_loop_body]
def sum_vec_ref_loop.body
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) :
  Result (ControlFlow ((core.slice.iter.Iter Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ← core.slice.iter.IteratorSliceIter.next iter
  match o with
  | none => ok (done t)
  | some x => let t1 ← t + x
              ok (cont (iter1, t1))
-- [l08_for_vec::sum_vec_ref]: loop 0:
@[rust_loop]
def sum_vec_ref_loop
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, t1) => sum_vec_ref_loop.body iter1 t1)
    (iter, t)
-- [l08_for_vec::sum_vec_ref]:
def sum_vec_ref (v : alloc.vec.Vec Std.U32) : Result Std.U32 := do
  let iter ←
    SharedAVec.Insts.CoreIterTraitsCollectIntoIteratorSharedATIter.into_iter
      alloc.alloc.Global v
```

```
  sum_vec_ref_loop iter 0#u32
end l08_for_vec
```

## L08b_for_vec_owned — CLEAN

```rust
pub fn sum_vec_owned(v: Vec<u32>) -> u32 {
    let mut t = 0;
    for x in v {
        t += x;
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l08b_for_vec_owned
-- [l08b_for_vec_owned::sum_vec_owned]: loop body 0:
@[rust_loop_body]
def sum_vec_owned_loop.body
  (iter : alloc.vec.into_iter.IntoIter Std.U32) (t : Std.U32) :
  Result (ControlFlow ((alloc.vec.into_iter.IntoIter Std.U32) × Std.U32)
    Std.U32)
  := do
  let (o, iter1) ← alloc.vec.into_iter.IteratorIntoIter.next iter
  match o with
  | none => ok (done t)
  | some x => let t1 ← t + x
              ok (cont (iter1, t1))
-- [l08b_for_vec_owned::sum_vec_owned]: loop 0:
@[rust_loop]
def sum_vec_owned_loop
  (iter : alloc.vec.into_iter.IntoIter Std.U32) (t : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, t1) => sum_vec_owned_loop.body iter1 t1)
    (iter, t)
-- [l08b_for_vec_owned::sum_vec_owned]:
def sum_vec_owned (v : alloc.vec.Vec Std.U32) : Result Std.U32 := do
  let iter ← alloc.vec.IntoIteratorVec.into_iter v
  sum_vec_owned_loop iter 0#u32
end l08b_for_vec_owned
```

## L08c_vec_iter_method — CLEAN

```rust
pub fn sum_vec_iter(v: &Vec<u32>) -> u32 {
    let mut t: u32 = 0;
    for x in v.iter() {
        t = t.wrapping_add(*x);
    }
    t
}
pub fn incr_vec(v: &mut Vec<u32>) {
    for x in v.iter_mut() {
        *x = x.wrapping_add(1);
    }
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l08c_vec_iter_method
-- [l08c_vec_iter_method::sum_vec_iter]: loop body 0:
@[rust_loop_body]
def sum_vec_iter_loop.body
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) :
  Result (ControlFlow ((core.slice.iter.Iter Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ← core.slice.iter.IteratorSliceIter.next iter
  match o with
  | none => ok (done t)
```

```
  | some x =>
    let t1 ← lift (core.num.U32.wrapping_add t x)
    ok (cont (iter1, t1))
-- [l08c_vec_iter_method::sum_vec_iter]: loop 0:
@[rust_loop]
def sum_vec_iter_loop
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, t1) => sum_vec_iter_loop.body iter1 t1)
    (iter, t)
-- [l08c_vec_iter_method::sum_vec_iter]:
def sum_vec_iter (v : alloc.vec.Vec Std.U32) : Result Std.U32 := do
  let s := alloc.vec.Vec.deref v
  let iter ← core.slice.Slice.iter s
  sum_vec_iter_loop iter 0#u32
-- [l08c_vec_iter_method::incr_vec]: loop body 0:
@[rust_loop_body]
def incr_vec_loop.body
  (iter : core.slice.iter.IterMut Std.U32)
  (back : core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32)
  :
  Result (ControlFlow ((core.slice.iter.IterMut Std.U32) ×
    (core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32))
    (core.slice.iter.IterMut Std.U32))
  := do
  let (o, iter1, next_back) ← core.slice.iter.IteratorIterMut.next iter
  match o with
  | none => ok (done (let im := next_back iter1 none
                      back im))
  | some x =>
    let x1 ← lift (core.num.U32.wrapping_add x 1#u32)
    ok (cont (iter1, fun im => let im1 := next_back im (some x1)
                               back im1))
-- [l08c_vec_iter_method::incr_vec]: loop 0:
@[rust_loop]
def incr_vec_loop
  (iter : core.slice.iter.IterMut Std.U32)
  (back : core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32)
  :
  Result (core.slice.iter.IterMut Std.U32)
  := do
  loop
    (fun (iter1, back1) => incr_vec_loop.body iter1 back1)
    (iter, back)
-- [l08c_vec_iter_method::incr_vec]:
def incr_vec (v : alloc.vec.Vec Std.U32) : Result (alloc.vec.Vec Std.U32) := do
  let (s, deref_mut_back) ← lift (alloc.vec.Vec.deref_mut v)
  let (iter, iter_mut_back) ← core.slice.Slice.iter_mut s
  let back ← incr_vec_loop iter (fun im => im)
  let s1 := iter_mut_back back
  ok (deref_mut_back s1)
end l08c_vec_iter_method
```

## L09_for_array — CAVEATS

```rust
pub fn sum_array(a: [u32; 4]) -> u32 {
    let mut t = 0;
    for x in a {
        t += x;
    }
    t
}
pub fn sum_array_ref(a: &[u32; 4]) -> u32 {
    let mut t = 0;
    for x in a {
        t += *x;
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l09_for_array
-- [core::array::iter::IntoIter]
@[rust_type "core::array::iter::IntoIter"]
axiom core.array.iter.IntoIter (T : Type _) (N : Std.Usize) : Type
-- [core::array::iter::{impl core::iter::traits::collect::IntoIterator<T, core::array::iter::IntoIter<T, N>> for
[T; N]}::into_iter]:
@[rust_fun
  "core::array::iter::{core::iter::traits::collect::IntoIterator<[@T; @N], @T, core::array::iter::IntoIter<@T,
@N>>}::into_iter"]
axiom Array.Insts.CoreIterTraitsCollectIntoIteratorTIntoIter.into_iter
  {T : Type _} {N : Std.Usize} :
  Array T N → Result (core.array.iter.IntoIter T N)
-- [core::array::iter::{impl core::iter::traits::iterator::Iterator<T> for core::array::iter::IntoIter<T,
N>}::next]:
@[rust_fun
  "core::array::iter::{core::iter::traits::iterator::Iterator<core::array::iter::IntoIter<@T, @N>, @T>}::next"]
axiom core.array.iter.IntoIter.Insts.CoreIterTraitsIteratorIterator.next
  {T : Type _} {N : Std.Usize} :
  core.array.iter.IntoIter T N → Result ((Option T) ×
    (core.array.iter.IntoIter T N))
-- [l09_for_array::sum_array]: loop body 0:
@[rust_loop_body]
def sum_array_loop.body
  (iter : core.array.iter.IntoIter Std.U32 4#usize) (t : Std.U32) :
  Result (ControlFlow ((core.array.iter.IntoIter Std.U32 4#usize) × Std.U32)
    Std.U32)
  := do
  let (o, iter1) ←
    core.array.iter.IntoIter.Insts.CoreIterTraitsIteratorIterator.next iter
  match o with
  | none => ok (done t)
  | some x => let t1 ← t + x
              ok (cont (iter1, t1))
-- [l09_for_array::sum_array]: loop 0:
@[rust_loop]
def sum_array_loop
  (iter : core.array.iter.IntoIter Std.U32 4#usize) (t : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, t1) => sum_array_loop.body iter1 t1)
    (iter, t)
-- [l09_for_array::sum_array]:
def sum_array (a : Array Std.U32 4#usize) : Result Std.U32 := do
  let iter ←
    Array.Insts.CoreIterTraitsCollectIntoIteratorTIntoIter.into_iter a
  sum_array_loop iter 0#u32
-- [l09_for_array::sum_array_ref]: loop body 0:
@[rust_loop_body]
def sum_array_ref_loop.body
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) :
  Result (ControlFlow ((core.slice.iter.Iter Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ← core.slice.iter.IteratorSliceIter.next iter
  match o with
  | none => ok (done t)
  | some x => let t1 ← t + x
              ok (cont (iter1, t1))
-- [l09_for_array::sum_array_ref]: loop 0:
@[rust_loop]
def sum_array_ref_loop
  (iter : core.slice.iter.Iter Std.U32) (t : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, t1) => sum_array_ref_loop.body iter1 t1)
    (iter, t)
-- [l09_for_array::sum_array_ref]:
def sum_array_ref (a : Array Std.U32 4#usize) : Result Std.U32 := do
  let iter ←
    SharedArray.Insts.CoreIterTraitsCollectIntoIteratorSharedIter.into_iter a
  sum_array_ref_loop iter 0#u32
end l09_for_array
```

## L10_iter — CLEAN

```
pub fn count_gt(s: &[u32], k: u32) -> u32 {
    let mut c = 0;
```

```
    for x in s.iter() {
        if *x > k {
            c += 1;
        }
    }
    c
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l10_iter
-- [l10_iter::count_gt]: loop body 0:
@[rust_loop_body]
def count_gt_loop.body
  (k : Std.U32) (iter : core.slice.iter.Iter Std.U32) (c : Std.U32) :
  Result (ControlFlow ((core.slice.iter.Iter Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ← core.slice.iter.IteratorSliceIter.next iter
  match o with
  | none => ok (done c)
  | some x =>
    if x > k
    then let c1 ← c + 1#u32
         ok (cont (iter1, c1))
    else ok (cont (iter1, c))
-- [l10_iter::count_gt]: loop 0:
@[rust_loop]
def count_gt_loop
  (iter : core.slice.iter.Iter Std.U32) (k : Std.U32) (c : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, c1) => count_gt_loop.body k iter1 c1)
    (iter, c)
-- [l10_iter::count_gt]:
def count_gt (s : Slice Std.U32) (k : Std.U32) : Result Std.U32 := do
  let iter ← core.slice.Slice.iter s
  count_gt_loop iter k 0#u32
end l10_iter
```

## L11_iter_mut — CLEAN

```
pub fn incr_all(s: &mut [u32]) {
    for x in s.iter_mut() {
        *x = x.wrapping_add(1);
    }
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l11_iter_mut
-- [l11_iter_mut::incr_all]: loop body 0:
@[rust_loop_body]
def incr_all_loop.body
  (iter : core.slice.iter.IterMut Std.U32)
  (back : core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32)
  :
  Result (ControlFlow ((core.slice.iter.IterMut Std.U32) ×
    (core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32))
    (core.slice.iter.IterMut Std.U32))
  := do
  let (o, iter1, next_back) ← core.slice.iter.IteratorIterMut.next iter
  match o with
  | none => ok (done (let im := next_back iter1 none
                      back im))
  | some x =>
    let x1 ← lift (core.num.U32.wrapping_add x 1#u32)
    ok (cont (iter1, fun im => let im1 := next_back im (some x1)
                               back im1))
-- [l11_iter_mut::incr_all]: loop 0:
@[rust_loop]
```

```
def incr_all_loop
  (iter : core.slice.iter.IterMut Std.U32)
  (back : core.slice.iter.IterMut Std.U32 → core.slice.iter.IterMut Std.U32)
  :
  Result (core.slice.iter.IterMut Std.U32)
  := do
  loop
    (fun (iter1, back1) => incr_all_loop.body iter1 back1)
    (iter, back)
-- [l11_iter_mut::incr_all]:
def incr_all (s : Slice Std.U32) : Result (Slice Std.U32) := do
  let (iter, iter_mut_back) ← core.slice.Slice.iter_mut s
  let back ← incr_all_loop iter (fun im => im)
  ok (iter_mut_back back)
end l11_iter_mut
```

## L11b_index_loop_mut — CLEAN

```rust
pub fn incr_all_idx(s: &mut [u32]) {
    let mut i = 0;
    while i < s.len() {
        s[i] = s[i].wrapping_add(1);
        i += 1;
    }
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l11b_index_loop_mut
-- [l11b_index_loop_mut::incr_all_idx]: loop body 0:
@[rust_loop_body]
def incr_all_idx_loop.body
  (s : Slice Std.U32) (i : Std.Usize) :
  Result (ControlFlow ((Slice Std.U32) × Std.Usize) (Slice Std.U32))
  := do
  let i1 := Slice.len s
  if i < i1
  then
    let i2 ← Slice.index_usize s i
    let i3 ← lift (core.num.U32.wrapping_add i2 1#u32)
    let s1 ← Slice.update s i i3
    let i4 ← i + 1#usize
    ok (cont (s1, i4))
  else ok (done s)
-- [l11b_index_loop_mut::incr_all_idx]: loop 0:
@[rust_loop]
def incr_all_idx_loop
  (s : Slice Std.U32) (i : Std.Usize) : Result (Slice Std.U32) := do
  loop
    (fun (s1, i1) => incr_all_idx_loop.body s1 i1)
    (s, i)
-- [l11b_index_loop_mut::incr_all_idx]:
@[reducible]
def incr_all_idx (s : Slice Std.U32) : Result (Slice Std.U32) := do
  incr_all_idx_loop s 0#usize
end l11b_index_loop_mut
```

## L12_into_iter — CLEAN

```rust
pub fn sum_into_iter(v: Vec<u32>) -> u32 {
    let mut t = 0;
    for x in v.into_iter() {
        t += x;
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l12_into_iter
-- [l12_into_iter::sum_into_iter]: loop body 0:
@[rust_loop_body]
def sum_into_iter_loop.body
  (iter : alloc.vec.into_iter.IntoIter Std.U32) (t : Std.U32) :
  Result (ControlFlow ((alloc.vec.into_iter.IntoIter Std.U32) × Std.U32)
    Std.U32)
  := do
  let (o, iter1) ← alloc.vec.into_iter.IteratorIntoIter.next iter
  match o with
  | none => ok (done t)
  | some x => let t1 ← t + x
              ok (cont (iter1, t1))
-- [l12_into_iter::sum_into_iter]: loop 0:
@[rust_loop]
def sum_into_iter_loop
  (iter : alloc.vec.into_iter.IntoIter Std.U32) (t : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, t1) => sum_into_iter_loop.body iter1 t1)
    (iter, t)
-- [l12_into_iter::sum_into_iter]:
def sum_into_iter (v : alloc.vec.Vec Std.U32) : Result Std.U32 := do
  let iter ← alloc.vec.IntoIteratorVec.into_iter v
  sum_into_iter_loop iter 0#u32
end l12_into_iter
```

## L13_map — FAIL

```rust
pub fn doubled(s: &[u32]) -> Vec<u32> {
    s.iter().map(|x| x * 2).collect()
}
```

Exit codes: charon=0, aeneas=0, lean=1.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'map'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'collect'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Lean errors:

```
../out/L13_map/lean/L13Map.lean:65:2: error: `map` is not a field of structure
`core.iter.traits.iterator.Iterator`
../out/L13_map/lean/L13Map.lean:71:2: error: `collect` is not a field of structure
`core.iter.traits.iterator.Iterator`
```

Generated Lean:

```
namespace l13_map
-- [core::iter::traits::iterator::Iterator::map]:
@[trait_default, rust_fun "core::iter::traits::iterator::Iterator::map"]
axiom core.iter.traits.iterator.Iterator.map.default
  {Self : Type _} {B : Type _} {F : Type _} {Clause0_Item : Type _}
  (IteratorInst : core.iter.traits.iterator.Iterator Self Clause0_Item)
  (opsfunctionFnMutFTupleClause0_ItemBInst : core.ops.function.FnMut F
  Clause0_Item B) :
  Self → F → Result (core.iter.adapters.map.Map Self F)
-- [core::iter::adapters::map::{impl core::iter::traits::iterator::Iterator<B> for
core::iter::adapters::map::Map<I, F>}::next]:
@[rust_fun
  "core::iter::adapters::map::{core::iter::traits::iterator::Iterator<core::iter::adapters::map::Map<@I, @F>,
@B>}::next"]
axiom core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator.next
  {B : Type _} {I : Type _} {F : Type _} {Clause0_Item : Type _}
  (traitsiteratorIteratorInst : core.iter.traits.iterator.Iterator I
  Clause0_Item) (opsfunctionFnMutFTupleClause0_ItemBInst :
```

```
  core.ops.function.FnMut F Clause0_Item B) :
  core.iter.adapters.map.Map I F → Result ((Option B) ×
    (core.iter.adapters.map.Map I F))
-- Trait implementation: [core::iter::adapters::map::{impl core::iter::traits::iterator::Iterator<B> for
core::iter::adapters::map::Map<I, F>}]
@[reducible, rust_trait_impl
  "core::iter::traits::iterator::Iterator<core::iter::adapters::map::Map<@I, @F>, @B>"]
impl_def core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator {B :
  Type _} {I : Type _} {F : Type _} {Clause0_Item : Type _}
  (traitsiteratorIteratorInst : core.iter.traits.iterator.Iterator I
  Clause0_Item) (opsfunctionFnMutFTupleClause0_ItemBInst :
  core.ops.function.FnMut F Clause0_Item B) :
  core.iter.traits.iterator.Iterator (core.iter.adapters.map.Map I F) B := {
  next := core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator.next
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst
  map := fun {B1 : Type} {F1 : Type} (opsfunctionFnMutPTupleBPInst :
    core.ops.function.FnMut F1 B B1) =>
    core.iter.traits.iterator.Iterator.map.default
    (core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst)
    opsfunctionFnMutPTupleBPInst
  collect := fun {B1 : Type} (collectFromIteratorInst :
    core.iter.traits.collect.FromIterator B1 B) =>
    core.iter.traits.iterator.Iterator.collect.default
    (core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst)
    collectFromIteratorInst
}
-- [core::ops::arith::{impl core::ops::arith::Mul<u32, u32> for &'_0 u32}::mul]:
@[rust_fun "core::ops::arith::{core::ops::arith::Mul<&'0 u32, u32, u32>}::mul"]
axiom Shared0U32.Insts.CoreOpsArithMulU32U32.mul
  : Std.U32 → Std.U32 → Result Std.U32
-- [l13_map::doubled::{closure}]
@[reducible]
def doubled.closure := Unit
-- [l13_map::doubled::{impl core::ops::function::FnMut<(&'_0 u32,), u32> for l13_map::doubled::{closure}
<'_0>}::call_mut]:
def doubled.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32.call_mut
  (c : doubled.closure) (tupled_args : Std.U32) :
  Result (Std.U32 × doubled.closure)
  := do
  let i ← Shared0U32.Insts.CoreOpsArithMulU32U32.mul tupled_args 2#u32
  ok (i, ())
-- [l13_map::doubled::{impl core::ops::function::FnOnce<(&'_0 u32,), u32> for l13_map::doubled::{closure}
<'_0>}::call_once]:
def doubled.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32.call_once
  (c : doubled.closure) (i : Std.U32) : Result Std.U32 := do
  let (i1, _) ←
    doubled.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32.call_mut () i
  ok i1
-- Trait implementation: [l13_map::doubled::{impl core::ops::function::FnOnce<(&'_0 u32,), u32> for
l13_map::doubled::{closure}<'_0>}]
@[reducible]
def doubled.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32 :
  core.ops.function.FnOnce doubled.closure Std.U32 Std.U32 := {
  call_once :=
    doubled.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32.call_once
}
-- Trait implementation: [l13_map::doubled::{impl core::ops::function::FnMut<(&'_0 u32,), u32> for
l13_map::doubled::{closure}<'_0>}]
@[reducible]
def doubled.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32 :
  core.ops.function.FnMut doubled.closure Std.U32 Std.U32 := {
  FnOnceInst := doubled.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32
  call_mut :=
    doubled.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32.call_mut
}
-- [l13_map::doubled]:
def doubled (s : Slice Std.U32) : Result (alloc.vec.Vec Std.U32) := do
  let i ← core.slice.Slice.iter s
  let m ←
    core.iter.traits.iterator.Iterator.map.default
      (core.iter.traits.iterator.IteratorSliceIter Std.U32)
      doubled.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32 i ()
  core.iter.traits.iterator.Iterator.collect.default
    (core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator
    (core.iter.traits.iterator.IteratorSliceIter Std.U32)
    doubled.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32)
    (core.iter.traits.collect.FromIteratorVec Std.U32) m
-- ... (1 more lines, see experiments/out/)
```

## L14_filter — CAVEATS

```rust
pub fn count_even(s: &[u32]) -> usize {
    s.iter().filter(|x| **x % 2 == 0).count()
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'count'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'filter'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l14_filter
-- [core::iter::adapters::filter::Filter]
@[rust_type "core::iter::adapters::filter::Filter"]
axiom core.iter.adapters.filter.Filter (I : Type _) (P : Type _) : Type
-- [core::iter::adapters::filter::{impl core::iter::traits::iterator::Iterator<Clause0_Item> for
core::iter::adapters::filter::Filter<I, P>}::count]:
@[rust_fun
  "core::iter::adapters::filter::
{core::iter::traits::iterator::Iterator<core::iter::adapters::filter::Filter<@I, @P>, @Clause0_Item>}::count"]
axiom
  core.iter.adapters.filter.Filter.Insts.CoreIterTraitsIteratorIterator.count
  {I : Type _} {P : Type _} {Clause0_Item : Type _} (traitsiteratorIteratorInst
  : core.iter.traits.iterator.Iterator I Clause0_Item)
  (opsfunctionFnMutPTupleSharedClause0_ItemBoolInst : core.ops.function.FnMut P
  Clause0_Item Bool) :
  core.iter.adapters.filter.Filter I P → Result Std.Usize
-- [core::iter::traits::iterator::Iterator::count]:
@[trait_default, rust_fun "core::iter::traits::iterator::Iterator::count"]
axiom core.iter.traits.iterator.Iterator.count.default
  {Self : Type _} {Clause0_Item : Type _} (IteratorInst :
  core.iter.traits.iterator.Iterator Self Clause0_Item) :
  Self → Result Std.Usize
-- [core::iter::traits::iterator::Iterator::filter]:
@[trait_default, rust_fun "core::iter::traits::iterator::Iterator::filter"]
axiom core.iter.traits.iterator.Iterator.filter.default
  {Self : Type _} {P : Type _} {Clause0_Item : Type _} (IteratorInst :
  core.iter.traits.iterator.Iterator Self Clause0_Item)
  (opsfunctionFnMutPTupleSharedClause0_ItemBoolInst : core.ops.function.FnMut P
  Clause0_Item Bool) :
  Self → P → Result (core.iter.adapters.filter.Filter Self P)
-- [core::slice::iter::{impl core::iter::traits::iterator::Iterator<&'a T> for core::slice::iter::Iter<'a,
T>}::count]:
@[rust_fun
  "core::slice::iter::{core::iter::traits::iterator::Iterator<core::slice::iter::Iter<'a, @T>, &'a @T>}::count"]
axiom core.slice.iter.Iter.Insts.CoreIterTraitsIteratorIteratorSharedAT.count
  {T : Type _} : core.slice.iter.Iter T → Result Std.Usize
-- [l14_filter::count_even::{closure}]
@[reducible]
def count_even.closure := Unit
-- [l14_filter::count_even::{impl core::ops::function::FnMut<(&'_1 &'_0 u32,), bool> for
l14_filter::count_even::{closure}<'_0>}::call_mut]:
def
  count_even.closure.Insts.CoreOpsFunctionFnMutTupleShared1Shared0U32Bool.call_mut
  (c : count_even.closure) (tupled_args : Std.U32) :
  Result (Bool × count_even.closure)
  := do
  let i ← tupled_args % 2#u32
  ok (i = 0#u32, ())
-- [l14_filter::count_even::{impl core::ops::function::FnOnce<(&'_1 &'_0 u32,), bool> for
l14_filter::count_even::{closure}<'_0>}::call_once]:
def
  count_even.closure.Insts.CoreOpsFunctionFnOnceTupleShared1Shared0U32Bool.call_once
  (c : count_even.closure) (i : Std.U32) : Result Bool := do
  let (b, _) ←
    count_even.closure.Insts.CoreOpsFunctionFnMutTupleShared1Shared0U32Bool.call_mut
```

```
      () i
  ok b
-- Trait implementation: [l14_filter::count_even::{impl core::ops::function::FnOnce<(&'_1 &'_0 u32,), bool> for
l14_filter::count_even::{closure}<'_0>}]
@[reducible]
def count_even.closure.Insts.CoreOpsFunctionFnOnceTupleShared1Shared0U32Bool :
  core.ops.function.FnOnce count_even.closure Std.U32 Bool := {
  call_once :=
    count_even.closure.Insts.CoreOpsFunctionFnOnceTupleShared1Shared0U32Bool.call_once
}
-- Trait implementation: [l14_filter::count_even::{impl core::ops::function::FnMut<(&'_1 &'_0 u32,), bool> for
l14_filter::count_even::{closure}<'_0>}]
@[reducible]
def count_even.closure.Insts.CoreOpsFunctionFnMutTupleShared1Shared0U32Bool :
  core.ops.function.FnMut count_even.closure Std.U32 Bool := {
  FnOnceInst :=
    count_even.closure.Insts.CoreOpsFunctionFnOnceTupleShared1Shared0U32Bool
  call_mut :=
    count_even.closure.Insts.CoreOpsFunctionFnMutTupleShared1Shared0U32Bool.call_mut
}
-- [l14_filter::count_even]:
def count_even (s : Slice Std.U32) : Result Std.Usize := do
  let i ← core.slice.Slice.iter s
  let f ←
    core.iter.traits.iterator.Iterator.filter.default
      (core.iter.traits.iterator.IteratorSliceIter Std.U32)
      count_even.closure.Insts.CoreOpsFunctionFnMutTupleShared1Shared0U32Bool i
      ()
  core.iter.adapters.filter.Filter.Insts.CoreIterTraitsIteratorIterator.count
    (core.iter.traits.iterator.IteratorSliceIter Std.U32)
    count_even.closure.Insts.CoreOpsFunctionFnMutTupleShared1Shared0U32Bool f
end l14_filter
```

## L15_enumerate — CLEAN

```rust
pub fn weighted(s: &[u32]) -> usize {
    let mut t = 0;
    for (i, x) in s.iter().enumerate() {
        t += i * (*x as usize);
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l15_enumerate
-- [l15_enumerate::weighted]: loop body 0:
@[rust_loop_body]
def weighted_loop.body
  (iter : core.iter.adapters.enumerate.Enumerate (core.slice.iter.Iter
  Std.U32)) (t : Std.Usize) :
  Result (ControlFlow ((core.iter.adapters.enumerate.Enumerate
    (core.slice.iter.Iter Std.U32)) × Std.Usize) Std.Usize)
  := do
  let (o, iter1) ←
    core.iter.adapters.enumerate.IteratorEnumerate.next
      (core.iter.traits.iterator.IteratorSliceIter Std.U32) iter
  match o with
  | none => ok (done t)
  | some p =>
    let (i, x) := p
    let i1 ← lift (UScalar.cast .Usize x)
    let i2 ← i * i1
    let t1 ← t + i2
    ok (cont (iter1, t1))
-- [l15_enumerate::weighted]: loop 0:
@[rust_loop]
def weighted_loop
  (iter : core.iter.adapters.enumerate.Enumerate (core.slice.iter.Iter
  Std.U32)) (t : Std.Usize) :
  Result Std.Usize
  := do
  loop
    (fun (iter1, t1) => weighted_loop.body iter1 t1)
    (iter, t)
```

```
-- [l15_enumerate::weighted]:
def weighted (s : Slice Std.U32) : Result Std.Usize := do
  let i ← core.slice.Slice.iter s
  let iter ←
    core.iter.traits.iterator.Iterator.enumerate.trait_default
      (core.iter.traits.iterator.IteratorSliceIter Std.U32) i
  weighted_loop iter 0#usize
end l15_enumerate
```

## L16_zip — CAVEATS

```rust
pub fn dot(a: &[u32], b: &[u32]) -> u32 {
    let mut t = 0;
    for (x, y) in a.iter().zip(b.iter()) {
        t += x * y;
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'zip'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l16_zip
-- [core::ops::arith::{impl core::ops::arith::Mul<&'_0 u32, u32> for &'_1 u32}::mul]:
@[rust_fun
  "core::ops::arith::{core::ops::arith::Mul<&'1 u32, &'0 u32, u32>}::mul"]
axiom Shared1U32.Insts.CoreOpsArithMulShared0U32U32.mul
  : Std.U32 → Std.U32 → Result Std.U32
-- [l16_zip::dot]: loop body 0:
@[rust_loop_body]
def dot_loop.body
  (iter : core.iter.adapters.zip.Zip (core.slice.iter.Iter Std.U32)
  (core.slice.iter.Iter Std.U32)) (t : Std.U32) :
  Result (ControlFlow ((core.iter.adapters.zip.Zip (core.slice.iter.Iter
    Std.U32) (core.slice.iter.Iter Std.U32)) × Std.U32) Std.U32)
  := do
  let (o, iter1) ←
    core.iter.adapters.zip.Zip.Insts.CoreIterTraitsIteratorIteratorPair.next
      (core.iter.traits.iterator.IteratorSliceIter Std.U32)
      (core.iter.traits.iterator.IteratorSliceIter Std.U32) iter
  match o with
  | none => ok (done t)
  | some p =>
    let (x, y) := p
    let i ← Shared1U32.Insts.CoreOpsArithMulShared0U32U32.mul x y
    let t1 ← t + i
    ok (cont (iter1, t1))
-- [l16_zip::dot]: loop 0:
@[rust_loop]
def dot_loop
  (iter : core.iter.adapters.zip.Zip (core.slice.iter.Iter Std.U32)
  (core.slice.iter.Iter Std.U32)) (t : Std.U32) :
  Result Std.U32
  := do
  loop
    (fun (iter1, t1) => dot_loop.body iter1 t1)
    (iter, t)
-- [l16_zip::dot]:
def dot (a : Slice Std.U32) (b : Slice Std.U32) : Result Std.U32 := do
  let i ← core.slice.Slice.iter a
  let i1 ← core.slice.Slice.iter b
  let iter ←
    core.iter.traits.iterator.Iterator.zip.trait_default
      (core.iter.traits.iterator.IteratorSliceIter Std.U32)
      (core.iter.traits.collect.IntoIterator.Blanket
      (core.iter.traits.iterator.IteratorSliceIter Std.U32)) i i1
```

```
  dot_loop iter 0#u32
end l16_zip
```

## L17_rev — CLEAN

```rust
pub fn last_nonzero(n: usize, s: &[u32]) -> Option<usize> {
    for i in (0..n).rev() {
        if s[i] != 0 {
            return Some(i);
        }
    }
    None
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'rev'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'backward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
```

Generated Lean:

```
namespace l17_rev
-- [l17_rev::last_nonzero]: loop body 0:
@[rust_loop_body]
def last_nonzero_loop.body
  (s : Slice Std.U32)
  (iter : core.iter.adapters.rev.Rev (core.ops.range.Range Std.Usize)) :
  Result (ControlFlow (core.iter.adapters.rev.Rev (core.ops.range.Range
    Std.Usize)) (Option Std.Usize))
  := do
  let (o, iter1) ←
    core.iter.adapters.rev.Rev.Insts.CoreIterTraitsIteratorIterator.next
      (core.ops.range.Range.Insts.DoubleEndedIterator
      core.iter.range.StepUsize) iter
  match o with
  | none => ok (done none)
  | some i =>
    let i1 ← Slice.index_usize s i
    if i1 != 0#u32
    then ok (done o)
    else ok (cont iter1)
-- [l17_rev::last_nonzero]: loop 0:
@[rust_loop]
def last_nonzero_loop
  (iter : core.iter.adapters.rev.Rev (core.ops.range.Range Std.Usize))
  (s : Slice Std.U32) :
  Result (Option Std.Usize)
  := do
  loop
    (fun iter1 => last_nonzero_loop.body s iter1)
    iter
-- [l17_rev::last_nonzero]:
def last_nonzero
  (n : Std.Usize) (s : Slice Std.U32) : Result (Option Std.Usize) := do
  let iter ←
    core.iter.traits.iterator.Iterator.rev.trait_default
      (core.iter.traits.iterator.IteratorRange core.iter.range.StepUsize)
      (core.ops.range.Range.Insts.DoubleEndedIterator
      core.iter.range.StepUsize) { start := 0#usize, «end» := n }
  last_nonzero_loop iter s
end l17_rev
```

## L18_fold — CAVEATS

```rust
pub fn sum_fold(s: &[u32]) -> u32 {
    s.iter().fold(0, |acc, x| acc + x)
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l18_fold
-- [core::ops::arith::{impl core::ops::arith::Add<&'_0 u32, u32> for u32}::add]:
@[rust_fun "core::ops::arith::{core::ops::arith::Add<u32, &'0 u32, u32>}::add"]
axiom U32.Insts.CoreOpsArithAddShared0U32U32.add
  : Std.U32 → Std.U32 → Result Std.U32
-- [core::slice::iter::{impl core::iter::traits::iterator::Iterator<&'a T> for core::slice::iter::Iter<'a,
T>}::fold]:
@[rust_fun
  "core::slice::iter::{core::iter::traits::iterator::Iterator<core::slice::iter::Iter<'a, @T>, &'a @T>}::fold"]
axiom core.slice.iter.Iter.Insts.CoreIterTraitsIteratorIteratorSharedAT.fold
  {T : Type _} {B : Type _} {F : Type _} (opsfunctionFnMutFPairBSharedATBInst :
  core.ops.function.FnMut F (B × T) B) :
  core.slice.iter.Iter T → B → F → Result B
-- [l18_fold::sum_fold::{closure}]
@[reducible]
def sum_fold.closure := Unit
-- [l18_fold::sum_fold::{impl core::ops::function::FnMut<(u32, &'_0 u32), u32> for l18_fold::sum_fold::{closure}
<'_0>}::call_mut]:
def sum_fold.closure.Insts.CoreOpsFunctionFnMutPairU32Shared0U32U32.call_mut
  (c : sum_fold.closure) (tupled_args : (Std.U32 × Std.U32)) :
  Result (Std.U32 × sum_fold.closure)
  := do
  let (acc, x) := tupled_args
  let i ← U32.Insts.CoreOpsArithAddShared0U32U32.add acc x
  ok (i, ())
-- [l18_fold::sum_fold::{impl core::ops::function::FnOnce<(u32, &'_0 u32), u32> for l18_fold::sum_fold::
{closure}<'_0>}::call_once]:
def sum_fold.closure.Insts.CoreOpsFunctionFnOncePairU32Shared0U32U32.call_once
  (c : sum_fold.closure) (p : (Std.U32 × Std.U32)) : Result Std.U32 := do
  let (i, _) ←
    sum_fold.closure.Insts.CoreOpsFunctionFnMutPairU32Shared0U32U32.call_mut ()
      p
  ok i
-- Trait implementation: [l18_fold::sum_fold::{impl core::ops::function::FnOnce<(u32, &'_0 u32), u32> for
l18_fold::sum_fold::{closure}<'_0>}]
@[reducible]
def sum_fold.closure.Insts.CoreOpsFunctionFnOncePairU32Shared0U32U32 :
  core.ops.function.FnOnce sum_fold.closure (Std.U32 × Std.U32) Std.U32 := {
  call_once :=
    sum_fold.closure.Insts.CoreOpsFunctionFnOncePairU32Shared0U32U32.call_once
}
-- Trait implementation: [l18_fold::sum_fold::{impl core::ops::function::FnMut<(u32, &'_0 u32), u32> for
l18_fold::sum_fold::{closure}<'_0>}]
@[reducible]
def sum_fold.closure.Insts.CoreOpsFunctionFnMutPairU32Shared0U32U32 :
  core.ops.function.FnMut sum_fold.closure (Std.U32 × Std.U32) Std.U32 := {
  FnOnceInst :=
    sum_fold.closure.Insts.CoreOpsFunctionFnOncePairU32Shared0U32U32
  call_mut :=
    sum_fold.closure.Insts.CoreOpsFunctionFnMutPairU32Shared0U32U32.call_mut
}
-- [l18_fold::sum_fold]:
def sum_fold (s : Slice Std.U32) : Result Std.U32 := do
  let i ← core.slice.Slice.iter s
  core.slice.iter.Iter.Insts.CoreIterTraitsIteratorIteratorSharedAT.fold
    sum_fold.closure.Insts.CoreOpsFunctionFnMutPairU32Shared0U32U32 i 0#u32 ()
end l18_fold
```

## L19_sum — CAVEATS

```rust
pub fn sum_sum(s: &[u32]) -> u32 {
    s.iter().sum()
```

```
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'sum'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l19_sum
-- [core::iter::traits::accum::{impl core::iter::traits::accum::Sum<&'a u32> for u32}::sum]:
@[rust_fun
  "core::iter::traits::accum::{core::iter::traits::accum::Sum<u32, &'a u32>}::sum"]
axiom U32.Insts.CoreIterTraitsAccumSumSharedAU32.sum
  {I : Type _} (iteratorIteratorISharedAU32Inst :
  core.iter.traits.iterator.Iterator I Std.U32) :
  I → Result Std.U32
-- Trait implementation: [core::iter::traits::accum::{impl core::iter::traits::accum::Sum<&'a u32> for u32}]
@[reducible, rust_trait_impl "core::iter::traits::accum::Sum<u32, &'a u32>"]
def U32.Insts.CoreIterTraitsAccumSumSharedAU32 : core.iter.traits.accum.Sum
  Std.U32 Std.U32 := {
  sum := fun {I : Type} (iteratorIteratorPSharedU32Inst :
    core.iter.traits.iterator.Iterator I Std.U32) =>
    U32.Insts.CoreIterTraitsAccumSumSharedAU32.sum
    iteratorIteratorPSharedU32Inst
}
-- [core::iter::traits::iterator::Iterator::sum]:
@[trait_default, rust_fun "core::iter::traits::iterator::Iterator::sum"]
axiom core.iter.traits.iterator.Iterator.sum.default
  {Self : Type _} {S : Type _} {Clause0_Item : Type _} (IteratorInst :
  core.iter.traits.iterator.Iterator Self Clause0_Item) (accumSumInst :
  core.iter.traits.accum.Sum S Clause0_Item) :
  Self → Result S
-- [l19_sum::sum_sum]:
def sum_sum (s : Slice Std.U32) : Result Std.U32 := do
  let i ← core.slice.Slice.iter s
  core.iter.traits.iterator.Iterator.sum.default
    (core.iter.traits.iterator.IteratorSliceIter Std.U32)
    U32.Insts.CoreIterTraitsAccumSumSharedAU32 i
end l19_sum
```

## L20_collect — CLEAN

```rust
pub fn range_vec(n: u32) -> Vec<u32> {
    (0..n).collect()
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'collect'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'backward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
```

Generated Lean:

```
namespace l20_collect
-- [l20_collect::range_vec]:
def range_vec (n : Std.U32) : Result (alloc.vec.Vec Std.U32) := do
  core.iter.traits.iterator.Iterator.collect.default
    (core.iter.traits.iterator.IteratorRange core.iter.range.StepU32)
```

```
    (core.iter.traits.collect.FromIteratorVec Std.U32)
    { start := 0#u32, «end» := n }
end l20_collect
```

## L21_closure_capture_iter — FAIL

```rust
pub fn add_k(s: &[u32], k: u32) -> Vec<u32> {
    s.iter().map(|x| x + k).collect()
}
pub fn any_gt(s: &[u32], k: u32) -> bool {
    s.iter().any(|x| *x > k)
}
```

Exit codes: charon=0, aeneas=0, lean=1.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'map'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'collect'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::traits::iterator::Iterator', could
not find the information for item 'any'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Lean errors:

```
../out/L21_closure_capture_iter/lean/L21ClosureCaptureIter.lean:77:2: error: `map` is not a field of structure
`core.iter.traits.iterator.Iterator`
../out/L21_closure_capture_iter/lean/L21ClosureCaptureIter.lean:83:2: error: `collect` is not a field of
structure `core.iter.traits.iterator.Iterator`
../out/L21_closure_capture_iter/lean/L21ClosureCaptureIter.lean:89:2: error: `any` is not a field of structure
`core.iter.traits.iterator.Iterator`
```

Generated Lean:

```
namespace l21_closure_capture_iter
-- [core::iter::traits::iterator::Iterator::any]:
@[trait_default, rust_fun "core::iter::traits::iterator::Iterator::any"]
axiom core.iter.traits.iterator.Iterator.any.default
  {Self : Type _} {F : Type _} {Clause0_Item : Type _} (IteratorInst :
  core.iter.traits.iterator.Iterator Self Clause0_Item)
  (opsfunctionFnMutFTupleClause0_ItemBoolInst : core.ops.function.FnMut F
  Clause0_Item Bool) :
  Self → F → Result (Bool × Self)
-- [core::iter::traits::iterator::Iterator::map]:
@[trait_default, rust_fun "core::iter::traits::iterator::Iterator::map"]
axiom core.iter.traits.iterator.Iterator.map.default
  {Self : Type _} {B : Type _} {F : Type _} {Clause0_Item : Type _}
  (IteratorInst : core.iter.traits.iterator.Iterator Self Clause0_Item)
  (opsfunctionFnMutFTupleClause0_ItemBInst : core.ops.function.FnMut F
  Clause0_Item B) :
  Self → F → Result (core.iter.adapters.map.Map Self F)
-- [core::iter::adapters::map::{impl core::iter::traits::iterator::Iterator<B> for
core::iter::adapters::map::Map<I, F>}::next]:
@[rust_fun
  "core::iter::adapters::map::{core::iter::traits::iterator::Iterator<core::iter::adapters::map::Map<@I, @F>,
@B>}::next"]
axiom core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator.next
  {B : Type _} {I : Type _} {F : Type _} {Clause0_Item : Type _}
  (traitsiteratorIteratorInst : core.iter.traits.iterator.Iterator I
  Clause0_Item) (opsfunctionFnMutFTupleClause0_ItemBInst :
  core.ops.function.FnMut F Clause0_Item B) :
  core.iter.adapters.map.Map I F → Result ((Option B) ×
    (core.iter.adapters.map.Map I F))
-- Trait implementation: [core::iter::adapters::map::{impl core::iter::traits::iterator::Iterator<B> for
core::iter::adapters::map::Map<I, F>}]
@[reducible, rust_trait_impl
  "core::iter::traits::iterator::Iterator<core::iter::adapters::map::Map<@I, @F>, @B>"]
impl_def core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator {B :
```

```
  Type _} {I : Type _} {F : Type _} {Clause0_Item : Type _}
  (traitsiteratorIteratorInst : core.iter.traits.iterator.Iterator I
  Clause0_Item) (opsfunctionFnMutFTupleClause0_ItemBInst :
  core.ops.function.FnMut F Clause0_Item B) :
  core.iter.traits.iterator.Iterator (core.iter.adapters.map.Map I F) B := {
  next := core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator.next
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst
  map := fun {B1 : Type} {F1 : Type} (opsfunctionFnMutPTupleBPInst :
    core.ops.function.FnMut F1 B B1) =>
    core.iter.traits.iterator.Iterator.map.default
    (core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst)
    opsfunctionFnMutPTupleBPInst
  collect := fun {B1 : Type} (collectFromIteratorInst :
    core.iter.traits.collect.FromIterator B1 B) =>
    core.iter.traits.iterator.Iterator.collect.default
    (core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst)
    collectFromIteratorInst
  any := fun {F1 : Type} (opsfunctionFnMutPTupleFBoolInst :
    core.ops.function.FnMut F1 B Bool) =>
    core.iter.traits.iterator.Iterator.any.default
    (core.iter.adapters.map.Map.Insts.CoreIterTraitsIteratorIterator
    traitsiteratorIteratorInst opsfunctionFnMutFTupleClause0_ItemBInst)
    opsfunctionFnMutPTupleFBoolInst
}
-- [core::slice::iter::{impl core::iter::traits::iterator::Iterator<&'a T> for core::slice::iter::Iter<'a,
T>}::any]:
@[rust_fun
  "core::slice::iter::{core::iter::traits::iterator::Iterator<core::slice::iter::Iter<'a, @T>, &'a @T>}::any"]
axiom core.slice.iter.Iter.Insts.CoreIterTraitsIteratorIteratorSharedAT.any
  {T : Type _} {F : Type _} (opsfunctionFnMutFTupleSharedATBoolInst :
  core.ops.function.FnMut F T Bool) :
  core.slice.iter.Iter T → F → Result (Bool × (core.slice.iter.Iter T))
-- [l21_closure_capture_iter::add_k::{closure}]
@[reducible]
def add_k.closure := Std.U32
-- [l21_closure_capture_iter::add_k::{impl core::ops::function::FnMut<(&'_0 u32,), u32> for
l21_closure_capture_iter::add_k::{closure}<'_0, '_1>}::call_mut]:
def add_k.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32.call_mut
  (c : add_k.closure) (tupled_args : Std.U32) :
  Result (Std.U32 × add_k.closure)
  := do
  let i ← SharedU32.Insts.CoreOpsArithAddU32U32.add tupled_args c
  ok (i, c)
-- [l21_closure_capture_iter::add_k::{impl core::ops::function::FnOnce<(&'_0 u32,), u32> for
l21_closure_capture_iter::add_k::{closure}<'_0, '_1>}::call_once]:
def add_k.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32.call_once
  (c : add_k.closure) (i : Std.U32) : Result Std.U32 := do
  let (i1, _) ←
    add_k.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32.call_mut c i
  ok i1
-- Trait implementation: [l21_closure_capture_iter::add_k::{impl core::ops::function::FnOnce<(&'_0 u32,), u32>
for l21_closure_capture_iter::add_k::{closure}<'_0, '_1>}]
@[reducible]
def add_k.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32 :
  core.ops.function.FnOnce add_k.closure Std.U32 Std.U32 := {
  call_once :=
    add_k.closure.Insts.CoreOpsFunctionFnOnceTupleShared0U32U32.call_once
}
-- Trait implementation: [l21_closure_capture_iter::add_k::{impl core::ops::function::FnMut<(&'_0 u32,), u32>
for l21_closure_capture_iter::add_k::{closure}<'_0, '_1>}]
@[reducible]
def add_k.closure.Insts.CoreOpsFunctionFnMutTupleShared0U32U32 :
-- ... (56 more lines, see experiments/out/)
```

## L22_loop_mut_borrow — CLEAN

```rust
pub fn zero_out(v: &mut Vec<u32>) {
    let mut i = 0;
    while i < v.len() {
        v[i] = 0;
        i += 1;
    }
}
pub struct Counter { pub n: u32 }
pub fn bump(c: &mut Counter, k: u32) {
    let mut i = 0;
    while i < k {
```

```
        c.n = c.n.wrapping_add(1);
        i += 1;
    }
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l22_loop_mut_borrow
-- [l22_loop_mut_borrow::zero_out]: loop body 0:
@[rust_loop_body]
def zero_out_loop.body
  (v : alloc.vec.Vec Std.U32) (i : Std.Usize) :
  Result (ControlFlow ((alloc.vec.Vec Std.U32) × Std.Usize) (alloc.vec.Vec
    Std.U32))
  := do
  let i1 := alloc.vec.Vec.len v
  if i < i1
  then
    let (_, index_mut_back) ←
      alloc.vec.Vec.index_mut (core.slice.index.SliceIndexUsizeSlice Std.U32) v
        i
    let i2 ← i + 1#usize
    let v1 := index_mut_back 0#u32
    ok (cont (v1, i2))
  else ok (done v)
-- [l22_loop_mut_borrow::zero_out]: loop 0:
@[rust_loop]
def zero_out_loop
  (v : alloc.vec.Vec Std.U32) (i : Std.Usize) :
  Result (alloc.vec.Vec Std.U32)
  := do
  loop
    (fun (v1, i1) => zero_out_loop.body v1 i1)
    (v, i)
-- [l22_loop_mut_borrow::zero_out]:
@[reducible]
def zero_out (v : alloc.vec.Vec Std.U32) : Result (alloc.vec.Vec Std.U32) := do
  zero_out_loop v 0#usize
-- [l22_loop_mut_borrow::Counter]
structure Counter where
  n : Std.U32
-- [l22_loop_mut_borrow::bump]: loop body 0:
@[rust_loop_body]
def bump_loop.body
  (k : Std.U32) (c : Counter) (i : Std.U32) :
  Result (ControlFlow (Counter × Std.U32) Counter)
  := do
  if i < k
  then
    let i1 ← lift (core.num.U32.wrapping_add c.n 1#u32)
    let i2 ← i + 1#u32
    ok (cont ({ n := i1 }, i2))
  else ok (done c)
-- [l22_loop_mut_borrow::bump]: loop 0:
@[rust_loop]
def bump_loop (c : Counter) (k : Std.U32) (i : Std.U32) : Result Counter := do
  loop
    (fun (c1, i1) => bump_loop.body k c1 i1)
    (c, i)
-- [l22_loop_mut_borrow::bump]:
@[reducible]
def bump (c : Counter) (k : Std.U32) : Result Counter := do
  bump_loop c k 0#u32
end l22_loop_mut_borrow
```

## L23_loop_return_borrow — CLEAN

```rust
pub enum List<T> {
    Cons(T, Box<List<T>>),
    Nil,
}
// Loop that walks a list and returns a mutable borrow into it.
pub fn nth_mut<T>(mut l: &mut List<T>, mut i: u32) -> Option<&mut T> {
    while let List::Cons(x, tl) = l {
        if i == 0 {
```

```
            return Some(x);
        }
        i -= 1;
        l = tl;
    }
    None
}
// Loop that returns a shared borrow into a slice.
pub fn first_ge(s: &[u32], k: u32) -> Option<&u32> {
    let mut i = 0;
    while i < s.len() {
        if s[i] >= k {
            return Some(&s[i]);
        }
        i += 1;
    }
    None
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l23_loop_return_borrow
-- [l23_loop_return_borrow::List]
@[discriminant isize]
inductive List (T : Type _) where
| Cons : T → List T → List T
| Nil : List T
-- [l23_loop_return_borrow::nth_mut]: loop 0:
@[rust_loop]
def nth_mut_loop
  {T : Type _} (l : List T) (i : Std.U32) :
  Result ((Option T) × (Option T → List T))
  := do
  match l with
  | List.Cons x tl =>
    if i = 0#u32
    then
      ok (some x,
        fun o => let t := match o with
                          | some t1 => t1
                          | _ => x
                 List.Cons t tl)
    else
      let i1 ← i - 1#u32
      let (o, back) ← nth_mut_loop tl i1
      let back1 := fun o1 => let l1 := back o1
                             List.Cons x l1
      ok (o, back1)
  | List.Nil => ok (none, fun o => List.Nil)
partial_fixpoint
-- [l23_loop_return_borrow::nth_mut]:
@[reducible]
def nth_mut
  {T : Type _} (l : List T) (i : Std.U32) :
  Result ((Option T) × (Option T → List T))
  := do
  nth_mut_loop l i
-- [l23_loop_return_borrow::first_ge]: loop body 0:
@[rust_loop_body]
def first_ge_loop.body
  (s : Slice Std.U32) (k : Std.U32) (i : Std.Usize) :
  Result (ControlFlow Std.Usize (Option Std.U32))
  := do
  let i1 := Slice.len s
  if i < i1
  then
    let i2 ← Slice.index_usize s i
    if i2 >= k
    then ok (done (some i2))
    else let i3 ← i + 1#usize
         ok (cont i3)
  else ok (done none)
-- [l23_loop_return_borrow::first_ge]: loop 0:
@[rust_loop]
def first_ge_loop
  (s : Slice Std.U32) (k : Std.U32) (i : Std.Usize) :
  Result (Option Std.U32)
```

```
  := do
  loop
    (fun i1 => first_ge_loop.body s k i1)
    i
-- [l23_loop_return_borrow::first_ge]:
@[reducible]
def first_ge (s : Slice Std.U32) (k : Std.U32) : Result (Option Std.U32) := do
  first_ge_loop s k 0#usize
end l23_loop_return_borrow
```

## L24_while_let_vec_pop — CAVEATS

```
pub fn drain_sum(mut v: Vec<u32>) -> u32 {
    let mut t: u32 = 0;
    while let Some(x) = v.pop() {
        t = t.wrapping_add(x);
    }
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l24_while_let_vec_pop
-- [alloc::vec::{alloc::vec::Vec<T>}::pop]:
@[rust_fun "alloc::vec::{alloc::vec::Vec<@T>}::pop"]
axiom alloc.vec.Vec.pop
  {T : Type _} (A : Type _) :
  alloc.vec.Vec T → Result ((Option T) × (alloc.vec.Vec T))
-- [l24_while_let_vec_pop::drain_sum]: loop body 0:
@[rust_loop_body]
def drain_sum_loop.body
  (v : alloc.vec.Vec Std.U32) (t : Std.U32) :
  Result (ControlFlow ((alloc.vec.Vec Std.U32) × Std.U32) Std.U32)
  := do
  let (o, v1) ← alloc.vec.Vec.pop alloc.alloc.Global v
  match o with
  | none => ok (done t)
  | some x =>
    let t1 ← lift (core.num.U32.wrapping_add t x)
    ok (cont (v1, t1))
-- [l24_while_let_vec_pop::drain_sum]: loop 0:
@[rust_loop]
def drain_sum_loop
  (v : alloc.vec.Vec Std.U32) (t : Std.U32) : Result Std.U32 := do
  loop
    (fun (v1, t1) => drain_sum_loop.body v1 t1)
    (v, t)
-- [l24_while_let_vec_pop::drain_sum]:
@[reducible]
def drain_sum (v : alloc.vec.Vec Std.U32) : Result Std.U32 := do
  drain_sum_loop v 0#u32
end l24_while_let_vec_pop
```

## L25_custom_iterator — CLEAN

```
pub struct Countdown { pub n: u32 }
impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        if self.n == 0 { None } else { self.n -= 1; Some(self.n) }
    }
}
pub fn total(n: u32) -> u32 {
    let mut t: u32 = 0;
    for x in (Countdown { n }) {
        t = t.wrapping_add(x);
    }
```

```
    t
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l25_custom_iterator
-- [l25_custom_iterator::Countdown]
structure Countdown where
  n : Std.U32
-- [l25_custom_iterator::{impl core::iter::traits::iterator::Iterator<u32> for
l25_custom_iterator::Countdown}::next]:
def Countdown.Insts.CoreIterTraitsIteratorIteratorU32.next
  (self : Countdown) : Result ((Option Std.U32) × Countdown) := do
  if self.n = 0#u32
  then ok (none, self)
  else let i ← self.n - 1#u32
       ok (some i, { n := i })
-- Trait implementation: [l25_custom_iterator::{impl core::iter::traits::iterator::Iterator<u32> for
l25_custom_iterator::Countdown}]
@[reducible]
def Countdown.Insts.CoreIterTraitsIteratorIteratorU32 :
  core.iter.traits.iterator.Iterator Countdown Std.U32 := {
  next := Countdown.Insts.CoreIterTraitsIteratorIteratorU32.next
}
-- [l25_custom_iterator::total]: loop body 0:
@[rust_loop_body]
def total_loop.body
  (iter : Countdown) (t : Std.U32) :
  Result (ControlFlow (Countdown × Std.U32) Std.U32)
  := do
  let (o, iter1) ←
    Countdown.Insts.CoreIterTraitsIteratorIteratorU32.next iter
  match o with
  | none => ok (done t)
  | some x =>
    let t1 ← lift (core.num.U32.wrapping_add t x)
    ok (cont (iter1, t1))
-- [l25_custom_iterator::total]: loop 0:
@[rust_loop]
def total_loop (iter : Countdown) (t : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, t1) => total_loop.body iter1 t1)
    (iter, t)
-- [l25_custom_iterator::total]:
@[reducible]
def total (n : Std.U32) : Result Std.U32 := do
  total_loop { n } 0#u32
end l25_custom_iterator
```

## L26_infinite_loop — FAIL

```
pub fn spin() -> ! {
    loop {}
}
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] (Infinite) loops which do not contain breaks are not supported yet
[Warn] Could not translate the body of function 'l26_infinite_loop::spin because of previous error
```

Generated Lean:

```
namespace l26_infinite_loop
-- [l26_infinite_loop::spin]:
def spin : Result Never := do
  sorry
end l26_infinite_loop
```

## L27_loop_exit_only_return — CLEAN

```rust
pub fn find_zero(s: &[u32]) -> usize {
    let mut i = 0;
    loop {
        if i >= s.len() || s[i] == 0 {
            return i;
        }
        i += 1;
    }
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l27_loop_exit_only_return
-- [l27_loop_exit_only_return::find_zero]: loop body 0:
@[rust_loop_body]
def find_zero_loop.body
  (s : Slice Std.U32) (i : Std.Usize) :
  Result (ControlFlow Std.Usize Std.Usize)
  := do
  let i1 := Slice.len s
  if i >= i1
  then ok (done i)
  else
    let i2 ← Slice.index_usize s i
    if i2 = 0#u32
    then ok (done i)
    else let i3 ← i + 1#usize
         ok (cont i3)
-- [l27_loop_exit_only_return::find_zero]: loop 0:
@[rust_loop]
def find_zero_loop (s : Slice Std.U32) (i : Std.Usize) : Result Std.Usize := do
  loop
    (fun i1 => find_zero_loop.body s i1)
    i
-- [l27_loop_exit_only_return::find_zero]:
@[reducible]
def find_zero (s : Slice Std.U32) : Result Std.Usize := do
  find_zero_loop s 0#usize
end l27_loop_exit_only_return
```

## L28_return_and_break_then_code — CLEAN

```rust
pub fn f(s: &[u32]) -> u32 {
    let mut i = 0;
    let mut r = 0;
    while i < s.len() {
        if s[i] == 0 {
            return 0;
        }
        if s[i] == 1 {
            break;
        }
        r += s[i];
        i += 1;
    }
    r + 1
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace l28_return_and_break_then_code
-- [l28_return_and_break_then_code::f]: loop body 0:
@[rust_loop_body]
def f_loop.body
  (s : Slice Std.U32) (i : Std.Usize) (r : Std.U32) :
  Result (ControlFlow (Std.Usize × Std.U32) Std.U32)
  := do
  let i1 := Slice.len s
  if i < i1
  then
    let i2 ← Slice.index_usize s i
```

```
    if i2 = 0#u32
    then ok (done 0#u32)
    else
      if i2 = 1#u32
      then let i3 ← r + 1#u32
           ok (done i3)
      else let r1 ← r + i2
           let i3 ← i + 1#usize
           ok (cont (i3, r1))
  else let i2 ← r + 1#u32
       ok (done i2)
-- [l28_return_and_break_then_code::f]: loop 0:
@[rust_loop]
def f_loop
  (s : Slice Std.U32) (i : Std.Usize) (r : Std.U32) : Result Std.U32 := do
  loop
    (fun (i1, r1) => f_loop.body s i1 r1)
    (i, r)
-- [l28_return_and_break_then_code::f]:
@[reducible]
def f (s : Slice Std.U32) : Result Std.U32 := do
  f_loop s 0#usize 0#u32
end l28_return_and_break_then_code
```

## L29_two_loops_with_return — FAIL

```rust
pub fn f(s: &[u32]) -> Option<usize> {
    let mut i = 0;
    while i < s.len() {
        if s[i] == 0 {
            return Some(i);
        }
        i += 1;
    }
    let mut j = 0;
    while j < s.len() {
        if s[j] == 1 {
            return Some(j);
        }
        j += 1;
    }
    None
}
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Breaks to outer loops are not supported yet
[Error] Ignoring the body of 'l29_two_loops_with_return::f' because of previous error
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace l29_two_loops_with_return
-- [l29_two_loops_with_return::f]:
axiom f : Slice Std.U32 → Result (Option Std.Usize)
end l29_two_loops_with_return
```

## L30_for_range_count — CLEAN

```rust
pub fn count_range(n: u32) -> u32 {
    let mut c = 0;
    for _i in 0..n {
        c += 1;
    }
    c
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'forward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
[Warn] When retrieving the builtin information for trait decl 'core::iter::range::Step', could not find the
information for item 'backward_overflowing'. The model defined in the Lean library seems to be missing the
corresponding field.
```

Generated Lean:

```
namespace l30_for_range_count
-- [l30_for_range_count::count_range]: loop body 0:
@[rust_loop_body]
def count_range_loop.body
  (iter : core.ops.range.Range Std.U32) (c : Std.U32) :
  Result (ControlFlow ((core.ops.range.Range Std.U32) × Std.U32) Std.U32)
  := do
  let (o, iter1) ←
    core.iter.range.IteratorRange.next core.iter.range.StepU32 iter
  match o with
  | none => ok (done c)
  | some _ => let c1 ← c + 1#u32
              ok (cont (iter1, c1))
-- [l30_for_range_count::count_range]: loop 0:
@[rust_loop]
def count_range_loop
  (iter : core.ops.range.Range Std.U32) (c : Std.U32) : Result Std.U32 := do
  loop
    (fun (iter1, c1) => count_range_loop.body iter1 c1)
    (iter, c)
-- [l30_for_range_count::count_range]:
@[reducible]
def count_range (n : Std.U32) : Result Std.U32 := do
  count_range_loop { start := 0#u32, «end» := n } 0#u32
end l30_for_range_count
```

# Appendix B — Other feature experiments

### A01_arith — CLEAN

```
pub fn add(a: u32, b: u32) -> u32 { a + b }
pub fn sub(a: i32, b: i32) -> i32 { a - b }
pub fn mul(a: u64, b: u64) -> u64 { a * b }
pub fn div(a: u8, b: u8) -> u8 { a / b }
pub fn rem(a: i16, b: i16) -> i16 { a % b }
pub fn neg(a: i32) -> i32 { -a }
pub fn big(a: u128, b: i128) -> u128 { a + (b as u128) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace a01_arith
-- [a01_arith::add]:
def add (a : Std.U32) (b : Std.U32) : Result Std.U32 := do
  a + b
-- [a01_arith::sub]:
def sub (a : Std.I32) (b : Std.I32) : Result Std.I32 := do
  a - b
-- [a01_arith::mul]:
def mul (a : Std.U64) (b : Std.U64) : Result Std.U64 := do
  a * b
-- [a01_arith::div]:
def div (a : Std.U8) (b : Std.U8) : Result Std.U8 := do
  a / b
-- [a01_arith::rem]:
def rem (a : Std.I16) (b : Std.I16) : Result Std.I16 := do
  a % b
-- [a01_arith::neg]:
def neg (a : Std.I32) : Result Std.I32 := do
  -. a
-- [a01_arith::big]:
def big (a : Std.U128) (b : Std.I128) : Result Std.U128 := do
  let i ← lift (IScalar.hcast .U128 b)
```

```
  a + i
end a01_arith
```

## A02_wrapping_checked — CLEAN

```rust
pub fn w(a: u32, b: u32) -> u32 { a.wrapping_add(b).wrapping_mul(3) }
pub fn c(a: u32, b: u32) -> Option<u32> { a.checked_add(b) }
pub fn o(a: u8, b: u8) -> (u8, bool) { a.overflowing_add(b) }
pub fn s(a: u8, b: u8) -> u8 { a.saturating_sub(b) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace a02_wrapping_checked
-- [a02_wrapping_checked::w]:
def w (a : Std.U32) (b : Std.U32) : Result Std.U32 := do
  let i ← lift (core.num.U32.wrapping_add a b)
  ok (core.num.U32.wrapping_mul i 3#u32)
-- [a02_wrapping_checked::c]:
def c (a : Std.U32) (b : Std.U32) : Result (Option Std.U32) := do
  ok (U32.checked_add a b)
-- [a02_wrapping_checked::o]:
def o (a : Std.U8) (b : Std.U8) : Result (Std.U8 × Bool) := do
  ok (core.num.U8.overflowing_add a b)
-- [a02_wrapping_checked::s]:
def s (a : Std.U8) (b : Std.U8) : Result Std.U8 := do
  ok (core.num.U8.saturating_sub a b)
end a02_wrapping_checked
```

## A03_casts — FAIL

```rust
pub fn c1(x: u32) -> u8 { x as u8 }
pub fn c2(x: i32) -> u64 { x as u64 }
pub fn c3(b: bool) -> u8 { b as u8 }
pub fn c4(x: u8) -> char { x as char }
pub fn c5(x: u16) -> u32 { u32::from(x) }
pub fn c6(x: u32) -> Result<u8, core::num::TryFromIntError> { u8::try_from(x) }
```

Exit codes: charon=0, aeneas=1, lean=1.

Aeneas messages:

```
[Error] Unreachable
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Lean errors:

```
../out/A03_casts/lean/A03Casts.lean:58:6: error: unexpected token '/--'; expected ')', '↑', '↥', '⇑' or term
```

Generated Lean:

```
namespace a03_casts
-- [core::convert::num::{impl core::convert::TryFrom<u32, core::num::error::TryFromIntError> for u8}::try_from]:
@[rust_fun
  "core::convert::num::{core::convert::TryFrom<u8, u32, core::num::error::TryFromIntError>}::try_from"]
axiom U8.Insts.CoreConvertTryFromU32TryFromIntError.try_from
  :
  Std.U32 → Result (core.result.Result Std.U8 core.num.error.TryFromIntError)
-- [a03_casts::c1]:
def c1 (x : Std.U32) : Result Std.U8 := do
  ok (UScalar.cast .U8 x)
-- [a03_casts::c2]:
def c2 (x : Std.I32) : Result Std.U64 := do
  ok (IScalar.hcast .U64 x)
-- [a03_casts::c3]:
def c3 (b : Bool) : Result Std.U8 := do
  ok (UScalar.cast_fromBool .U8 b)
-- [a03_casts::c4]:
def c4 (x : Std.U8) : Result Char := do
```

```
  ok (
    -- [a03_casts::c5]:
    def c5 (x : Std.U16) : Result Std.U32 := do
      ok (core.convert.num.FromU32U16.from x)
    -- [a03_casts::c6]:
    def c6
      (x : Std.U32) :
      Result (core.result.Result Std.U8 core.num.error.TryFromIntError)
      := do
      U8.Insts.CoreConvertTryFromU32TryFromIntError.try_from x
end a03_casts
```

## A04_bits — CAVEATS

```rust
pub fn b(a: u32, b: u32) -> u32 { (a & b) | (a ^ b) | !a }
pub fn sh(a: u32, k: u32) -> u32 { (a << k) >> 1 }
pub fn rot(a: u32) -> u32 { a.rotate_left(3) }
pub fn cnt(a: u64) -> u32 { a.count_ones() }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace a04_bits
-- [core::num::{u64}::count_ones]:
@[rust_fun "core::num::{u64}::count_ones"]
axiom core.num.U64.count_ones : Std.U64 → Result Std.U32
-- [a04_bits::b]:
def b (a : Std.U32) (b1 : Std.U32) : Result Std.U32 := do
  let i ← lift (a &&& b1)
  let i1 ← lift (a ^^^ b1)
  let i2 ← lift (i ||| i1)
  let i3 ← lift (~~~ a)
  ok (i2 ||| i3)
-- [a04_bits::sh]:
def sh (a : Std.U32) (k : Std.U32) : Result Std.U32 := do
  let i ← a <<< k
  i >>> 1#i32
-- [a04_bits::rot]:
def rot (a : Std.U32) : Result Std.U32 := do
  ok (core.num.U32.rotate_left a 3#u32)
-- [a04_bits::cnt]:
def cnt (a : Std.U64) : Result Std.U32 := do
  core.num.U64.count_ones a
end a04_bits
```

## A05_panics — CLEAN

```rust
pub fn p(x: u32) -> u32 { if x == 0 { panic!("zero") } x }
pub fn a(x: u32) -> u32 { assert!(x > 1); x }
pub fn ae(x: u32) { assert_eq!(x, 1) }
pub fn u(x: Option<u32>) -> u32 { x.unwrap() }
pub fn e(x: Option<u32>) -> u32 { x.expect("none") }
pub fn un() -> u32 { unreachable!() }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace a05_panics
-- [a05_panics::p]:
def p (x : Std.U32) : Result Std.U32 := do
  massert (¬ (x = 0#u32))
  ok x
-- [a05_panics::a]:
def a (x : Std.U32) : Result Std.U32 := do
  massert (x > 1#u32)
  ok x
```

```
-- [a05_panics::ae]:
def ae (x : Std.U32) : Result Unit := do
  massert (x = 1#u32)
-- [a05_panics::u]:
def u (x : Option Std.U32) : Result Std.U32 := do
  core.option.Option.unwrap x
-- [a05_panics::e]:
def e (x : Option Std.U32) : Result Std.U32 := do
  core.option.Option.expect x (toStr "none")
-- [a05_panics::un]:
def un : Result Std.U32 := do
  fail panic
end a05_panics
```

### A06_floats — FAIL

```rust
pub fn f(a: f64, b: f64) -> f64 { a * b + 1.0 }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Invalid inputs for binop
[Warn] Could not translate the body of function 'a06_floats::f because of previous error
```

Generated Lean:

```
namespace a06_floats
-- [a06_floats::f]:
def f (a : F64) (b : F64) : Result F64 := do
  sorry
end a06_floats
```

### B01_nested_borrows — CLEAN

```rust
pub fn incr_nested(x: &mut &mut u32) { **x += 1; }
pub fn use_nested() -> u32 { let mut a = 0; let mut r = &mut a; incr_nested(&mut r); a }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace b01_nested_borrows
-- [b01_nested_borrows::incr_nested]:
def incr_nested (x : Std.U32) : Result (Std.U32 × (Std.U32 → Std.U32)) := do
  let x1 ← x + 1#u32
  ok (x1, fun x2 => x2)
-- [b01_nested_borrows::use_nested]:
def use_nested : Result Std.U32 := do
  let (a, incr_nested_back) ← incr_nested 0#u32
  ok (incr_nested_back a)
end b01_nested_borrows
```

### B02_struct_with_lifetime — CLEAN

```rust
pub struct View<'a> { pub data: &'a mut [u32], pub pos: usize }
impl<'a> View<'a> {
    pub fn set(&mut self, x: u32) { self.data[self.pos] = x; }
}
pub fn use_view(v: &mut [u32]) { let mut w = View { data: v, pos: 0 }; w.set(7); }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace b02_struct_with_lifetime
-- [b02_struct_with_lifetime::View]
structure View where
  data : Slice Std.U32
```

```
  pos : Std.Usize
-- [b02_struct_with_lifetime::{b02_struct_with_lifetime::View<'a>}::set]:
def View.set
  (self : View) (x : Std.U32) : Result (View × (View → View)) := do
  let s ← Slice.update self.data self.pos x
  let back'a := fun self1 => { self with data := self1.data }
  ok ({ self with data := s }, back'a)
-- [b02_struct_with_lifetime::use_view]:
def use_view (v : Slice Std.U32) : Result (Slice Std.U32) := do
  let (w, set_back) ← View.set { data := v, pos := 0#usize } 7#u32
  ok (set_back w).data
end b02_struct_with_lifetime
```

## B03_return_borrow — CLEAN

```
pub fn choose<'a>(b: bool, x: &'a mut u32, y: &'a mut u32) -> &'a mut u32 { if b { x } else { y } }
pub fn use_choose() -> (u32, u32) {
    let mut x = 0; let mut y = 0;
    { let r = choose(true, &mut x, &mut y); *r = 1; }
    (x, y)
}
pub fn get_mut_field(p: &mut (u32, u32)) -> &mut u32 { &mut p.1 }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace b03_return_borrow
-- [b03_return_borrow::choose]:
def choose
  (b : Bool) (x : Std.U32) (y : Std.U32) :
  Result (Std.U32 × (Std.U32 → (Std.U32 × Std.U32)))
  := do
  if b
  then let back := fun x1 => (x1, y)
       ok (x, back)
  else let back := fun y1 => (x, y1)
       ok (y, back)
-- [b03_return_borrow::use_choose]:
def use_choose : Result (Std.U32 × Std.U32) := do
  let (_, choose_back) ← choose true 0#u32 0#u32
  ok (choose_back 1#u32)
-- [b03_return_borrow::get_mut_field]:
def get_mut_field
  (p : (Std.U32 × Std.U32)) :
  Result (Std.U32 × (Std.U32 → (Std.U32 × Std.U32)))
  := do
  let (i, i1) := p
  let back := fun i2 => (i, i2)
  ok (i1, back)
end b03_return_borrow
```

## B04_generic_with_mut_ref — FAIL

```
pub fn id<T>(x: T) -> T { x }
pub fn use_id() -> u32 { let mut x = 0; *id(&mut x) += 1; x }
```

Exit codes: charon=0, aeneas=0, lean=1.

Lean errors:

```
../out/B04_generic_with_mut_ref/lean/B04GenericWithMutRef.lean:32:31: error: expected a product type, got
```

Generated Lean:

```
namespace b04_generic_with_mut_ref
-- [b04_generic_with_mut_ref::id]:
def id {T : Type _} (x : T) : Result T := do
  ok x
-- [b04_generic_with_mut_ref::use_id]:
def use_id : Result Std.U32 := do
  let (i, id_back) ← id 0#u32
  let i1 ← i + 1#u32
```

```
  ok (id_back i1)
end b04_generic_with_mut_ref
```

### B05_two_phase — CLEAN

```
pub fn push_len(v: &mut Vec<usize>) { v.push(v.len()); }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace b05_two_phase
-- [b05_two_phase::push_len]:
def push_len
  (v : alloc.vec.Vec Std.Usize) : Result (alloc.vec.Vec Std.Usize) := do
  let i := alloc.vec.Vec.len v
  alloc.vec.Vec.push v i
end b05_two_phase
```

### B06_option_mut_ref_in_struct — CLEAN

```
pub struct Node { pub v: u32, pub next: Option<Box<Node>> }
pub fn last_mut(n: &mut Node) -> &mut u32 {
    match &mut n.next { None => &mut n.v, Some(nx) => last_mut(nx) }
}
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace b06_option_mut_ref_in_struct
-- [b06_option_mut_ref_in_struct::Node]
inductive Node where
| mk : Std.U32 → Option Node → Node
def Node.v (x : Node) := match x with | Node.mk x1 _ => x1
def Node.next (x : Node) := match x with | Node.mk _ x1 => x1
@[simp]
theorem Node.v._simpLemma_ (v : Std.U32) (next : Option Node) :
  (Node.mk v next).v = v := by rfl
@[simp]
theorem Node.next._simpLemma_ (v : Std.U32) (next : Option Node) :
  (Node.mk v next).next = next := by rfl
-- [b06_option_mut_ref_in_struct::last_mut]:
def last_mut (n : Node) : Result (Std.U32 × (Std.U32 → Node)) := do
  match n.next with
  | none => let back := fun i => Node.mk i none
            ok (n.v, back)
  | some nx =>
    let (i, last_mut_back) ← last_mut nx
    let back := fun i1 => let nx1 := last_mut_back i1
                          Node.mk n.v (some nx1)
    ok (i, back)
partial_fixpoint
end b06_option_mut_ref_in_struct
```

### C01_closure_local — CLEAN

```
pub fn f(x: u32) -> u32 { let add1 = |y: u32| y + 1; add1(x) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace c01_closure_local
-- [c01_closure_local::f::{closure}]
@[reducible]
def f.closure := Unit
-- [c01_closure_local::f::{impl core::ops::function::Fn<(u32,), u32> for c01_closure_local::f::
{closure}}::call]:
def f.closure.Insts.CoreOpsFunctionFnTupleU32U32.call
  (c : f.closure) (tupled_args : Std.U32) : Result Std.U32 := do
```

```
  tupled_args + 1#u32
-- [c01_closure_local::f]:
def f (x : Std.U32) : Result Std.U32 := do
  f.closure.Insts.CoreOpsFunctionFnTupleU32U32.call () x
end c01_closure_local
```

## C02_closure_generic_fn — CLEAN

```
pub fn apply<F: Fn(u32) -> u32>(f: F, x: u32) -> u32 { f(x) }
pub fn use_apply(k: u32) -> u32 { apply(|y| y + k, 3) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace c02_closure_generic_fn
-- [c02_closure_generic_fn::apply]:
def apply
  {F : Type _} (coreopsfunctionFnFTupleU32U32Inst : core.ops.function.Fn F
  Std.U32 Std.U32) (f : F) (x : Std.U32) :
  Result Std.U32
  := do
  coreopsfunctionFnFTupleU32U32Inst.call f x
-- [c02_closure_generic_fn::use_apply::{closure}]
@[reducible]
def use_apply.closure := Std.U32
-- [c02_closure_generic_fn::use_apply::{impl core::ops::function::Fn<(u32,), u32> for
c02_closure_generic_fn::use_apply::{closure}<'_0>}::call]:
def use_apply.closure.Insts.CoreOpsFunctionFnTupleU32U32.call
  (c : use_apply.closure) (tupled_args : Std.U32) : Result Std.U32 := do
  tupled_args + c
-- [c02_closure_generic_fn::use_apply::{impl core::ops::function::FnMut<(u32,), u32> for
c02_closure_generic_fn::use_apply::{closure}<'_0>}::call_mut]:
def use_apply.closure.Insts.CoreOpsFunctionFnMutTupleU32U32.call_mut
  (state : use_apply.closure) (args : Std.U32) :
  Result (Std.U32 × use_apply.closure)
  := do
  let i ←
    use_apply.closure.Insts.CoreOpsFunctionFnTupleU32U32.call state args
  ok (i, state)
-- [c02_closure_generic_fn::use_apply::{impl core::ops::function::FnOnce<(u32,), u32> for
c02_closure_generic_fn::use_apply::{closure}<'_0>}::call_once]:
def use_apply.closure.Insts.CoreOpsFunctionFnOnceTupleU32U32.call_once
  (c : use_apply.closure) (i : Std.U32) : Result Std.U32 := do
  let (i1, _) ←
    use_apply.closure.Insts.CoreOpsFunctionFnMutTupleU32U32.call_mut c i
  ok i1
-- Trait implementation: [c02_closure_generic_fn::use_apply::{impl core::ops::function::FnOnce<(u32,), u32> for
c02_closure_generic_fn::use_apply::{closure}<'_0>}]
@[reducible]
def use_apply.closure.Insts.CoreOpsFunctionFnOnceTupleU32U32 :
  core.ops.function.FnOnce use_apply.closure Std.U32 Std.U32 := {
  call_once :=
    use_apply.closure.Insts.CoreOpsFunctionFnOnceTupleU32U32.call_once
}
-- Trait implementation: [c02_closure_generic_fn::use_apply::{impl core::ops::function::FnMut<(u32,), u32> for
c02_closure_generic_fn::use_apply::{closure}<'_0>}]
@[reducible]
def use_apply.closure.Insts.CoreOpsFunctionFnMutTupleU32U32 :
  core.ops.function.FnMut use_apply.closure Std.U32 Std.U32 := {
  FnOnceInst := use_apply.closure.Insts.CoreOpsFunctionFnOnceTupleU32U32
  call_mut := use_apply.closure.Insts.CoreOpsFunctionFnMutTupleU32U32.call_mut
}
-- Trait implementation: [c02_closure_generic_fn::use_apply::{impl core::ops::function::Fn<(u32,), u32> for
c02_closure_generic_fn::use_apply::{closure}<'_0>}]
@[reducible]
-- ... (9 more lines, see experiments/out/)
```

## C03_closure_fnmut — FAIL

```
pub fn call_twice<F: FnMut()>(mut f: F) { f(); f(); }
pub fn count() -> u32 { let mut c = 0; call_twice(|| c += 1); c }
```

Exit codes: charon=0, aeneas=0, lean=1.

Lean errors:

```
../out/C03_closure_fnmut/lean/C03ClosureFnmut.lean:63:15: error: Type mismatch
../out/C03_closure_fnmut/lean/C03ClosureFnmut.lean:72:14: error: Type mismatch
../out/C03_closure_fnmut/lean/C03ClosureFnmut.lean:79:2: error: Type mismatch
```

Generated Lean:

```
namespace c03_closure_fnmut
-- [c03_closure_fnmut::call_twice]:
def call_twice
  {F : Type _} (coreopsfunctionFnMutFTupleTupleInst : core.ops.function.FnMut F
  Unit Unit) (f : F) :
  Result Unit
  := do
  let (_, f1) ← coreopsfunctionFnMutFTupleTupleInst.call_mut f ()
  let _ ← coreopsfunctionFnMutFTupleTupleInst.call_mut f1 ()
  ok ()
-- [c03_closure_fnmut::count::{closure}]
@[reducible]
def count.closure := Std.U32
-- [c03_closure_fnmut::count::{impl core::ops::function::FnMut<(), ()> for c03_closure_fnmut::count::{closure}
<'_0>}::call_mut]:
def count.closure.Insts.CoreOpsFunctionFnMutTupleTuple.call_mut
  (c : count.closure) (_ : Unit) :
  Result (count.closure × (count.closure → count.closure))
  := do
  let i ← c + 1#u32
  let back := fun c1 => c1
  ok (i, back)
-- [c03_closure_fnmut::count::{impl core::ops::function::FnOnce<(), ()> for c03_closure_fnmut::count::{closure}
<'_0>}::call_once]:
def count.closure.Insts.CoreOpsFunctionFnOnceTupleTuple.call_once
  (c : count.closure) (_ : Unit) : Result count.closure := do
  let (c1, call_mut_back) ←
    count.closure.Insts.CoreOpsFunctionFnMutTupleTuple.call_mut c ()
  ok (call_mut_back c1)
-- Trait implementation: [c03_closure_fnmut::count::{impl core::ops::function::FnOnce<(), ()> for
c03_closure_fnmut::count::{closure}<'_0>}]
@[reducible]
def count.closure.Insts.CoreOpsFunctionFnOnceTupleTuple :
  core.ops.function.FnOnce count.closure Unit Unit := {
  call_once := count.closure.Insts.CoreOpsFunctionFnOnceTupleTuple.call_once
}
-- Trait implementation: [c03_closure_fnmut::count::{impl core::ops::function::FnMut<(), ()> for
c03_closure_fnmut::count::{closure}<'_0>}]
@[reducible]
def count.closure.Insts.CoreOpsFunctionFnMutTupleTuple :
  core.ops.function.FnMut count.closure Unit Unit := {
  FnOnceInst := count.closure.Insts.CoreOpsFunctionFnOnceTupleTuple
  call_mut := count.closure.Insts.CoreOpsFunctionFnMutTupleTuple.call_mut
}
-- [c03_closure_fnmut::count]:
def count : Result Std.U32 := do
  call_twice count.closure.Insts.CoreOpsFunctionFnMutTupleTuple 0#u32
end c03_closure_fnmut
```

## C04_fn_pointer — FAIL

```rust
fn inc(x: u32) -> u32 { x + 1 }
pub fn apply_ptr(f: fn(u32) -> u32, x: u32) -> u32 { f(x) }
pub fn use_ptr() -> u32 { apply_ptr(inc, 1) }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Arrow types are not supported yet
[Warn] Could not translate the function signature of 'c04_fn_pointer::apply_ptr because of previous error
[Error] Invalid input for unop: fn_def_to_fn_ptr on c04_fn_pointer::inc
[Warn] Could not translate the body of function 'c04_fn_pointer::use_ptr because of previous error
[Error] Function pointers are not supported yet
[Warn] Could not translate the body of function 'c04_fn_pointer::apply_ptr because of previous error
```

```
[Error] Internal error: please file an issue
[Warn] Could not translate the function 'c04_fn_pointer::apply_ptr because of previous error.
```

Generated Lean:

```
namespace c04_fn_pointer
-- [c04_fn_pointer::inc]:
def inc (x : Std.U32) : Result Std.U32 := do
  x + 1#u32
-- [c04_fn_pointer::use_ptr]:
def use_ptr : Result Std.U32 := do
  sorry
end c04_fn_pointer
```

## C05_closure_fnonce_move — CLEAN

```
pub fn consume<F: FnOnce() -> Vec<u32>>(f: F) -> Vec<u32> { f() }
pub fn mk(v: Vec<u32>) -> Vec<u32> { consume(move || v) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace c05_closure_fnonce_move
-- [c05_closure_fnonce_move::consume]:
def consume
  {F : Type _} (coreopsfunctionFnOnceFTupleVecU32Inst :
  core.ops.function.FnOnce F Unit (alloc.vec.Vec Std.U32)) (f : F) :
  Result (alloc.vec.Vec Std.U32)
  := do
  coreopsfunctionFnOnceFTupleVecU32Inst.call_once f ()
-- [c05_closure_fnonce_move::mk::{closure}]
@[reducible]
def mk.closure := alloc.vec.Vec Std.U32
-- [c05_closure_fnonce_move::mk::{impl core::ops::function::FnOnce<(), alloc::vec::Vec<u32>> for
c05_closure_fnonce_move::mk::{closure}}::call_once]:
def mk.closure.Insts.CoreOpsFunctionFnOnceTupleVecU32.call_once
  (c : mk.closure) (_ : Unit) : Result (alloc.vec.Vec Std.U32) := do
  ok c
-- Trait implementation: [c05_closure_fnonce_move::mk::{impl core::ops::function::FnOnce<(),
alloc::vec::Vec<u32>> for c05_closure_fnonce_move::mk::{closure}}]
@[reducible]
def mk.closure.Insts.CoreOpsFunctionFnOnceTupleVecU32 :
  core.ops.function.FnOnce mk.closure Unit (alloc.vec.Vec Std.U32) := {
  call_once := mk.closure.Insts.CoreOpsFunctionFnOnceTupleVecU32.call_once
}
-- [c05_closure_fnonce_move::mk]:
def mk (v : alloc.vec.Vec Std.U32) : Result (alloc.vec.Vec Std.U32) := do
  consume mk.closure.Insts.CoreOpsFunctionFnOnceTupleVecU32 v
end c05_closure_fnonce_move
```

## D01_arrays — CLEAN

```
pub fn arr() -> u32 { let mut a = [1u32, 2, 3, 4]; a[2] = 10; a[0] + a[2] + (a.len() as u32) }
pub fn swap01(a: &mut [u32; 2]) { let t = a[0]; a[0] = a[1]; a[1] = t; }
pub fn arr2d(m: [[u8; 2]; 2]) -> u8 { m[1][0] }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace d01_arrays
-- [d01_arrays::arr]:
def arr : Result Std.U32 := do
  let a ←
    Array.update (Array.make 4#usize [ 1#u32, 2#u32, 3#u32, 4#u32 ] : Array
      Std.U32 4#usize) 2#usize 10#u32
  let i ← Array.index_usize a 0#usize
  let i1 ← Array.index_usize a 2#usize
  let i2 ← i + i1
  let s ← lift (Array.to_slice a)
  let i3 := Slice.len s
  let i4 ← lift (UScalar.cast .U32 i3)
```

```
  i2 + i4
-- [d01_arrays::swap01]:
def swap01 (a : Array Std.U32 2#usize) : Result (Array Std.U32 2#usize) := do
  let t ← Array.index_usize a 0#usize
  let i ← Array.index_usize a 1#usize
  let a1 ← Array.update a 0#usize i
  Array.update a1 1#usize t
-- [d01_arrays::arr2d]:
def arr2d (m : Array (Array Std.U8 2#usize) 2#usize) : Result Std.U8 := do
  let a ← Array.index_usize m 1#usize
  Array.index_usize a 0#usize
end d01_arrays
```

## D02_slices — CLEAN

```rust
pub fn first(s: &[u32]) -> u32 { s[0] }
pub fn sub(s: &[u32]) -> &[u32] { &s[1..3] }
pub fn len(s: &[u32]) -> usize { s.len() }
pub fn split(s: &mut [u32]) -> u32 { let (a, b) = s.split_at_mut(1); a[0] + b[0] }
pub fn swap(s: &mut [u32]) { s.swap(0, 1) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace d02_slices
-- [d02_slices::first]:
def first (s : Slice Std.U32) : Result Std.U32 := do
  Slice.index_usize s 0#usize
-- [d02_slices::sub]:
def sub (s : Slice Std.U32) : Result (Slice Std.U32) := do
  core.slice.index.Slice.index (core.slice.index.SliceIndexRangeUsizeSlice
    Std.U32) s { start := 1#usize, «end» := 3#usize }
-- [d02_slices::len]:
def len (s : Slice Std.U32) : Result Std.Usize := do
  ok (Slice.len s)
-- [d02_slices::split]:
def split (s : Slice Std.U32) : Result (Std.U32 × (Slice Std.U32)) := do
  let ((a, b), split_at_mut_back) ← core.slice.Slice.split_at_mut s 1#usize
  let i ← Slice.index_usize a 0#usize
  let i1 ← Slice.index_usize b 0#usize
  let i2 ← i + i1
  let s1 := split_at_mut_back (a, b)
  ok (i2, s1)
-- [d02_slices::swap]:
def swap (s : Slice Std.U32) : Result (Slice Std.U32) := do
  core.slice.Slice.swap s 0#usize 1#usize
end d02_slices
```

## D03_vec — CAVEATS

```rust
pub fn build() -> Vec<u32> { let mut v = Vec::new(); v.push(1); v.push(2); v[0] = 5; v }
pub fn get(v: &Vec<u32>, i: usize) -> u32 { v[i] }
pub fn pop(v: &mut Vec<u32>) -> Option<u32> { v.pop() }
pub fn mac() -> Vec<u32> { vec![1, 2, 3] }
pub fn zeros(n: usize) -> Vec<u8> { vec![0; n] }
pub fn ins(v: &mut Vec<u32>) { v.insert(0, 9); v.remove(1); }
pub fn ext(v: &mut Vec<u32>, w: &[u32]) { v.extend_from_slice(w) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace d03_vec
-- [core::mem::maybe_uninit::MaybeUninit]
@[rust_type "core::mem::maybe_uninit::MaybeUninit"]
axiom core.mem.maybe_uninit.MaybeUninit (T : Type _) : Type
-- [alloc::vec::{alloc::vec::Vec<T>}::remove]:
```

```
@[rust_fun "alloc::vec::{alloc::vec::Vec<@T>}::remove"]
axiom alloc.vec.Vec.remove
  {T : Type _} (A : Type _) :
  alloc.vec.Vec T → Std.Usize → Result (T × (alloc.vec.Vec T))
-- [alloc::vec::{alloc::vec::Vec<T>}::pop]:
@[rust_fun "alloc::vec::{alloc::vec::Vec<@T>}::pop"]
axiom alloc.vec.Vec.pop
  {T : Type _} (A : Type _) :
  alloc.vec.Vec T → Result ((Option T) × (alloc.vec.Vec T))
-- [d03_vec::build]:
def build : Result (alloc.vec.Vec Std.U32) := do
  let v ← alloc.vec.Vec.push (alloc.vec.Vec.new Std.U32) 1#u32
  let v1 ← alloc.vec.Vec.push v 2#u32
  let (_, index_mut_back) ←
    alloc.vec.Vec.index_mut (core.slice.index.SliceIndexUsizeSlice Std.U32) v1
      0#usize
  ok (index_mut_back 5#u32)
-- [d03_vec::get]:
def get (v : alloc.vec.Vec Std.U32) (i : Std.Usize) : Result Std.U32 := do
  alloc.vec.Vec.index (core.slice.index.SliceIndexUsizeSlice Std.U32) v i
-- [d03_vec::pop]:
def pop
  (v : alloc.vec.Vec Std.U32) :
  Result ((Option Std.U32) × (alloc.vec.Vec Std.U32))
  := do
  alloc.vec.Vec.pop alloc.alloc.Global v
-- [d03_vec::mac]:
def mac : Result (alloc.vec.Vec Std.U32) := do
  let y ←
    lift (Std.Array.to_slice (Array.make 3#usize [ 1#u32, 2#u32, 3#u32 ] :
      Array Std.U32 3#usize))
  ok (alloc.slice.Slice.into_vec y)
-- [d03_vec::zeros]:
def zeros (n : Std.Usize) : Result (alloc.vec.Vec Std.U8) := do
  alloc.vec.from_elem core.clone.CloneU8 0#u8 n
-- [d03_vec::ins]:
def ins (v : alloc.vec.Vec Std.U32) : Result (alloc.vec.Vec Std.U32) := do
  let v1 ← alloc.vec.Vec.insert v 0#usize 9#u32
  let (_, v2) ← alloc.vec.Vec.remove alloc.alloc.Global v1 1#usize
  ok v2
-- ... (7 more lines, see experiments/out/)
```

### D04_strings — CAVEATS

```rust
pub fn s_len(s: &str) -> usize { s.len() }
pub fn mk() -> String { let mut s = String::new(); s.push_str("ab"); s }
pub fn eq(a: &str, b: &str) -> bool { a == b }
pub fn lit() -> &'static str { "hello" }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace d04_strings
-- [core::str::{str}::len]:
@[rust_fun "core::str::{str}::len"]
axiom core.str.Str.len : Str → Result Std.Usize
-- [core::str::traits::{impl core::cmp::PartialEq<str> for str}::eq]:
@[rust_fun "core::str::traits::{core::cmp::PartialEq<str, str>}::eq"]
axiom Str.Insts.CoreCmpPartialEqStr.eq : Str → Str → Result Bool
-- Trait implementation: [core::str::traits::{impl core::cmp::PartialEq<str> for str}]
@[reducible, rust_trait_impl "core::cmp::PartialEq<str, str>"]
def Str.Insts.CoreCmpPartialEqStr : core.cmp.PartialEq Str Str := {
  eq := Str.Insts.CoreCmpPartialEqStr.eq
}
-- [alloc::string::{alloc::string::String}::new]:
@[rust_fun "alloc::string::{alloc::string::String}::new"]
axiom alloc.string.String.new : Result String
-- [alloc::string::{alloc::string::String}::push_str]:
@[rust_fun "alloc::string::{alloc::string::String}::push_str"]
axiom alloc.string.String.push_str : String → Str → Result String
```

```
-- [d04_strings::s_len]:
def s_len (s : Str) : Result Std.Usize := do
  core.str.Str.len s
-- [d04_strings::mk]:
def mk : Result String := do
  let s ← alloc.string.String.new
  alloc.string.String.push_str s (toStr "ab")
-- [d04_strings::eq]:
def eq (a : Str) (b : Str) : Result Bool := do
  Str.Insts.CoreCmpPartialEqStr.eq a b
-- [d04_strings::lit]:
def lit : Result Str := do
  ok (toStr "hello")
end d04_strings
```

## D05_option_result — CAVEATS

```
pub fn div(a: u32, b: u32) -> Result<u32, ()> { if b == 0 { Err(()) } else { Ok(a / b) } }
pub fn chain(a: u32, b: u32, c: u32) -> Result<u32, ()> { let x = div(a, b)?; div(x, c) }
pub fn opt(x: Option<u32>) -> u32 { match x { Some(v) => v, None => 0 } }
pub fn unwrap_or(x: Option<u32>) -> u32 { x.unwrap_or(7) }
pub fn map_opt(x: Option<u32>) -> Option<u32> { x.map(|v| v + 1) }
pub fn q(x: Option<u32>) -> Option<u32> { let v = x?; Some(v) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace d05_option_result
-- [core::option::{impl core::ops::try_trait::Try for core::option::Option<T>}::branch]:
@[rust_fun
  "core::option::{core::ops::try_trait::Try<core::option::Option<@T>>}::branch"]
axiom core.option.Option.Insts.CoreOpsTry_traitTry.branch
  {T : Type _} :
  Option T → Result (core.ops.control_flow.ControlFlow (Option Never) T)
-- [core::option::{impl core::ops::try_trait::FromResidual<core::option::Option<!>> for
core::option::Option<T>}::from_residual]:
@[rust_fun
  "core::option::{core::ops::try_trait::FromResidual<core::option::Option<@T>,
core::option::Option<!>>}::from_residual"]
axiom
  core.option.Option.Insts.CoreOpsTry_traitFromResidualOptionNever.from_residual
  (T : Type _) : Option Never → Result (Option T)
-- [d05_option_result::div]:
def div
  (a : Std.U32) (b : Std.U32) : Result (core.result.Result Std.U32 Unit) := do
  if b = 0#u32
  then ok (core.result.Result.Err ())
  else let i ← a / b
       ok (core.result.Result.Ok i)
-- [d05_option_result::chain]:
def chain
  (a : Std.U32) (b : Std.U32) (c : Std.U32) :
  Result (core.result.Result Std.U32 Unit)
  := do
  let r ← div a b
  let cf ← core.result.Result.Insts.CoreOpsTry.branch r
  match cf with
  | core.ops.control_flow.ControlFlow.Continue val => div val c
  | core.ops.control_flow.ControlFlow.Break residual =>
    core.result.Result.Insts.CoreOpsTry_traitFromResidualResult.from_residual
      Std.U32 (core.convert.FromSame Unit) residual
-- [d05_option_result::opt]:
def opt (x : Option Std.U32) : Result Std.U32 := do
  match x with
  | none => ok 0#u32
  | some v => ok v
-- [d05_option_result::unwrap_or]:
def unwrap_or (x : Option Std.U32) : Result Std.U32 := do
  ok (core.option.Option.unwrap_or x 7#u32)
-- [d05_option_result::map_opt::{closure}]
```

```
@[reducible]
def map_opt.closure := Unit
-- [d05_option_result::map_opt::{impl core::ops::function::FnOnce<(u32,), u32> for d05_option_result::map_opt::
{closure}}::call_once]:
def map_opt.closure.Insts.CoreOpsFunctionFnOnceTupleU32U32.call_once
-- ... (21 more lines, see experiments/out/)
```

## D06_box — CLEAN

```
pub fn b() -> u32 { let x = Box::new(5u32); *x + 1 }
pub fn bmut(x: &mut Box<u32>) { **x = 3; }
pub enum List { Cons(u32, Box<List>), Nil }
pub fn head(l: &List) -> u32 { match l { List::Cons(h, _) => *h, List::Nil => 0 } }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace d06_box
-- [d06_box::b]:
def b : Result Std.U32 := do
  5#u32 + 1#u32
-- [d06_box::bmut]:
def bmut (x : Std.U32) : Result Std.U32 := do
  ok 3#u32
-- [d06_box::List]
@[discriminant isize]
inductive List where
| Cons : Std.U32 → List → List
| Nil : List
-- [d06_box::head]:
def head (l : List) : Result Std.U32 := do
  match l with
  | List.Cons h _ => ok h
  | List.Nil => ok 0#u32
end d06_box
```

## D07_struct_enum_match — CLEAN

```
#[derive(Clone, Copy, PartialEq)]
pub enum Color { R, G, B(u8) }
pub struct P { pub x: i32, pub y: i32 }
pub fn f(c: Color) -> u8 { match c { Color::R => 0, Color::G => 1, Color::B(v) if v > 10 => v, Color::B(_) => 2
        } }
pub fn mv(p: &mut P) { p.x += 1; }
pub fn eqc(a: Color, b: Color) -> bool { a == b }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace d07_struct_enum_match
-- [d07_struct_enum_match::Color]
@[discriminant isize]
inductive Color where
| R : Color
| G : Color
| B : Std.U8 → Color
-- [d07_struct_enum_match::{impl core::clone::Clone for d07_struct_enum_match::Color}::clone]:
def Color.Insts.CoreCloneClone.clone (self : Color) : Result Color := do
  ok self
-- Trait implementation: [d07_struct_enum_match::{impl core::clone::Clone for d07_struct_enum_match::Color}]
@[reducible]
def Color.Insts.CoreCloneClone : core.clone.Clone Color := {
  clone := Color.Insts.CoreCloneClone.clone
}
-- Trait implementation: [d07_struct_enum_match::{impl core::marker::Copy for d07_struct_enum_match::Color}]
@[reducible]
def Color.Insts.CoreMarkerCopy : core.marker.Copy Color := {
  cloneInst := Color.Insts.CoreCloneClone
}
-- Trait implementation: [d07_struct_enum_match::{impl core::marker::StructuralPartialEq for
d07_struct_enum_match::Color}]
@[reducible]
```

```
def Color.Insts.CoreMarkerStructuralPartialEq : core.marker.StructuralPartialEq
  Color := {
}
-- [d07_struct_enum_match::{impl core::cmp::PartialEq<d07_struct_enum_match::Color> for
d07_struct_enum_match::Color}::eq]:
def Color.Insts.CoreCmpPartialEqColor.eq
  (self : Color) (other : Color) : Result Bool := do
  let self1 := read_discriminant self
  let other1 := read_discriminant other
  if self1 = other1
  then
    match self with
    | Color.R => ok true
    | Color.G => ok true
    | Color.B __self_0 =>
      match other with
      | Color.R => ok true
      | Color.G => ok true
      | Color.B __arg1_0 =>
        lift (core.cmp.impls.PartialEqU8.eq __self_0 __arg1_0)
  else ok false
-- Trait implementation: [d07_struct_enum_match::{impl core::cmp::PartialEq<d07_struct_enum_match::Color> for
d07_struct_enum_match::Color}]
@[reducible]
def Color.Insts.CoreCmpPartialEqColor : core.cmp.PartialEq Color Color := {
-- ... (22 more lines, see experiments/out/)
```

## D08_tuple_struct_derive — CAVEATS

```rust
#[derive(Clone, Debug, PartialEq, Eq, Default)]
pub struct W(pub u32, pub u64);
pub fn c(w: &W) -> W { w.clone() }
pub fn d() -> W { W::default() }
pub fn e(a: &W, b: &W) -> bool { a == b }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace d08_tuple_struct_derive
-- [core::fmt::{core::fmt::Formatter<'a>}::debug_tuple_field2_finish]:
@[rust_fun "core::fmt::{core::fmt::Formatter<'a>}::debug_tuple_field2_finish"]
axiom core.fmt.Formatter.debug_tuple_field2_finish
  :
  core.fmt.Formatter → Str → Dyn (fun _dyn => core.fmt.Debug _dyn) → Dyn
    (fun _dyn => core.fmt.Debug _dyn) → Result ((core.result.Result Unit
    core.fmt.Error) × core.fmt.Formatter)
-- [d08_tuple_struct_derive::W]
def W := Std.U32 × Std.U64
-- [d08_tuple_struct_derive::{impl core::clone::Clone for d08_tuple_struct_derive::W}::clone]:
def W.Insts.CoreCloneClone.clone (self : W) : Result W := do
  let (i, i1) := self
  let i2 ← lift (core.clone.impls.CloneU32.clone i)
  let i3 ← lift (core.clone.impls.CloneU64.clone i1)
  ok (i2, i3)
-- Trait implementation: [d08_tuple_struct_derive::{impl core::clone::Clone for d08_tuple_struct_derive::W}]
@[reducible]
def W.Insts.CoreCloneClone : core.clone.Clone W := {
  clone := W.Insts.CoreCloneClone.clone
}
-- [d08_tuple_struct_derive::{impl core::fmt::Debug for d08_tuple_struct_derive::W}::fmt]:
def W.Insts.CoreFmtDebug.fmt
  (self : W) (f : core.fmt.Formatter) :
  Result ((core.result.Result Unit core.fmt.Error) × core.fmt.Formatter)
  := do
  let (i, i1) := self
  let dyn := Dyn.mk _ core.fmt.DebugU32 i
  let dyn1 := Dyn.mk _ (core.fmt.DebugShared core.fmt.DebugU64) i1
  core.fmt.Formatter.debug_tuple_field2_finish f (toStr "W") dyn dyn1
-- Trait implementation: [d08_tuple_struct_derive::{impl core::fmt::Debug for d08_tuple_struct_derive::W}]
@[reducible]
```

```
def W.Insts.CoreFmtDebug : core.fmt.Debug W := {
  fmt := W.Insts.CoreFmtDebug.fmt
}
-- Trait implementation: [d08_tuple_struct_derive::{impl core::marker::StructuralPartialEq for
d08_tuple_struct_derive::W}]
@[reducible]
def W.Insts.CoreMarkerStructuralPartialEq : core.marker.StructuralPartialEq W
  := {
}
-- [d08_tuple_struct_derive::{impl core::cmp::PartialEq<d08_tuple_struct_derive::W> for
d08_tuple_struct_derive::W}::eq]:
def W.Insts.CoreCmpPartialEqW.eq (self : W) (other : W) : Result Bool := do
  let (i, i1) := self
  let (i2, i3) := other
  if i = i2
-- ... (34 more lines, see experiments/out/)
```

### F01_extern_c — CAVEATS

```rust
extern "C" { fn ext_add(a: u32, b: u32) -> u32; }
pub fn call(a: u32) -> u32 { unsafe { ext_add(a, 1) } }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace f01_extern_c
-- [f01_extern_c::ext_add]:
axiom ext_add : Std.U32 → Std.U32 → Result Std.U32
-- [f01_extern_c::call]:
def call (a : Std.U32) : Result Std.U32 := do
  ext_add a 1#u32
end f01_extern_c
```

### F02_opaque_module — CAVEATS

```rust
// This module is marked opaque via `--opaque crate::hidden` on the Charon command line.
pub mod hidden {
    pub fn secret(x: u32) -> u32 { x.wrapping_mul(31) }
}
pub fn use_hidden(x: u32) -> u32 { hidden::secret(x) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace f02_opaque_module
-- [f02_opaque_module::hidden::secret]:
axiom hidden.secret : Std.U32 → Result Std.U32
-- [f02_opaque_module::use_hidden]:
def use_hidden (x : Std.U32) : Result Std.U32 := do
  hidden.secret x
end f02_opaque_module
```

### F03_no_mangle_export — CLEAN

```rust
#[no_mangle]
pub extern "C" fn exported(x: u32) -> u32 { x.wrapping_add(1) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace f03_no_mangle_export
-- [f03_no_mangle_export::exported]:
def exported (x : Std.U32) : Result Std.U32 := do
  ok (core.num.U32.wrapping_add x 1#u32)
end f03_no_mangle_export
```

## I01_cell — CAVEATS

```
use std::cell::Cell;
pub fn c(x: &Cell<u32>) { x.set(x.get() + 1) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace i01_cell
-- [core::cell::Cell]
@[rust_type "core::cell::Cell"]
axiom core.cell.Cell (T : Type _) : Type
-- [core::cell::{core::cell::Cell<T>}::set]:
@[rust_fun "core::cell::{core::cell::Cell<@T>}::set"]
axiom core.cell.Cell.set {T : Type _} : core.cell.Cell T → T → Result Unit
-- [core::cell::{core::cell::Cell<T>}::get]:
@[rust_fun "core::cell::{core::cell::Cell<@T>}::get"]
axiom core.cell.Cell.get
  {T : Type _} (markerCopyInst : core.marker.Copy T) :
  core.cell.Cell T → Result T
-- [i01_cell::c]:
def c (x : core.cell.Cell Std.U32) : Result Unit := do
  let i ← core.cell.Cell.get core.marker.CopyU32 x
  let i1 ← i + 1#u32
  core.cell.Cell.set x i1
end i01_cell
```

## I02_refcell — CAVEATS

```
use std::cell::RefCell;
pub fn r(x: &RefCell<u32>) { *x.borrow_mut() += 1 }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace i02_refcell
-- [core::cell::RefCell]
@[rust_type "core::cell::RefCell"]
axiom core.cell.RefCell (T : Type _) : Type
-- [core::cell::RefMut]
@[rust_type "core::cell::RefMut" (mutRegions := #[0])]
axiom core.cell.RefMut (T : Type _) : Type
-- [core::cell::{core::cell::RefCell<T>}::borrow_mut]:
@[rust_fun "core::cell::{core::cell::RefCell<@T>}::borrow_mut"]
axiom core.cell.RefCell.borrow_mut
  {T : Type _} :
  core.cell.RefCell T → Result ((core.cell.RefMut T) × (core.cell.RefMut T
    → Unit))
```

```
-- [core::cell::{impl core::ops::deref::DerefMut<T> for core::cell::RefMut<'_0, T>}::deref_mut]:
@[rust_fun
  "core::cell::{core::ops::deref::DerefMut<core::cell::RefMut<'0, @T>, @T>}::deref_mut"]
axiom core.cell.RefMut.Insts.CoreOpsDerefDerefMut.deref_mut
  {T : Type _} :
  core.cell.RefMut T → Result (T × (T → core.cell.RefMut T) ×
    (core.cell.RefMut T → core.cell.RefMut T))
-- [i02_refcell::r]:
def r (x : core.cell.RefCell Std.U32) : Result Unit := do
  let (rm, _) ← core.cell.RefCell.borrow_mut x
  let (i, _, _) ← core.cell.RefMut.Insts.CoreOpsDerefDerefMut.deref_mut rm
  let _ ← i + 1#u32
  ok ()
end i02_refcell
```

## I03_rc — CAVEATS

```rust
use std::rc::Rc;
pub fn rc(x: u32) -> u32 { let a = Rc::new(x); let b = Rc::clone(&a); *a + *b }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace i03_rc
-- Trait declaration: [core::alloc::AllocatorClone]
@[rust_trait "core::alloc::AllocatorClone"
  (parentClauses := ["cloneCloneInst"])]
structure core.alloc.AllocatorClone (Self : Type _) where
  cloneCloneInst : core.clone.Clone Self
-- Trait implementation: [alloc::alloc::{impl core::alloc::AllocatorClone for alloc::alloc::Global}]
@[reducible, rust_trait_impl
  "core::alloc::AllocatorClone<alloc::alloc::Global>"]
def alloc.alloc.Global.Insts.CoreAllocAllocatorClone :
  core.alloc.AllocatorClone alloc.alloc.Global := {
  cloneCloneInst := core.clone.CloneGlobal
}
-- [alloc::rc::Rc]
@[rust_type "alloc::rc::Rc"]
axiom alloc.rc.Rc (T : Type _) : Type
-- [alloc::rc::{alloc::rc::Rc<T>}::new]:
@[rust_fun "alloc::rc::{alloc::rc::Rc<@T>}::new"]
axiom alloc.rc.Rc.new {T : Type _} : T → Result (alloc.rc.Rc T)
-- [alloc::rc::{impl core::ops::deref::Deref<T> for alloc::rc::Rc<T>}::deref]:
@[rust_fun
  "alloc::rc::{core::ops::deref::Deref<alloc::rc::Rc<@T>, @T>}::deref"]
axiom alloc.rc.Rc.Insts.CoreOpsDerefDeref.deref
  {T : Type _} (A : Type _) : alloc.rc.Rc T → Result T
-- [alloc::rc::{impl core::clone::Clone for alloc::rc::Rc<T>}::clone]:
@[rust_fun "alloc::rc::{core::clone::Clone<alloc::rc::Rc<@T>>}::clone"]
axiom alloc.rc.Rc.Insts.CoreCloneClone.clone
  {T : Type _} {A : Type _} (coreallocAllocatorCloneInst :
  core.alloc.AllocatorClone A) :
  alloc.rc.Rc T → Result (alloc.rc.Rc T)
-- [i03_rc::rc]:
def rc (x : Std.U32) : Result Std.U32 := do
  let a ← alloc.rc.Rc.new x
  let b ←
    alloc.rc.Rc.Insts.CoreCloneClone.clone
      alloc.alloc.Global.Insts.CoreAllocAllocatorClone a
  let i ← alloc.rc.Rc.Insts.CoreOpsDerefDeref.deref alloc.alloc.Global a
  let i1 ← alloc.rc.Rc.Insts.CoreOpsDerefDeref.deref alloc.alloc.Global b
  i + i1
end i03_rc
```

## I04_arc — CAVEATS

```rust
use std::sync::Arc;
pub fn arc(x: u32) -> u32 { let a = Arc::new(x); let b = a.clone(); *b }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace i04_arc
-- Trait declaration: [core::alloc::AllocatorClone]
@[rust_trait "core::alloc::AllocatorClone"
  (parentClauses := ["cloneCloneInst"])]
structure core.alloc.AllocatorClone (Self : Type _) where
  cloneCloneInst : core.clone.Clone Self
-- Trait implementation: [alloc::alloc::{impl core::alloc::AllocatorClone for alloc::alloc::Global}]
@[reducible, rust_trait_impl
  "core::alloc::AllocatorClone<alloc::alloc::Global>"]
def alloc.alloc.Global.Insts.CoreAllocAllocatorClone :
  core.alloc.AllocatorClone alloc.alloc.Global := {
  cloneCloneInst := core.clone.CloneGlobal
}
-- [alloc::sync::Arc]
@[rust_type "alloc::sync::Arc"]
axiom alloc.sync.Arc (T : Type _) : Type
-- [alloc::sync::{alloc::sync::Arc<T>}::new]:
@[rust_fun "alloc::sync::{alloc::sync::Arc<@T>}::new"]
axiom alloc.sync.Arc.new {T : Type _} : T → Result (alloc.sync.Arc T)
-- [alloc::sync::{impl core::clone::Clone for alloc::sync::Arc<T>}::clone]:
@[rust_fun "alloc::sync::{core::clone::Clone<alloc::sync::Arc<@T>>}::clone"]
axiom alloc.sync.Arc.Insts.CoreCloneClone.clone
  {T : Type _} {A : Type _} (coreallocAllocatorCloneInst :
  core.alloc.AllocatorClone A) :
  alloc.sync.Arc T → Result (alloc.sync.Arc T)
-- [alloc::sync::{impl core::ops::deref::Deref<T> for alloc::sync::Arc<T>}::deref]:
@[rust_fun
  "alloc::sync::{core::ops::deref::Deref<alloc::sync::Arc<@T>, @T>}::deref"]
axiom alloc.sync.Arc.Insts.CoreOpsDerefDeref.deref
  {T : Type _} (A : Type _) : alloc.sync.Arc T → Result T
-- [i04_arc::arc]:
def arc (x : Std.U32) : Result Std.U32 := do
  let a ← alloc.sync.Arc.new x
  let b ←
    alloc.sync.Arc.Insts.CoreCloneClone.clone
      alloc.alloc.Global.Insts.CoreAllocAllocatorClone a
  alloc.sync.Arc.Insts.CoreOpsDerefDeref.deref alloc.alloc.Global b
end i04_arc
```

## R01_recursion — CLEAN

```
pub fn fact(n: u64) -> u64 { if n == 0 { 1 } else { n * fact(n - 1) } }
pub enum L { Cons(u32, Box<L>), Nil }
pub fn len(l: &L) -> u32 { match l { L::Cons(_, t) => 1 + len(t), L::Nil => 0 } }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace r01_recursion
-- [r01_recursion::fact]:
def fact (n : Std.U64) : Result Std.U64 := do
  if n = 0#u64
  then ok 1#u64
  else let i ← n - 1#u64
       let i1 ← fact i
       n * i1
partial_fixpoint
-- [r01_recursion::L]
@[discriminant isize]
inductive L where
| Cons : Std.U32 → L → L
| Nil : L
-- [r01_recursion::len]:
def len (l : L) : Result Std.U32 := do
  match l with
```

```
  | L.Cons _ t => let i ← len t
                  1#u32 + i
  | L.Nil => ok 0#u32
partial_fixpoint
end r01_recursion
```

### R02_mutual — CLEAN

```
pub fn even(n: u32) -> bool { if n == 0 { true } else { odd(n - 1) } }
pub fn odd(n: u32) -> bool { if n == 0 { false } else { even(n - 1) } }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace r02_mutual
mutual
-- [r02_mutual::even]:
def even (n : Std.U32) : Result Bool := do
  if n = 0#u32
  then ok true
  else let i ← n - 1#u32
       odd i
partial_fixpoint
-- [r02_mutual::odd]:
def odd (n : Std.U32) : Result Bool := do
  if n = 0#u32
  then ok false
  else let i ← n - 1#u32
       even i
partial_fixpoint
end
end r02_mutual
```

### R03_mutual_inductive — FAIL

```
pub enum Tree { Node(u32, Forest) }
pub enum Forest { Nil, Cons(Box<Tree>, Box<Forest>) }
pub fn tsum(t: &Tree) -> u32 { match t { Tree::Node(v, f) => v + fsum(f) } }
pub fn fsum(f: &Forest) -> u32 { match f { Forest::Nil => 0, Forest::Cons(t, r) => tsum(t) + fsum(r) } }
```

Exit codes: charon=0, aeneas=0, lean=1.

Lean errors:

```
../out/R03_mutual_inductive/lean/R03MutualInductive.lean:49:10: error: Could not prove
'r03_mutual_inductive.tsum' to be monotone in its recursive calls:
```

Generated Lean:

```
namespace r03_mutual_inductive
mutual
-- [r03_mutual_inductive::Tree]
@[discriminant isize]
inductive Tree where
| Node : Std.U32 → Forest → Tree
-- [r03_mutual_inductive::Forest]
@[discriminant isize]
inductive Forest where
| Nil : Forest
| Cons : Tree → Forest → Forest
end
mutual
-- [r03_mutual_inductive::tsum]:
def tsum (t : Tree) : Result Std.U32 := do
  let ⟨ v, f ⟩ := t
  let i ← fsum f
  SharedU32.Insts.CoreOpsArithAddU32U32.add v i
partial_fixpoint
-- [r03_mutual_inductive::fsum]:
def fsum (f : Forest) : Result Std.U32 := do
  match f with
  | Forest.Nil => ok 0#u32
```

```
  | Forest.Cons t r => let i ← tsum t
                       let i1 ← fsum r
                       i + i1
partial_fixpoint
end
end r03_mutual_inductive
```

## S01_hashmap — CAVEATS

```rust
use std::collections::HashMap;
pub fn h() -> Option<u32> { let mut m = HashMap::new(); m.insert(1u32, 2u32); m.get(&1).copied() }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace s01_hashmap
-- Trait declaration: [core::hash::BuildHasher]
@[rust_trait "core::hash::BuildHasher" (parentClauses := ["HasherInst"])]
structure core.hash.BuildHasher (Self : Type _) (Self_Hasher : Type _) where
  HasherInst : core.hash.Hasher Self_Hasher
  build_hasher : Self → Result Self_Hasher
-- [core::hash::impls::{impl core::hash::Hash for u32}::hash]:
@[rust_fun "core::hash::impls::{core::hash::Hash<u32>}::hash"]
axiom U32.Insts.CoreHashHash.hash
  {H : Type _} (HasherInst : core.hash.Hasher H) : Std.U32 → H → Result H
-- Trait implementation: [core::hash::impls::{impl core::hash::Hash for u32}]
@[reducible, rust_trait_impl "core::hash::Hash<u32>"]
def U32.Insts.CoreHashHash : core.hash.Hash Std.U32 := {
  hash := fun {H : Type} (HasherInst : core.hash.Hasher H) =>
    U32.Insts.CoreHashHash.hash HasherInst
}
-- [core::option::{core::option::Option<&'_0 T>}::copied]:
@[rust_fun "core::option::{core::option::Option<&'0 @T>}::copied"]
axiom core.option.OptionShared0T.copied
  {T : Type _} (markerCopyInst : core.marker.Copy T) :
  Option T → Result (Option T)
-- [std::collections::hash::map::HashMap]
@[rust_type "std::collections::hash::map::HashMap"]
axiom std.collections.hash.map.HashMap (K : Type _) (V : Type _) (S : Type _)
  (A : Type _) : Type
-- [std::hash::random::RandomState]
@[rust_type "std::hash::random::RandomState"]
axiom std.hash.random.RandomState : Type
-- [std::collections::hash::map::{std::collections::hash::map::HashMap<K, V, std::hash::random::RandomState,
alloc::alloc::Global>}::new]:
@[rust_fun
  "std::collections::hash::map::{std::collections::hash::map::HashMap<@K, @V, std::hash::random::RandomState,
alloc::alloc::Global>}::new"]
axiom std.collections.hash.map.HashMapKVRandomStateGlobal.new
  (K : Type _) (V : Type _) :
  Result (std.collections.hash.map.HashMap K V std.hash.random.RandomState
    alloc.alloc.Global)
-- [std::collections::hash::map::{std::collections::hash::map::HashMap<K, V, S, A>}::get]:
@[rust_fun
  "std::collections::hash::map::{std::collections::hash::map::HashMap<@K, @V, @S, @A>}::get"]
axiom std.collections.hash.map.HashMap.get
  {K : Type _} {V : Type _} {S : Type _} {A : Type _} {Q : Type _}
  {Clause2_Hasher : Type _} (corecmpEqInst : core.cmp.Eq K) (corehashHashInst :
  core.hash.Hash K) (corehashBuildHasherInst : core.hash.BuildHasher S
  Clause2_Hasher) (coreborrowBorrowInst : core.borrow.Borrow K Q)
  (corehashHashInst1 : core.hash.Hash Q) (corecmpEqInst1 : core.cmp.Eq Q) :
  std.collections.hash.map.HashMap K V S A → Q → Result (Option V)
-- ... (64 more lines, see experiments/out/)
```

## S02_cmp_minmax — CLEAN

```rust
use std::cmp::Ordering;
pub fn mx(a: u32, b: u32) -> u32 { std::cmp::max(a, b) }
```

```
pub fn mn(a: u32, b: u32) -> u32 { a.min(b) }
pub fn c(a: u32, b: u32) -> Ordering { a.cmp(&b) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace s02_cmp_minmax
-- [s02_cmp_minmax::mx]:
def mx (a : Std.U32) (b : Std.U32) : Result Std.U32 := do
  core.cmp.max core.cmp.OrdU32 a b
-- [s02_cmp_minmax::mn]:
def mn (a : Std.U32) (b : Std.U32) : Result Std.U32 := do
  ok (core.cmp.impls.OrdU32.min a b)
-- [s02_cmp_minmax::c]:
def c (a : Std.U32) (b : Std.U32) : Result Ordering := do
  ok (core.cmp.impls.OrdU32.cmp a b)
end s02_cmp_minmax
```

## S03_mem_swap_replace — CAVEATS

```
pub fn s(a: &mut u32, b: &mut u32) { core::mem::swap(a, b) }
pub fn r(a: &mut u32) -> u32 { core::mem::replace(a, 0) }
pub fn t(v: &mut Vec<u32>) -> Vec<u32> { core::mem::take(v) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace s03_mem_swap_replace
-- [core::mem::take]:
@[rust_fun "core::mem::take"]
axiom core.mem.take
  {T : Type _} (defaultDefaultInst : core.default.Default T) :
  T → Result (T × T)
-- [alloc::vec::{impl core::default::Default for alloc::vec::Vec<T>}::default]:
@[rust_fun
  "alloc::vec::{core::default::Default<alloc::vec::Vec<@T>>}::default"]
axiom alloc.vec.Vec.Insts.CoreDefaultDefault.default
  (T : Type _) : Result (alloc.vec.Vec T)
-- Trait implementation: [alloc::vec::{impl core::default::Default for alloc::vec::Vec<T>}]
@[reducible, rust_trait_impl "core::default::Default<alloc::vec::Vec<@T>>"]
def alloc.vec.Vec.Insts.CoreDefaultDefault (T : Type _) : core.default.Default
  (alloc.vec.Vec T) := {
  default := alloc.vec.Vec.Insts.CoreDefaultDefault.default T
}
-- [s03_mem_swap_replace::s]:
def s (a : Std.U32) (b : Std.U32) : Result (Std.U32 × Std.U32) := do
  ok (core.mem.swap a b)
-- [s03_mem_swap_replace::r]:
def r (a : Std.U32) : Result (Std.U32 × Std.U32) := do
  ok (core.mem.replace a 0#u32)
-- [s03_mem_swap_replace::t]:
def t
  (v : alloc.vec.Vec Std.U32) :
  Result ((alloc.vec.Vec Std.U32) × (alloc.vec.Vec Std.U32))
  := do
  core.mem.take (alloc.vec.Vec.Insts.CoreDefaultDefault Std.U32) v
end s03_mem_swap_replace
```

## S04_slice_methods — CAVEATS

```
pub fn c(s: &[u32]) -> bool { s.contains(&3) }
pub fn srt(s: &mut [u32]) { s.sort() }
pub fn cp(d: &mut [u32], s: &[u32]) { d.copy_from_slice(s) }
pub fn fl(d: &mut [u8]) { d.fill(0) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace s04_slice_methods
-- [alloc::slice::{[T]}::sort]:
@[rust_fun "alloc::slice::{[@T]}::sort"]
axiom alloc.slice.Slice.sort
  {T : Type _} (corecmpOrdInst : core.cmp.Ord T) : Slice T → Result (Slice T)
-- [s04_slice_methods::c]:
def c (s : Slice Std.U32) : Result Bool := do
  core.slice.Slice.contains core.cmp.PartialEqU32 s 3#u32
-- [s04_slice_methods::srt]:
def srt (s : Slice Std.U32) : Result (Slice Std.U32) := do
  alloc.slice.Slice.sort core.cmp.OrdU32 s
-- [s04_slice_methods::cp]:
def cp (d : Slice Std.U32) (s : Slice Std.U32) : Result (Slice Std.U32) := do
  core.slice.Slice.copy_from_slice core.marker.CopyU32 d s
-- [s04_slice_methods::fl]:
def fl (d : Slice Std.U8) : Result (Slice Std.U8) := do
  core.slice.Slice.fill core.clone.CloneU8 d 0#u8
end s04_slice_methods
```

## S05_fmt_print — CAVEATS

```rust
pub fn p(x: u32) { println!("{}", x) }
pub fn f(x: u32) -> String { format!("{}", x) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace s05_fmt_print
-- Trait implementation: [core::fmt::num::imp::{impl core::fmt::Display for u32}]
@[reducible, rust_trait_impl "core::fmt::Display<u32>"]
def U32.Insts.CoreFmtDisplay : core.fmt.Display Std.U32 := {
  fmt := core.fmt.num.imp.DisplayU32.fmt
}
-- [core::hint::must_use]:
@[rust_fun "core::hint::must_use"]
axiom core.hint.must_use {T : Type _} : T → Result T
-- [alloc::fmt::format]:
@[rust_fun "alloc::fmt::format"]
axiom alloc.fmt.format : core.fmt.Arguments → Result String
-- [s05_fmt_print::p]:
def p (x : Std.U32) : Result Unit := do
  let a ← core.fmt.rt.Argument.new_display U32.Insts.CoreFmtDisplay x
  let a1 ←
    core.fmt.Arguments.new (Array.make 4#usize [ 192#u8, 1#u8, 10#u8, 0#u8 ])
      (Array.make 1#usize [ a ])
  std.io.stdio._print a1
-- [s05_fmt_print::f]:
def f (x : Std.U32) : Result String := do
  let a ← core.fmt.rt.Argument.new_display U32.Insts.CoreFmtDisplay x
  let a1 ←
    core.fmt.Arguments.new (Array.make 2#usize [ 192#u8, 0#u8 ])
      (Array.make 1#usize [ a ])
  let s ← alloc.fmt.format a1
  core.hint.must_use s
end s05_fmt_print
```

## S06_vecdeque_btree — CAVEATS

```rust
use std::collections::{BTreeMap, VecDeque};
pub fn q() -> Option<u32> { let mut d = VecDeque::new(); d.push_back(1u32); d.pop_front() }
pub fn b() -> usize { let mut m = BTreeMap::new(); m.insert(1u32, 1u32); m.len() }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace s06_vecdeque_btree
-- Trait declaration: [core::alloc::AllocatorClone]
@[rust_trait "core::alloc::AllocatorClone"
  (parentClauses := ["cloneCloneInst"])]
structure core.alloc.AllocatorClone (Self : Type _) where
  cloneCloneInst : core.clone.Clone Self
-- Trait implementation: [alloc::alloc::{impl core::alloc::AllocatorClone for alloc::alloc::Global}]
@[reducible, rust_trait_impl
  "core::alloc::AllocatorClone<alloc::alloc::Global>"]
def alloc.alloc.Global.Insts.CoreAllocAllocatorClone :
  core.alloc.AllocatorClone alloc.alloc.Global := {
  cloneCloneInst := core.clone.CloneGlobal
}
-- [alloc::collections::btree::map::BTreeMap]
@[rust_type "alloc::collections::btree::map::BTreeMap"]
axiom alloc.collections.btree.map.BTreeMap (K : Type _) (V : Type _) (A :
  Type _) : Type
-- [alloc::collections::btree::map::{alloc::collections::btree::map::BTreeMap<K, V,
alloc::alloc::Global>}::new]:
@[rust_fun
  "alloc::collections::btree::map::{alloc::collections::btree::map::BTreeMap<@K, @V,
alloc::alloc::Global>}::new"]
axiom alloc.collections.btree.map.BTreeMapKVGlobal.new
  (K : Type _) (V : Type _) :
  Result (alloc.collections.btree.map.BTreeMap K V alloc.alloc.Global)
-- [alloc::collections::btree::map::{alloc::collections::btree::map::BTreeMap<K, V, A>}::insert]:
@[rust_fun
  "alloc::collections::btree::map::{alloc::collections::btree::map::BTreeMap<@K, @V, @A>}::insert"]
axiom alloc.collections.btree.map.BTreeMap.insert
  {K : Type _} {V : Type _} {A : Type _} (coreallocAllocatorCloneInst :
  core.alloc.AllocatorClone A) (corecmpOrdInst : core.cmp.Ord K) :
  alloc.collections.btree.map.BTreeMap K V A → K → V → Result ((Option V)
    × (alloc.collections.btree.map.BTreeMap K V A))
-- [alloc::collections::btree::map::{alloc::collections::btree::map::BTreeMap<K, V, A>}::len]:
@[rust_fun
  "alloc::collections::btree::map::{alloc::collections::btree::map::BTreeMap<@K, @V, @A>}::len"]
axiom alloc.collections.btree.map.BTreeMap.len
  {K : Type _} {V : Type _} {A : Type _} (coreallocAllocatorCloneInst :
  core.alloc.AllocatorClone A) :
  alloc.collections.btree.map.BTreeMap K V A → Result Std.Usize
-- [alloc::collections::vec_deque::VecDeque]
@[rust_type "alloc::collections::vec_deque::VecDeque"]
axiom alloc.collections.vec_deque.VecDeque (T : Type _) (A : Type _) : Type
-- [alloc::collections::vec_deque::{alloc::collections::vec_deque::VecDeque<T, alloc::alloc::Global>}::new]:
@[rust_fun
  "alloc::collections::vec_deque::{alloc::collections::vec_deque::VecDeque<@T, alloc::alloc::Global>}::new"]
axiom alloc.collections.vec_deque.VecDequeTGlobal.new
-- ... (32 more lines, see experiments/out/)
```

## S07_from_into — CLEAN

```rust
pub struct M(pub u32);
impl From<u32> for M { fn from(x: u32) -> M { M(x) } }
pub fn conv(x: u32) -> M { x.into() }
pub fn b(x: u32) -> [u8; 4] { x.to_le_bytes() }
pub fn fb(b: [u8; 4]) -> u32 { u32::from_le_bytes(b) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace s07_from_into
-- [s07_from_into::M]
@[reducible]
def M := Std.U32
-- [s07_from_into::{impl core::convert::From<u32> for s07_from_into::M}::from]:
def M.Insts.CoreConvertFromU32.from (x : Std.U32) : Result M := do
  ok x
-- Trait implementation: [s07_from_into::{impl core::convert::From<u32> for s07_from_into::M}]
@[reducible]
def M.Insts.CoreConvertFromU32 : core.convert.From M Std.U32 := {
  «from» := M.Insts.CoreConvertFromU32.from
}
-- [s07_from_into::conv]:
def conv (x : Std.U32) : Result M := do
  core.convert.IntoFrom.into M.Insts.CoreConvertFromU32 x
-- [s07_from_into::b]:
def b (x : Std.U32) : Result (Array Std.U8 4#usize) := do
  ok (core.num.U32.to_le_bytes x)
-- [s07_from_into::fb]:
def fb (b1 : Array Std.U8 4#usize) : Result Std.U32 := do
  ok (core.num.U32.from_le_bytes b1)
end s07_from_into
```

### S08_drop_impl — CLEAN

```
pub struct G { pub x: u32 }
impl Drop for G { fn drop(&mut self) {} }
pub fn d() -> u32 { let g = G { x: 1 }; g.x }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace s08_drop_impl
-- [s08_drop_impl::G]
structure G where
  x : Std.U32
-- [s08_drop_impl::{impl core::ops::drop::Drop for s08_drop_impl::G}::drop]:
def G.Insts.CoreOpsDropDrop.drop (self : G) : Result G := do
  ok self
-- Trait implementation: [s08_drop_impl::{impl core::ops::drop::Drop for s08_drop_impl::G}]
@[reducible]
def G.Insts.CoreOpsDropDrop : core.ops.drop.Drop G := {
  drop := G.Insts.CoreOpsDropDrop.drop
}
-- [s08_drop_impl::d]:
def d : Result Std.U32 := do
  ok 1#u32
end s08_drop_impl
```

### T01_trait_generic — CLEAN

```
pub trait Shape { fn area(&self) -> u32; }
pub struct Sq { pub s: u32 }
impl Shape for Sq { fn area(&self) -> u32 { self.s * self.s } }
pub fn total_area<T: Shape>(a: &T, b: &T) -> u32 { a.area() + b.area() }
pub fn use_it() -> u32 { total_area(&Sq { s: 2 }, &Sq { s: 3 }) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t01_trait_generic
-- Trait declaration: [t01_trait_generic::Shape]
structure Shape (Self : Type _) where
  area : Self → Result Std.U32
-- [t01_trait_generic::Sq]
structure Sq where
  s : Std.U32
-- [t01_trait_generic::{impl t01_trait_generic::Shape for t01_trait_generic::Sq}::area]:
def Sq.Insts.T01_trait_genericShape.area (self : Sq) : Result Std.U32 := do
  self.s * self.s
-- Trait implementation: [t01_trait_generic::{impl t01_trait_generic::Shape for t01_trait_generic::Sq}]
```

```
@[reducible]
def Sq.Insts.T01_trait_genericShape : Shape Sq := {
  area := Sq.Insts.T01_trait_genericShape.area
}
-- [t01_trait_generic::total_area]:
def total_area
  {T : Type _} (ShapeInst : Shape T) (a : T) (b : T) : Result Std.U32 := do
  let i ← ShapeInst.area a
  let i1 ← ShapeInst.area b
  i + i1
-- [t01_trait_generic::use_it]:
def use_it : Result Std.U32 := do
  total_area Sq.Insts.T01_trait_genericShape { s := 2#u32 } { s := 3#u32 }
end t01_trait_generic
```

### T02_assoc_types — CLEAN

```
pub trait Container { type Item; fn get(&self, i: usize) -> Option<&Self::Item>; }
pub struct Wrap { pub v: Vec<u32> }
impl Container for Wrap {
    type Item = u32;
    fn get(&self, i: usize) -> Option<&u32> { if i < self.v.len() { Some(&self.v[i]) } else { None } }
}
pub fn first<C: Container>(c: &C) -> Option<&C::Item> { c.get(0) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t02_assoc_types
-- Trait declaration: [t02_assoc_types::Container]
structure Container (Self : Type _) (Self_Item : Type _) where
  get : Self → Std.Usize → Result (Option Self_Item)
-- [t02_assoc_types::Wrap]
structure Wrap where
  v : alloc.vec.Vec Std.U32
-- [t02_assoc_types::{impl t02_assoc_types::Container<u32> for t02_assoc_types::Wrap}::get]:
def Wrap.Insts.T02_assoc_typesContainerU32.get
  (self : Wrap) (i : Std.Usize) : Result (Option Std.U32) := do
  let i1 := alloc.vec.Vec.len self.v
  if i < i1
  then
    let i2 ←
      alloc.vec.Vec.index (core.slice.index.SliceIndexUsizeSlice Std.U32)
        self.v i
    ok (some i2)
  else ok none
-- Trait implementation: [t02_assoc_types::{impl t02_assoc_types::Container<u32> for t02_assoc_types::Wrap}]
@[reducible]
def Wrap.Insts.T02_assoc_typesContainerU32 : Container Wrap Std.U32 := {
  get := Wrap.Insts.T02_assoc_typesContainerU32.get
}
-- [t02_assoc_types::first]:
def first
  {C : Type _} {Clause0_Item : Type _} (ContainerInst : Container C
  Clause0_Item) (c : C) :
  Result (Option Clause0_Item)
  := do
  ContainerInst.get c 0#usize
end t02_assoc_types
```

### T03_const_generics — CLEAN

```
pub struct Buf<const N: usize> { pub data: [u8; N] }
pub fn len_of<const N: usize>(b: &Buf<N>) -> usize { N }
pub fn get0<const N: usize>(b: &Buf<N>) -> u8 { b.data[0] }
pub fn mk() -> Buf<4> { Buf { data: [0; 4] } }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t03_const_generics
-- [t03_const_generics::Buf]
```

```
structure Buf (N : Std.Usize) where
  data : Array Std.U8 N
-- [t03_const_generics::len_of]:
def len_of {N : Std.Usize} (b : Buf N) : Result Std.Usize := do
  ok N
-- [t03_const_generics::get0]:
def get0 {N : Std.Usize} (b : Buf N) : Result Std.U8 := do
  Array.index_usize b.data 0#usize
-- [t03_const_generics::mk]:
def mk : Result (Buf 4#usize) := do
  let a := Array.repeat 4#usize 0#u8
  ok { data := a }
end t03_const_generics
```

## T04_dyn_trait — FAIL

```
pub trait Shape { fn area(&self) -> u32; }
pub struct Sq { pub s: u32 }
impl Shape for Sq { fn area(&self) -> u32 { self.s * self.s } }
pub fn area_dyn(x: &dyn Shape) -> u32 { x.area() }
pub fn call() -> u32 { let s = Sq { s: 3 }; area_dyn(&s) }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Function pointers are not supported yet
[Warn] Could not translate the body of function 't04_dyn_trait::area_dyn because of previous error
```

Generated Lean:

```
namespace t04_dyn_trait
-- Trait declaration: [t04_dyn_trait::Shape]
structure Shape (Self : Type _) where
  area : Self → Result Std.U32
-- [t04_dyn_trait::Sq]
structure Sq where
  s : Std.U32
-- [t04_dyn_trait::{impl t04_dyn_trait::Shape for t04_dyn_trait::Sq}::area]:
def Sq.Insts.T04_dyn_traitShape.area (self : Sq) : Result Std.U32 := do
  self.s * self.s
-- Trait implementation: [t04_dyn_trait::{impl t04_dyn_trait::Shape for t04_dyn_trait::Sq}]
@[reducible]
def Sq.Insts.T04_dyn_traitShape : Shape Sq := {
  area := Sq.Insts.T04_dyn_traitShape.area
}
-- [t04_dyn_trait::area_dyn]:
def area_dyn (x : Dyn (fun _dyn => Shape _dyn)) : Result Std.U32 := do
  sorry
-- [t04_dyn_trait::call]:
def call : Result Std.U32 := do
  let dyn := Dyn.mk _ Sq.Insts.T04_dyn_traitShape ({ s := 3#u32 } : Sq)
  area_dyn dyn
end t04_dyn_trait
```

## T04b_box_dyn — FAIL

```
pub trait Shape { fn area(&self) -> u32; }
pub struct Sq { pub s: u32 }
impl Shape for Sq { fn area(&self) -> u32 { self.s * self.s } }
pub fn mk() -> Box<dyn Shape> { Box::new(Sq { s: 3 }) }
pub fn use_box(b: Box<dyn Shape>) -> u32 { b.area() }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Function pointers are not supported yet
[Warn] Could not translate the body of function 't04b_box_dyn::use_box because of previous error
```

Generated Lean:

```
namespace t04b_box_dyn
-- Trait declaration: [t04b_box_dyn::Shape]
structure Shape (Self : Type _) where
  area : Self → Result Std.U32
-- [t04b_box_dyn::Sq]
structure Sq where
  s : Std.U32
-- [t04b_box_dyn::{impl t04b_box_dyn::Shape for t04b_box_dyn::Sq}::area]:
def Sq.Insts.T04b_box_dynShape.area (self : Sq) : Result Std.U32 := do
  self.s * self.s
-- Trait implementation: [t04b_box_dyn::{impl t04b_box_dyn::Shape for t04b_box_dyn::Sq}]
@[reducible]
def Sq.Insts.T04b_box_dynShape : Shape Sq := {
  area := Sq.Insts.T04b_box_dynShape.area
}
-- [t04b_box_dyn::mk]:
def mk : Result (Dyn (fun _dyn => Shape _dyn)) := do
  ok (Dyn.mk _ Sq.Insts.T04b_box_dynShape ({ s := 3#u32 } : Sq))
-- [t04b_box_dyn::use_box]:
def use_box (b : Dyn (fun _dyn => Shape _dyn)) : Result Std.U32 := do
  sorry
end t04b_box_dyn
```

## T05_default_supertrait — CLEAN

```
pub trait Named { fn id(&self) -> u32; }
pub trait Greet: Named {
    fn greet(&self) -> u32 { self.id() + 1 }
}
pub struct P;
impl Named for P { fn id(&self) -> u32 { 41 } }
impl Greet for P {}
pub fn g<T: Greet>(x: &T) -> u32 { x.greet() }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t05_default_supertrait
-- Trait declaration: [t05_default_supertrait::Named]
structure Named (Self : Type _) where
  id : Self → Result Std.U32
-- Trait declaration: [t05_default_supertrait::Greet]
structure Greet (Self : Type _) where
  NamedInst : Named Self
  greet : Self → Result Std.U32
-- [t05_default_supertrait::Greet::greet]:
@[trait_default]
def Greet.greet.default
  {Self : Type _} (GreetInst : Greet Self) (self : Self) : Result Std.U32 := do
  let i ← GreetInst.NamedInst.id self
  i + 1#u32
-- [t05_default_supertrait::P]
@[reducible]
def P := Unit
-- [t05_default_supertrait::{impl t05_default_supertrait::Named for t05_default_supertrait::P}::id]:
def P.Insts.T05_default_supertraitNamed.id (self : P) : Result Std.U32 := do
  ok 41#u32
-- Trait implementation: [t05_default_supertrait::{impl t05_default_supertrait::Named for
t05_default_supertrait::P}]
@[reducible]
def P.Insts.T05_default_supertraitNamed : Named P := {
  id := P.Insts.T05_default_supertraitNamed.id
}
-- Trait implementation: [t05_default_supertrait::{impl t05_default_supertrait::Greet for
t05_default_supertrait::P}]
@[reducible]
impl_def P.Insts.T05_default_supertraitGreet : Greet P := {
  NamedInst := P.Insts.T05_default_supertraitNamed
  greet := Greet.greet.default P.Insts.T05_default_supertraitGreet
}
-- [t05_default_supertrait::g]:
def g {T : Type _} (GreetInst : Greet T) (x : T) : Result Std.U32 := do
  GreetInst.greet x
end t05_default_supertrait
```

## T06_generic_struct_impl — CLEAN

```rust
pub struct Pair<A, B> { pub a: A, pub b: B }
impl<A: Copy, B: Copy> Pair<A, B> {
    pub fn swap(&self) -> Pair<B, A> { Pair { a: self.b, b: self.a } }
}
pub fn use_pair() -> u8 { Pair { a: 1u8, b: true }.swap().b }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t06_generic_struct_impl
-- [t06_generic_struct_impl::Pair]
structure Pair (A : Type _) (B : Type _) where
  a : A
  b : B
-- [t06_generic_struct_impl::{t06_generic_struct_impl::Pair<A, B>}::swap]:
def Pair.swap
  {A : Type _} {B : Type _} (coremarkerCopyInst : core.marker.Copy A)
  (coremarkerCopyInst1 : core.marker.Copy B) (self : Pair A B) :
  Result (Pair B A)
  := do
  ok { a := self.b, b := self.a }
-- [t06_generic_struct_impl::use_pair]:
def use_pair : Result Std.U8 := do
  let p ←
    Pair.swap core.marker.CopyU8 core.marker.CopyBool { a := 1#u8, b := true }
  ok p.b
end t06_generic_struct_impl
```

## T07_impl_trait_arg_ret — CLEAN

```rust
pub fn evens(n: u32) -> impl Fn(u32) -> bool { move |x| x % 2 == 0 && x < n }
pub fn apply(f: impl Fn(u32) -> u32, x: u32) -> u32 { f(x) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t07_impl_trait_arg_ret
-- [t07_impl_trait_arg_ret::evens::{closure}]
@[reducible]
def evens.closure := Std.U32
-- [t07_impl_trait_arg_ret::evens]:
def evens (n : Std.U32) : Result evens.closure := do
  ok n
-- [t07_impl_trait_arg_ret::evens::{impl core::ops::function::Fn<(u32,), bool> for
t07_impl_trait_arg_ret::evens::{closure}}::call]:
def evens.closure.Insts.CoreOpsFunctionFnTupleU32Bool.call
  (c : evens.closure) (tupled_args : Std.U32) : Result Bool := do
  let i ← tupled_args % 2#u32
  if i = 0#u32
  then ok (tupled_args < c)
  else ok false
-- [t07_impl_trait_arg_ret::apply]:
def apply
  {T0 : Type _} (coreopsfunctionFnT0TupleU32U32Inst : core.ops.function.Fn T0
  Std.U32 Std.U32) (f : T0) (x : Std.U32) :
  Result Std.U32
  := do
  coreopsfunctionFnT0TupleU32U32Inst.call f x
end t07_impl_trait_arg_ret
```

## T08_generic_trait_static — CLEAN

```rust
pub trait Zero { fn zero() -> Self; }
impl Zero for u32 { fn zero() -> u32 { 0 } }
impl Zero for u64 { fn zero() -> u64 { 0 } }
pub fn z<T: Zero>() -> T { T::zero() }
pub fn both() -> (u32, u64) { (z(), z()) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Generated Lean:

```
namespace t08_generic_trait_static
-- Trait declaration: [t08_generic_trait_static::Zero]
structure Zero (Self : Type _) where
  zero : Result Self
-- [t08_generic_trait_static::{impl t08_generic_trait_static::Zero for u32}::zero]:
def U32.Insts.T08_generic_trait_staticZero.zero : Result Std.U32 := do
  ok 0#u32
-- Trait implementation: [t08_generic_trait_static::{impl t08_generic_trait_static::Zero for u32}]
@[reducible]
def U32.Insts.T08_generic_trait_staticZero : Zero Std.U32 := {
  zero := U32.Insts.T08_generic_trait_staticZero.zero
}
-- [t08_generic_trait_static::{impl t08_generic_trait_static::Zero for u64}::zero]:
def U64.Insts.T08_generic_trait_staticZero.zero : Result Std.U64 := do
  ok 0#u64
-- Trait implementation: [t08_generic_trait_static::{impl t08_generic_trait_static::Zero for u64}]
@[reducible]
def U64.Insts.T08_generic_trait_staticZero : Zero Std.U64 := {
  zero := U64.Insts.T08_generic_trait_staticZero.zero
}
-- [t08_generic_trait_static::z]:
def z {T : Type _} (ZeroInst : Zero T) : Result T := do
  ZeroInst.zero
-- [t08_generic_trait_static::both]:
def both : Result (Std.U32 × Std.U64) := do
  let i ← z U32.Insts.T08_generic_trait_staticZero
  let i1 ← z U64.Insts.T08_generic_trait_staticZero
  ok (i, i1)
end t08_generic_trait_static
```

## U01_raw_ptr — FAIL

```
pub fn rp(x: &mut u32) -> u32 { let p = x as *mut u32; unsafe { *p = 3; *p } }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Unsupported operation: &raw mut (*x)
[Warn] Could not translate the body of function 'u01_raw_ptr::rp because of previous error
```

Generated Lean:

```
namespace u01_raw_ptr
-- [u01_raw_ptr::rp]:
def rp (x : Std.U32) : Result (Std.U32 × Std.U32) := do
  sorry
end u01_raw_ptr
```

## U02_ptr_cast — FAIL

```
pub fn pc(x: &u32) -> u8 { let p = x as *const u32 as *const u8; unsafe { *p } }
pub fn addr(x: &u32) -> usize { x as *const u32 as usize }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Unsupported operation: &raw const (*x)
[Warn] Could not translate the body of function 'u02_ptr_cast::pc because of previous error
[Warn] Could not translate the body of function 'u02_ptr_cast::addr because of previous error
```

Generated Lean:

```
namespace u02_ptr_cast
-- [u02_ptr_cast::pc]:
```

```
def pc (x : Std.U32) : Result Std.U8 := do
  sorry
-- [u02_ptr_cast::addr]:
def addr (x : Std.U32) : Result Std.Usize := do
  sorry
end u02_ptr_cast
```

### U03_union — FAIL

```
pub union U { pub a: u32, pub b: f32 }
pub fn read(u: U) -> u32 { unsafe { u.a } }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] unions are not supported
[Error] Encountered an error when analyzing the following type declarations group:
[Error] Internal error: please file an issue
[Warn] Could not translate type decl 'u03_union::U because of previous error
[Warn] Could not translate the function signature of 'u03_union::read because of previous error
[Warn] Could not translate the function 'u03_union::read because of previous error.
```

Generated Lean:

```
namespace u03_union
end u03_union
```

### U04_transmute — CAVEATS

```
pub fn t(x: u32) -> [u8; 4] { unsafe { core::mem::transmute(x) } }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace u04_transmute
-- [core::intrinsics::transmute]:
@[rust_fun "core::intrinsics::transmute"]
axiom core.intrinsics.transmute
  {Src : Type _} (Dst : Type _) : Src → Result Dst
-- [u04_transmute::t]:
def t (x : Std.U32) : Result (Array Std.U8 4#usize) := do
  core.intrinsics.transmute (Array Std.U8 4#usize) x
end u04_transmute
```

### U05_unsafe_fn_ptr_read — FAIL

```
pub unsafe fn rd(p: *const u32) -> u32 { core::ptr::read(p) }
pub fn safe_call(x: &u32) -> u32 { unsafe { rd(x) } }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Unsupported operation: &raw const (*x)
[Warn] Could not translate the body of function 'u05_unsafe_fn_ptr_read::safe_call because of previous error
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace u05_unsafe_fn_ptr_read
-- [core::ptr::read]:
@[rust_fun "core::ptr::read"]
axiom core.ptr.read {T : Type _} : ConstRawPtr T → Result T
-- [u05_unsafe_fn_ptr_read::rd]:
def rd (p : ConstRawPtr Std.U32) : Result Std.U32 := do
  core.ptr.read p
-- [u05_unsafe_fn_ptr_read::safe_call]:
def safe_call (x : Std.U32) : Result Std.U32 := do
  sorry
end u05_unsafe_fn_ptr_read
```

## U06_static_mut — FAIL

```rust
pub static K: u32 = 7;
pub const C: u32 = 3;
pub static mut G: u32 = 0;
pub fn k() -> u32 { K + C }
pub fn g() -> u32 { unsafe { G += 1; G } }
```

Exit codes: charon=0, aeneas=1, lean=0.

Aeneas messages:

```
[Error] Unexpected access to a global
[Warn] Could not translate the body of function 'u06_static_mut::g because of previous error
```

Generated Lean:

```
namespace u06_static_mut
-- [u06_static_mut::K]
@[global_simps, irreducible] def K : Std.U32 := 7#u32
-- [u06_static_mut::C]
@[global_simps, irreducible] def C : Std.U32 := 3#u32
-- [u06_static_mut::G]
@[global_simps, irreducible] def G : Std.U32 := 0#u32
-- [u06_static_mut::k]:
def k : Result Std.U32 := do
  K + C
-- [u06_static_mut::g]:
def g : Result Std.U32 := do
  sorry
end u06_static_mut
```

## X01_thread — FAIL

```rust
pub fn spawn() -> u32 { std::thread::spawn(|| 1u32).join().unwrap() }
```

Exit codes: charon=0, aeneas=0, lean=1.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Lean errors:

```
../out/X01_thread/lean/X01Thread.lean:126:0: error: declaration `x01_thread.spawn` contains universe level
metavariables at the expression
```

Generated Lean:

```
namespace x01_thread
-- [core::any::TypeId]
@[rust_type "core::any::TypeId"]
axiom core.any.TypeId : Type
-- Trait declaration: [core::any::Any]
@[rust_trait "core::any::Any"]
structure core.any.Any (Self : Type _) where
```

```
  type_id : Self → Result core.any.TypeId
-- [core::any::{impl core::fmt::Debug for (dyn core::any::Any + 'static)}::fmt]:
@[rust_fun "core::any::{core::fmt::Debug<@>}::fmt"]
axiom P.Insts.CoreFmtDebug.fmt
  :
  Dyn (fun _dyn => core.any.Any _dyn) → core.fmt.Formatter → Result
    ((core.result.Result Unit core.fmt.Error) × core.fmt.Formatter)
-- Trait implementation: [core::any::{impl core::fmt::Debug for (dyn core::any::Any + 'static)}]
@[reducible, rust_trait_impl "core::fmt::Debug<@>"]
def P.Insts.CoreFmtDebug : core.fmt.Debug (Dyn (fun _dyn => core.any.Any _dyn))
  := {
  fmt := P.Insts.CoreFmtDebug.fmt
}
-- [std::thread::join_handle::JoinHandle]
@[rust_type "std::thread::join_handle::JoinHandle"]
axiom std.thread.join_handle.JoinHandle (T : Type _) : Type
-- [std::thread::functions::spawn]:
@[rust_fun "std::thread::functions::spawn"]
axiom std.thread.functions.spawn
  {F : Type _} {T : Type _} (coreopsfunctionFnOnceFTupleTInst :
  core.ops.function.FnOnce F Unit T) :
  F → Result (std.thread.join_handle.JoinHandle T)
-- [std::thread::join_handle::{std::thread::join_handle::JoinHandle<T>}::join]:
@[rust_fun
  "std::thread::join_handle::{std::thread::join_handle::JoinHandle<@T>}::join"]
axiom std.thread.join_handle.JoinHandle.join
  {T : Type _} :
  std.thread.join_handle.JoinHandle T → Result (core.result.Result T (Dyn
    (fun _dyn => core.any.Any _dyn)))
-- [alloc::boxed::{impl core::fmt::Debug for alloc::boxed::Box<T>}::fmt]:
@[rust_fun "alloc::boxed::{core::fmt::Debug<Box<@T>>}::fmt"]
axiom Box.Insts.CoreFmtDebug.fmt
  {T : Type _} (A : Type _) (corefmtDebugInst : core.fmt.Debug T) :
  T → core.fmt.Formatter → Result ((core.result.Result Unit core.fmt.Error)
    × core.fmt.Formatter)
-- Trait implementation: [alloc::boxed::{impl core::fmt::Debug for alloc::boxed::Box<T>}]
@[reducible, rust_trait_impl "core::fmt::Debug<Box<@T>>"]
def Box.Insts.CoreFmtDebug {T : Type _} (A : Type _) (corefmtDebugInst :
-- ... (25 more lines, see experiments/out/)
```

## X02_mutex — CAVEATS

```rust
use std::sync::Mutex;
pub fn m(x: &Mutex<u32>) { *x.lock().unwrap() += 1 }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace x02_mutex
-- [std::sync::poison::mutex::Mutex]
@[rust_type "std::sync::poison::mutex::Mutex"]
axiom std.sync.poison.mutex.Mutex (T : Type _) : Type
-- [std::sync::poison::mutex::MutexGuard]
@[rust_type "std::sync::poison::mutex::MutexGuard"]
axiom std.sync.poison.mutex.MutexGuard (T : Type _) : Type
-- [std::sync::poison::PoisonError]
@[rust_type "std::sync::poison::PoisonError"]
axiom std.sync.poison.PoisonError (T : Type _) : Type
-- [std::sync::poison::mutex::{std::sync::poison::mutex::Mutex<T>}::lock]:
@[rust_fun
  "std::sync::poison::mutex::{std::sync::poison::mutex::Mutex<@T>}::lock"]
axiom std.sync.poison.mutex.Mutex.lock
  {T : Type _} :
  std.sync.poison.mutex.Mutex T → Result (core.result.Result
    (std.sync.poison.mutex.MutexGuard T) (std.sync.poison.PoisonError
    (std.sync.poison.mutex.MutexGuard T)))
-- [std::sync::poison::mutex::{impl core::ops::deref::DerefMut<T> for std::sync::poison::mutex::MutexGuard<'_0,
T>}::deref_mut]:
@[rust_fun
  "std::sync::poison::mutex::{core::ops::deref::DerefMut<std::sync::poison::mutex::MutexGuard<'0, @T>,
```

```
@T>}::deref_mut"]
axiom std.sync.poison.mutex.MutexGuard.Insts.CoreOpsDerefDerefMut.deref_mut
  {T : Type _} :
  std.sync.poison.mutex.MutexGuard T → Result (T × (T →
    std.sync.poison.mutex.MutexGuard T))
-- [std::sync::poison::{impl core::fmt::Debug for std::sync::poison::PoisonError<T>}::fmt]:
@[rust_fun
  "std::sync::poison::{core::fmt::Debug<std::sync::poison::PoisonError<@T>>}::fmt"]
axiom std.sync.poison.PoisonError.Insts.CoreFmtDebug.fmt
  {T : Type _} :
  std.sync.poison.PoisonError T → core.fmt.Formatter → Result
    ((core.result.Result Unit core.fmt.Error) × core.fmt.Formatter)
-- Trait implementation: [std::sync::poison::{impl core::fmt::Debug for std::sync::poison::PoisonError<T>}]
@[reducible, rust_trait_impl
  "core::fmt::Debug<std::sync::poison::PoisonError<@T>>"]
def std.sync.poison.PoisonError.Insts.CoreFmtDebug (T : Type _) :
  core.fmt.Debug (std.sync.poison.PoisonError T) := {
  fmt := std.sync.poison.PoisonError.Insts.CoreFmtDebug.fmt
}
-- [x02_mutex::m]:
def m (x : std.sync.poison.mutex.Mutex Std.U32) : Result Unit := do
  let r ← std.sync.poison.mutex.Mutex.lock x
  let mg ←
    core.result.Result.unwrap (std.sync.poison.PoisonError.Insts.CoreFmtDebug
      (std.sync.poison.mutex.MutexGuard Std.U32)) r
-- ... (5 more lines, see experiments/out/)
```

## X03_atomic — CAVEATS

```rust
use std::sync::atomic::{AtomicU32, Ordering};
pub fn a(x: &AtomicU32) -> u32 { x.fetch_add(1, Ordering::SeqCst) }
```

Exit codes: charon=0, aeneas=0, lean=0.

Aeneas messages:

```
[Warn] The crate contains extracted external, unknown definitions: we advise using the option -split-files to
allow manually providing these definitions in separate files.
```

Generated Lean:

```
namespace x03_atomic
-- [core::sync::atomic::private::Align4]
@[rust_type "core::sync::atomic::private::Align4"]
axiom core.sync.atomic.private.Align4 (T : Type _) : Type
-- [core::sync::atomic::Atomic]
@[rust_type "core::sync::atomic::Atomic"]
axiom core.sync.atomic.Atomic (T : Type _) (Clause0_Storage : Type _) : Type
-- [core::sync::atomic::Ordering]
@[discriminant isize, rust_type "core::sync::atomic::Ordering"]
inductive core.sync.atomic.Ordering where
| Relaxed : core.sync.atomic.Ordering
| Release : core.sync.atomic.Ordering
| Acquire : core.sync.atomic.Ordering
| AcqRel : core.sync.atomic.Ordering
| SeqCst : core.sync.atomic.Ordering
-- [core::sync::atomic::{core::sync::atomic::Atomic<u32, core::sync::atomic::private::Align4<u32>>}::fetch_add]:
@[rust_fun
  "core::sync::atomic::{core::sync::atomic::Atomic<u32, core::sync::atomic::private::Align4<u32>>}::fetch_add"]
axiom core.sync.atomic.AtomicU32Align4U32.fetch_add
  :
  core.sync.atomic.Atomic Std.U32 (core.sync.atomic.private.Align4 Std.U32) →
    Std.U32 → core.sync.atomic.Ordering → Result Std.U32
-- [x03_atomic::a]:
def a
  (x : core.sync.atomic.Atomic Std.U32 (core.sync.atomic.private.Align4
  Std.U32)) :
  Result Std.U32
  := do
  core.sync.atomic.AtomicU32Align4U32.fetch_add x 1#u32
    core.sync.atomic.Ordering.SeqCst
end x03_atomic
```

## X04_async — FAIL

```
pub async fn f(x: u32) -> u32 { x + 1 }
pub async fn g() -> u32 { f(1).await }
```

Exit codes: charon=0, aeneas=1, lean=0.

Charon:

```
warning: Coroutine types are not supported yet
warning: Coroutines are not supported
warning: The extraction generated 4 warnings
```

Aeneas messages:

```
[Error] Found type error in the output of charon
[Warn] Could not translate the function signature of 'x04_async::f because of previous error
[Warn] Could not translate the function signature of 'x04_async::g because of previous error
[Error] Internal error: please file an issue
[Warn] Could not translate the function 'x04_async::g because of previous error.
[Warn] Could not translate the function 'x04_async::f because of previous error.
```

Generated Lean:

```
namespace x04_async
end x04_async
```

# Appendix C — Loop proofs (checked with Lean 4.36.0-rc2, exit code 0)

Each file below is appended to the Aeneas-generated Lean file shown in Appendix A and checked with `lake env lean` against the Aeneas library (`experiments/proofs/*.lean`, logs `*.log`, exit codes `*.exit`).

## L04, default mode (loop combinator, loop.spec_decr_nat)

```
/-! ## Proof (hand-written): default mode, `find_loop` is defined with the `loop` combinator.
    We apply `loop.spec_decr_nat` and supply: a Nat measure, an invariant, a postcondition,
    and prove that one iteration of `find_loop.body` preserves the invariant and decreases the measure. -/
open Aeneas Aeneas.Std Result l04_early_return

def FindPost (s : Slice Std.U32) (x : Std.U32) : Option Std.Usize → Prop
  | some j => j.val < s.length ∧ s.val[j.val]! = x
  | none => ∀ k, k < s.length → s.val[k]! ≠ x

theorem find_loop_spec (s : Slice Std.U32) (x : Std.U32) (i : Std.Usize)
    (hi : i.val ≤ s.length) (hprev : ∀ k, k < i.val → s.val[k]! ≠ x) :
    find_loop s x i ⦃ r => FindPost s x r ⦄ := by
  unfold find_loop
  apply loop.spec_decr_nat
    (measure := fun (i : Std.Usize) => s.length - i.val)
    (inv := fun (i : Std.Usize) => i.val ≤ s.length ∧ ∀ k, k < i.val → s.val[k]! ≠ x)
  · rintro i' ⟨hi', hprev'⟩
    unfold find_loop.body
    simp only []
    split
    · step as ⟨ v, hv ⟩
      have hlt : i'.val < s.val.length := by scalar_tac
      split
      · simp_all [FindPost]
      · step as ⟨ i3, hi3 ⟩
        refine ⟨by scalar_tac, ?_, by scalar_tac⟩
        intro k hk
        by_cases hk' : k = i'.val
        · subst hk'; simp_all
        · exact hprev' k (by scalar_tac)
    · simp only [WP.spec_ok, FindPost]
      intro k hk; exact hprev' k (by scalar_tac)
  · exact ⟨hi, hprev⟩

theorem find_spec (s : Slice Std.U32) (x : Std.U32) :
    find s x ⦃ r => FindPost s x r ⦄ := by
  unfold find
  apply find_loop_spec <;> simp
```

## L04 with -loops-to-rec (recursion + termination_by)

```
/-! ## Proof (hand-written): `-loops-to-rec` mode, recursive `find_loop` defined by `partial_fixpoint`.
    Termination is proved *extrinsically* inside the spec via `termination_by`/`decreasing_by`. -/
open Aeneas Aeneas.Std Result l04_early_return

theorem find_loop_spec (s : Slice Std.U32) (x : Std.U32) (i : Std.Usize)
    (hi : i.val ≤ s.length) (hprev : ∀ k, k < i.val → s.val[k]! ≠ x) :
    find_loop s x i ⦃ r =>
      match r with
      | some j => j.val < s.length ∧ s.val[j.val]! = x
      | none => ∀ k, k < s.length → s.val[k]! ≠ x ⦄ := by
  unfold find_loop
  simp only []
  split
  · step as ⟨ v, hv ⟩
    have hlt : i.val < s.val.length := by scalar_tac
    split
    · simp_all
    · step as ⟨ i3, hi3 ⟩
      apply find_loop_spec
      · scalar_tac
```

```
      · intro k hk
        by_cases hk' : k = i.val
        · subst hk'; simp_all
        · exact hprev k (by scalar_tac)
  · simp only [WP.spec_ok]
    intro k hk; exact hprev k (by scalar_tac)
termination_by s.length - i.val
decreasing_by scalar_decr_tac

theorem find_spec (s : Slice Std.U32) (x : Std.U32) :
    find s x ⦃ r =>
      match r with
      | some j => j.val < s.length ∧ s.val[j.val]! = x
      | none => ∀ k, k < s.length → s.val[k]! ≠ x ⦄ := by
  unfold find
  apply find_loop_spec <;> simp
```

## L30, for-over-range, default mode

```
/-! ## Proof (hand-written): `for _ in 0..n` loop, default `loop`-combinator mode.
    The iterator state (`Range U32`) is part of the loop state; we reason about
    `IteratorRange.next` with the library's `@[step]` lemma. -/
open Aeneas Aeneas.Std Result l30_for_range_count

theorem count_range_loop_spec (n : Std.U32) (it : core.ops.range.Range Std.U32) (c : Std.U32)
    (hend : it.end = n) (hle : it.start.val ≤ n.val) (hc : c.val = it.start.val) :
    count_range_loop it c ⦃ r => r.val = n.val ⦄ := by
  unfold count_range_loop
  apply loop.spec_decr_nat
    (measure := fun (p : core.ops.range.Range Std.U32 × Std.U32) => n.val - p.1.start.val)
    (inv := fun p => p.1.end = n ∧ p.1.start.val ≤ n.val ∧ p.2.val = p.1.start.val)
  · rintro ⟨it', c'⟩ ⟨hend', hle', hc'⟩
    unfold count_range_loop.body
    step as ⟨ o, it1, hnext ⟩
    split at hnext
    · obtain ⟨⟨rfl, hst⟩, hend1⟩ := hnext
      simp only []
      step as ⟨ c1, hc1 ⟩
      refine ⟨by simp_all, by scalar_tac, by scalar_tac, by scalar_tac⟩
    · obtain ⟨⟨rfl, hst⟩, hend1⟩ := hnext
      simp only [WP.spec_ok]
      scalar_tac
  · exact ⟨hend, hle, hc⟩

theorem count_range_spec (n : Std.U32) : count_range n ⦃ r => r.val = n.val ⦄ := by
  unfold count_range
  apply count_range_loop_spec <;> simp
```